

Marathi News Intelligence Pipeline

Source code listing

1. Aim

Mini Project based on Real Life NLP Application.

2. Objectives

- To understand natural language processing and to learn how to apply basic algorithms in this field.
- To design and implement applications based on natural language processing.

Outcomes: Be able to apply NLP techniques to design real world NLP applications such as machine translation, text categorization, text summarization, information extraction, etc.

3. Hardware / Software Required

LANGUAGE	Python 3.10 or newer (the analyser, pipeline, scripts and apps); JavaScript for the in-browser demo engine.
LIBRARIES	PyTorch, Hugging Face Transformers, NumPy, pandas, scikit-learn, Streamlit, BeautifulSoup, pytest.
MODELS	MahaBERT topic / NER / sentiment / sentence-similarity models (L3Cube, Pune) and NLLB-200 distilled 600M (Meta AI).
DATASETS	XL-Sum Marathi (13,627 BBC Marathi articles with summaries), MahaNews (11,721 labelled headlines), and live RSS feeds.
HARDWARE	Any laptop. A GPU (NVIDIA CUDA or Apple Silicon) only makes it faster; developed on a 4 GB RTX 3050 and an Apple Silicon Mac.
OTHER	A web browser for the single-file demo page; Node.js only for the JavaScript/Python equivalence check.

4. Theory

Natural Language Processing (NLP) is the branch of artificial intelligence that lets computers work with human language. This project applies seven standard NLP tasks to Marathi news, a language with far fewer ready-made tools than English.

4.1 Morphological analysis

Marathi is an **agglutinative** language: grammatical information is added by joining suffixes to a root, so what English writes as several words becomes one word.

घर	house	(root)
घरात	= घर + आत	in the house
घरातून	= घर + आतून	from inside the house
घरांमधून	= घर + ां + मधून	from inside the houses
घराच्या	= घर + आ + च्या	of the house

A whitespace tokeniser treats these as five unrelated words, which harms every component that compares words. A **morphological analyser** reduces each surface form to its root. The analyser here is **rule-based**: it strips the suffix chain in four ordered layers (clitic, postposition, verb ending, case and number) and then repairs the stem. Length guards count **aksharas** (written syllables) rather than Unicode code points, because Devanagari vowel signs are separate code points. In the 3,000-article corpus, 36% of tokens carry at least one removable suffix, and normalising them reduces the index vocabulary from 186,593 to 84,480 types (54.7% smaller).

4.2 Information retrieval

BM25 is the standard ranking function for keyword search. A document scores highly when it contains the query terms often (term frequency), when those terms are rare across the collection (inverse document frequency), and it is penalised for length. **Dense retrieval** instead embeds text as vectors with a neural encoder and ranks by cosine similarity, which can match paraphrases sharing no words. **Reciprocal Rank Fusion** combines the two ranked lists by rank position rather than by score, because BM25 scores and cosine similarities are on different scales. Ranking quality is reported with **recall@k** and **MRR** (mean reciprocal rank).

4.3 Text summarisation

TextRank is an extractive, graph-based method. Sentences are nodes; edges are weighted by TF-IDF cosine similarity; **PageRank** then scores each sentence by how central it is in that graph. **Maximal Marginal Relevance (MMR)** selects the final sentences, trading relevance against redundancy so near-duplicates are not all chosen. Summaries are scored with **ROUGE**, the word overlap with human-written summaries.

4.4 Classification, extraction and sentiment

These three use **BERT**-family transformer models pre-trained on Marathi (MahaBERT, from L3Cube Pune). **Text categorisation** predicts one of 12 news topics. **Named entity recognition** is token classification: each token is labelled as part of a person, location, organisation, date and so on. **Sentiment analysis** classifies polarity; because the model is trained on short text, it is applied per sentence and aggregated into a document tone. **Fine-tuning** (training a pre-trained model further on a task's own data) raised topic accuracy from 0.847 to 0.954 on a 3-class benchmark.

4.5 Machine translation

Translation uses NLLB-200, a multilingual sequence-to-sequence transformer, applied sentence by sentence so long inputs stay within its reliable length.

4.6 Knowledge distillation

The browser demo cannot load 700 MB models, so small linear **student** models were trained to reproduce the large models' outputs over 12,262 articles. Trained on the teacher's probability distributions, they agree with it on 83.0% of topics, 75.7% of sentence sentiment labels and reach F1 0.72 on entities, while fitting in a few megabytes.

4.7 System design

The seven tasks form one pipeline rather than seven programs. The analyser runs first because retrieval, summarisation and entity counting all consume its roots; the enriched articles are then indexed for search. Every claim is measured against a simple baseline, and the negative results are reported alongside the positive ones.

5. Source code

The complete source of the project, in the order the pipeline uses it. Generated files (the built demo page, model weights, downloaded data) are not listed.

- **Configuration and dependencies**

- `config.py` — Paths, model names, device selection and batch settings.
- `requirements.txt` — Python packages the project needs.

- **Core modules — `src/`**

- `src/morph.py` — (I) Morphological analyser. Written from scratch; no training data.
- `src/models.py` — Lazy, cached loading of every transformer model.
- `src/categorize.py` — (D) Topic categorisation into 12 news topics.
- `src/summarize.py` — (E) Extractive summarisation: TextRank over a sentence graph, then MMR.
- `src/ner.py` — (G) Entity extraction, span repair and alias merging.
- `src/sentiment.py` — (F) Sentence, document and per-entity sentiment.
- `src/translate.py` — (A) Marathi to English translation with NLLB-200.
- `src/retrieve.py` — (C) BM25, dense retrieval and reciprocal rank fusion.
- `src/pipeline.py` — Runs every stage over one article.
- `src/ingest.py` — Live news ingestion from RSS feeds.
- `src/lite.py` — Small in-browser student models used by the demo page.

- **Scripts — `scripts/`**

- `scripts/download_data.py` — Downloads the XL-Sum and MahaNews datasets.
- `scripts/build_corpus.py` — Runs the pipeline over the corpus.
- `scripts/build_index.py` — Builds the BM25 and dense search indexes.
- `scripts/train_classifier.py` — Fine-tunes MahaBERT as a topic classifier.
- `scripts/evaluate.py` — All experiments, plus a Marathi-aware ROUGE implementation.
- `scripts/distill_label.py` — Labels the corpus with the full models, to train the students.
- `scripts/train_lite.py` — Trains and scores the in-browser student models.
- `scripts/export_demo.py` — Exports the demo page's data.

- `scripts/build_demo_page.py` — Assembles the single-file demo page.
- `scripts/verify_demo.py` — Checks the page's JavaScript against the Python.
- **Applications — app/**
 - `app/dashboard.py` — Streamlit dashboard.
 - `app/server.py` — Local model server the demo page can use.
 - `app/demo_engine.js` — The demo page's NLP engine: a JavaScript port of the modules above.
 - `app/demo_template.html` — The demo page's layout and interface code.
- **Tests — tests/**
 - `tests/test_pipeline.py` — 51 fast tests plus 8 that load the models.

Configuration and dependencies

config.py

68 lines

Paths, model names, device selection and batch settings.

```
1  """Central configuration for the Marathi News Intelligence Pipeline."""
2  from pathlib import Path
3  import os
4
5  ROOT = Path(__file__).resolve().parent
6  DATA = ROOT / "data"
7  RAW = DATA / "raw"
8  PROCESSED = DATA / "processed"
9  INDEX = DATA / "index"
10 OUTPUTS = ROOT / "outputs"
11 MODELS_LOCAL = ROOT / "models"
12
13 for _p in (RAW, PROCESSED, INDEX, OUTPUTS):
14     _p.mkdir(parents=True, exist_ok=True)
15
16 # ----- models -----
17 # All verified present on the HuggingFace Hub.
18 MODEL_TOPIC = "l3cube-pune/marathi-topic-all-doc" # 12-class MahaNews
19 MODEL_NER = "l3cube-pune/marathi-ner" # 8-tag flat NER
20 MODEL_SENTIMENT = "l3cube-pune/marathi-sentiment-md" # Neg/Neu/Pos
21 MODEL_SBERT = "l3cube-pune/marathi-sentence-similarity-sbert"
22 MODEL_BASE_BERT = "l3cube-pune/marathi-bert-v2" # backbone we fine-tune
23 MODEL_TRANSLATE = "facebook/nllb-200-distilled-600M" # mar_Deva -> eng_Latn
24
25 NLLB_SRC_LANG = "mar_Deva"
26 NLLB_TGT_LANG = "eng_Latn"
27
28 # Our own fine-tuned topic classifier (produced by scripts/train_classifier.py)
29 CUSTOM_TOPIC_DIR = MODELS_LOCAL / "topic-clf"
30
31 # ----- data -----
32 XLSUM_TRAIN = RAW / "marathi_train.jsonl"
33 XLSUM_VAL = RAW / "marathi_val.jsonl"
34 XLSUM_TEST = RAW / "marathi_test.jsonl"
35 MAHANews_TRAIN = RAW / "mahanews_train.parquet"
36 MAHANews_TEST = RAW / "mahanews_test.parquet"
37
38 CORPUS_JSONL = PROCESSED / "corpus.jsonl" # enriched documents
39 BM25_INDEX = INDEX / "bm25.pkl"
40 DENSE_INDEX = INDEX / "dense.npz"
41
42 # ----- runtime -----
43 import torch # noqa: E402
44 def _pick_device() -> str:
45     forced = os.environ.get("MNIP_DEVICE", "").strip().lower()
46     if forced:
47         return forced
48     if torch.cuda.is_available():
49         return "cuda"
50     # Apple Silicon GPU. Kept in fp32 (fp16 is only enabled on CUDA below).
51     if getattr(torch.backends, "mps", None) and torch.backends.mps.is_available():
52         return "mps"
53     return "cpu"
54
55
56 DEVICE = _pick_device()
57 # RTX 3050 Laptop = 4 GB VRAM; keep batches small and use fp16 on GPU.
58 BATCH_SIZE = int(os.environ.get("MNIP_BATCH", 16))
59 MAX_LEN = 256
```

```
60 FP16 = DEVICE == "cuda"
61
62 # The MTEB parquet ships integer labels with no names. This mapping was
63 # verified by joining the parquet against mexplorer008/marathi_news_
64 # classification (same 9,673 rows, but labelled with strings) on the headline
65 # text: 0->entertainment, 1->sports, 2->state, with only a handful of
66 # off-diagonal rows caused by duplicate headlines in the source data.
67 MAHANEWS_LABELS = {0: "entertainment", 1: "sports", 2: "state"}
```

Python packages the project needs.

```
1 # Core ML stack. Install torch first, matched to your CUDA version:
2 #   https://pytorch.org/get-started/locally/
3 torch>=2.2
4 transformers>=4.40
5 sentencepiece>=0.2          # NLLB / ALBERT tokenizers
6 huggingface_hub>=0.23
7
8 # Data, metrics, app
9 numpy>=1.26
10 pandas>=2.0
11 pyarrow>=15.0              # MahaNews parquet
12 scikit-learn>=1.4          # classification_report / confusion_matrix
13 streamlit>=1.30
14 beautifulsoup4>=4.12      # RSS article-body extraction
15
16 # Tests
17 pytest>=8.0
```

Core modules — src/

src/morph.py

337 lines

(I) Morphological analyser. Written from scratch; no training data.

```
1  # -*- coding: utf-8 -*-
2  """
3  Rule-based morphological analyser for Marathi (module I of the pipeline).
4
5  Marathi is agglutinative and heavily suffixing: a single nominal root takes a
6  number suffix, an oblique-stem change, and then one or more postpositions that
7  are written joined to the stem:
8
9      घर (house)
10     घरत      = घर + आत      locative      "in the house"
11     घरातून   = घर + आतून   ablative      "from inside the house"
12     घरामधून = घर + ां + मधून plural + abl. "from inside the houses"
13     घरच्या   = घर + आ(obl) + च्या genitive
14
15  A whitespace tokeniser therefore treats all four as unrelated types, which is
16  why plain keyword search performs poorly on Marathi. This module strips the
17  suffix chain, undoes the oblique stem change, and returns a normalised root
18  plus the morphological features it removed.
19
20  The analyser is deliberately rule-based rather than statistical: it needs no
21  training data, runs at roughly 100k tokens/sec, and its output is inspectable,
22  which is what makes the retrieval ablation in scripts/evaluate.py meaningful.
23
24  Design note -- why aksharas, not characters
25  -----
26  Devanagari vowel signs (matras) are separate Unicode code points, so len() is
27  not a measure of how much word is left. "घरा" is three code points but only
28  two aksharas (घ + र). Every length guard below therefore counts aksharas,
29  otherwise the minimum-stem check fires inconsistently and related forms such
30  as शाळा / शाळेत end up with different roots -- which would defeat the whole
31  purpose of the analyser.
32  """
33  from __future__ import annotations
34
35  import re
36  import unicodedata
37  from dataclasses import dataclass, field
38  from functools import lru_cache
39  from typing import List, Optional, Tuple
40
41  # -----
42  # Character classes
43  # -----
44  DEVANAGARI = r"ॠ-ॡ"
45  _TOKEN_RE = re.compile(rf"[{DEVANAGARI}]+|[A-Za-z]+|\d+")
46  _SENT_RE = re.compile(r"(?<=[\.\!\?])\s+")
47
48  VIRAMA = "ँ"
49
50  # Combining marks: anusvara/visarga block, nukta, all vowel signs, virama,
51  # accents and vocalic marks. These attach to a base akshara.
52  _MARKS = frozenset(
53      [chr(c) for c in range(0x0900, 0x0904)] # candrabindu, anusvara, visarga
54      + [chr(0x093A), chr(0x093B), chr(0x093C)] # vowel signs oe/ooe, nukta
55      + [chr(c) for c in range(0x093E, 0x0950)] # matras + virama
56      + [chr(c) for c in range(0x0951, 0x0958)] # accents
57      + [chr(0x0962), chr(0x0963)] # vocalic l matras
58  )
59
```

```

60 # Vowel signs that may be shed when normalising a stem's final syllable.
61 _FINAL_MATRAS = frozenset("िीँ")
62
63
64 def akshara_len(s: str) -> int:
65     """Number of base characters, ignoring combining vowel signs and virama."""
66     return sum(1 for ch in s if ch not in _MARKS)
67
68
69 # -----
70 # Suffix inventory.
71 #
72 # Entries are (surface_suffix, feature_tag) or (suffix, feature, replacement)
73 # where 'replacement' is re-attached to the stem after stripping -- used where
74 # removing the suffix also removes part of the root, e.g. नद्या = नदी + PL.
75 #
76 # Layers are stripped from the outside in:
77 #     clitic -> postposition -> verb -> case/number
78 # Within a layer the table is applied longest-match-first.
79 # -----
80 CLITICS = [
81     ("सुद्धा", "EMPH"), ("देखील", "EMPH"),
82     ("ही", "EMPH"), ("च", "EMPH"),
83 ]
84
85 POSTPOSITIONS = [
86     # ablative / source
87     ("पासूनच", "ABL"), ("पासून", "ABL"), ("कडून", "ABL"), ("हून", "ABL"),
88     ("ामधून", "ABL"), ("मधून", "ABL"), ("ातून", "ABL"), ("तून", "ABL"),
89     ("वरून", "ABL"), ("खालून", "ABL"),
90     # locative (incl. the adjectival -il / -la forms)
91     ("ामध्ये", "LOC"), ("मध्ये", "LOC"), ("मधे", "LOC"), ("मधील", "LOC"),
92     ("मधला", "LOC"), ("मधली", "LOC"), ("मधले", "LOC"),
93     ("ातील", "LOC"), ("तील", "LOC"), ("ातला", "LOC"), ("ातली", "LOC"),
94     ("ातले", "LOC"),
95     ("वरील", "LOC"), ("वरचा", "LOC"), ("वरची", "LOC"), ("वरचे", "LOC"),
96     ("वर", "LOC"), ("जवळ", "LOC"), ("खाली", "LOC"), ("समोर", "LOC"),
97     ("माणे", "LOC"), ("पुढे", "LOC"), ("शेजारी", "LOC"),
98     # goal / benefactive / comitative / other relational
99     ("पर्यंत", "TERM"), ("साठी", "BEN"), ("करिता", "BEN"), ("पुढे", "CAUS"),
100     ("कडील", "GEN"), ("कडे", "DAT"), ("विषयी", "ABOUT"), ("बद्दल", "ABOUT"),
101     ("प्रमाणे", "SIM"), ("सारखा", "SIM"), ("सारखी", "SIM"), ("सारखे", "SIM"),
102     ("नुसार", "SIM"), ("शिवाय", "PRIV"), ("विना", "PRIV"), ("सकट", "COM"),
103 ]
104
105 # Verbal inflection (tense / aspect / agreement / non-finite).
106 VERB_SUFFIXES = [
107     ("प्यासाठी", "INF.BEN"), ("प्यात", "INF.LOC"), ("प्याचा", "INF.GEN"),
108     ("प्याची", "INF.GEN"), ("प्याचे", "INF.GEN"),
109     ("णारा", "AGT"), ("णारी", "AGT"), ("णारे", "AGT"), ("णार", "FUT"),
110     ("णे", "INF"),
111     ("ायला", "INF.DAT"), ("ायचा", "PROSP"), ("ायची", "PROSP"),
112     ("ायचे", "PROSP"),
113     ("तात", "PRS.3P"), ("तोय", "PRS.PROG"), ("ताय", "PRS.PROG"),
114     ("तोस", "PRS.2S"), ("ते", "PRS.3SM"), ("ते", "PRS.3SN"), ("ती", "PRS.3SF"),
115     ("लेला", "PTCP"), ("लेली", "PTCP"), ("लेले", "PTCP"),
116     ("ल्या", "PST.PL"), ("ला", "PST.3SM"), ("ले", "PST.3PM"),
117     ("ली", "PST.3SF"), ("लो", "PST.1S"),
118     ("ईल", "FUT.3S"),
119     ("ून", "CONJ"), ("ऊन", "CONJ"),
120 ]
121
122 # Case + number endings written directly on the (oblique) stem.
123 CASE_NUMBER = [
124     # plural oblique + case, longest first
125     ("ानी", "PL.INS"), ("ाना", "PL.DAT"), ("ांचा", "PL.GEN"),
126     ("ांची", "PL.GEN"), ("ांचे", "PL.GEN"), ("ांच्या", "PL.GEN"),
127     ("ातून", "PL.ABL"), ("ांकडे", "PL.DAT"), ("ांत", "PL.LOC"),
128     ("ां", "PL.OBL"),

```

```

129     # singular genitive
130     ("च्या", "GEN"), ("चा", "GEN"), ("ची", "GEN"), ("चे", "GEN"),
131     # core cases
132     ("ला", "DAT"), ("ला", "DAT"), ("ने", "INS"), ("नी", "INS"),
133     ("शी", "COM"), ("त", "LOC"), ("स", "DAT"),
134     # ya-plural of i-final nouns: नद्या -> नदी
135     ("्या", "PL", "ी"),
136 ]
137
138 # Derivational suffixes are NOT part of the default chain: stripping कर off
139 # सरकार ("government") yields सर, which is a different word. They are kept
140 # here for the morphology demo but only applied when explicitly requested.
141 DERIVATIONAL = [
142     ("पणा", "NMLZ"), ("त्व", "NMLZ"), ("कर", "AGT"),
143     ("वाला", "AGT"), ("गिरी", "NMLZ"), ("दर", "AGT"),
144 ]
145
146
147 def _norm_table(table) -> List[Tuple[str, str, str]]:
148     """Pad 2-tuples with an empty replacement and sort longest-match-first."""
149     out = [(t[0], t[1], t[2] if len(t) > 2 else "") for t in table]
150     return sorted(out, key=lambda kv: -len(kv[0]))
151
152
153 _LAYERS = [
154     ("clitic", _norm_table(CLITICS)),
155     ("postp", _norm_table(POSTPOSITIONS)),
156     ("verb", _norm_table(VERB_SUFFIXES)),
157     ("case", _norm_table(CASE_NUMBER)),
158 ]
159 _DERIV_LAYER = ("deriv", _norm_table(DERIVATIONAL))
160
161 MIN_STEM = 2          # minimum aksharas that must survive a strip
162 MIN_DERIV_STEM = 3    # derivation needs a longer residue to be plausible
163 MAX_STRIP = 4         # guard against runaway peeling
164
165 # -----
166 # Marathi stopwords (function words that carry no topical signal)
167 # -----
168 STOPWORDS = set("""
169 आणि किंवा पण तर म्हणून की हे हा ही ते तो ती या यांचा यांची यांचे त्या त्याचा
170 त्याची त्याचे आहे आहेत होते होता होती असे असा अशी नाही नसून एक दोन काही सर्व
171 खूप अधिक फक्त सुद्धा देखील तसेच मात्र परंतु कारण जर तरी आता मग इथे तिथे कुठे
172 कसे काय कोण कधी अशा त्यांनी त्यांना त्यांचा आपण आम्ही तुम्ही मी माझा तुमचा
173 वर खाली मध्ये पासून पर्यंत साठी बद्दल नंतर आधी दरम्यान विरुद्ध सोबत
174 करण्यात आला आले आली गेला गेले गेली दिला दिले दिली केली केले केला
175 यात यांच्या त्यांच्या अजून पुन्हा होणार असून येथे
176 """).split())
177
178 # Frequent forms whose rule-based analysis is wrong or misleading. Kept small
179 # and explicit -- this is the escape hatch a rule system needs, not a lexicon.
180 EXCEPTIONS = {
181     "सरकार": "सरकार", "सरकारने": "सरकार", "सरकारच्या": "सरकार",
182     "सरकारला": "सरकार", "सरकारी": "सरकार",
183     "पुणे": "पुण", "पुण्यात": "पुण", "पुण्यातील": "पुण", "पुण्याच्या": "पुण",
184     "मुंबई": "मुंबई", "मुंबईत": "मुंबई", "मुंबईतील": "मुंबई",
185     "भारत": "भारत", "भारताने": "भारत", "भारतीय": "भारत", "भारतात": "भारत",
186 }
187
188
189 @dataclass
190 class Analysis:
191     """Result of analysing a single surface token."""
192     surface: str
193     root: str
194     features: List[str] = field(default_factory=list)
195     stripped: List[str] = field(default_factory=list)
196
197     @property

```

```

198     def is_inflected(self) -> bool:
199         return bool(self.striped)
200
201     def __str__(self) -> str:
202         feats = "+".join(self.features) if self.features else "-"
203         return f"{self.surface} -> {self.root} [{feats}]"
204
205
206     # -----
207     # Stem restoration
208     # -----
209     def _restore_stem(stem_form: str) -> str:
210         """
211         Normalise the stem left behind after suffix removal.
212
213         The oblique stem of a consonant-final noun ends in -ा (घर -> घरा-), and
214         plural/oblique forms surface with -े or -ी. Shedding one final vowel sign
215         -- as long as two aksharas remain -- collapses घर / घरा / घरे, and
216         शाळा / शाळे, onto a single root.
217         """
218         if not stem_form:
219             return stem_form
220
221         # A dangling virama is never a valid word end (पुण्य- from पुण्यात).
222         while stem_form.endswith(VIRAMA):
223             stem_form = stem_form[:-1]
224
225         if stem_form and stem_form[-1] in _FINAL_MATRAS and akshara_len(stem_form) >= MIN_STEM:
226             stem_form = stem_form[:-1]
227
228         return stem_form
229
230
231     def _strip_once(word: str, layers) -> Optional[Tuple[str, str, str]]:
232         """Remove at most one suffix. Returns (stem, suffix, feature) or None."""
233         for layer_name, table in layers:
234             floor = MIN_DERIV_STEM if layer_name == "deriv" else MIN_STEM
235             for suf, feat, repl in table:
236                 if not word.endswith(suf):
237                     continue
238                 candidate = word[: -len(suf)] + repl
239                 if akshara_len(candidate) >= floor:
240                     return candidate, suf, feat
241         return None
242
243
244     @lru_cache(maxsize=200_000)
245     def analyse(word: str, derivational: bool = False) -> Analysis:
246         """Analyse one token into root + morphological features."""
247         surface = word
248         word = unicodedata.normalize("NFC", word)
249
250         if surface in EXCEPTIONS:
251             return Analysis(surface=surface, root=EXCEPTIONS[surface])
252
253         layers = _LAYERS + [_DERIV_LAYER] if derivational else _LAYERS
254         feats: List[str] = []
255         strips: List[str] = []
256
257         for _ in range(MAX_STRIPS):
258             hit = _strip_once(word, layers)
259             if hit is None:
260                 break
261             new_stem, suf, feat = hit
262             word = new_stem
263             feats.append(feat)
264             strips.append(suf)
265
266         root = _restore_stem(word)

```

```

267     return Analysis(surface=surface, root=root, features=feats, stripped=strips)
268
269
270 def stem(word: str) -> str:
271     """Convenience wrapper: surface form -> normalised root."""
272     return analyse(word).root
273
274
275 # -----
276 # Tokenisation helpers used by the rest of the pipeline
277 # -----
278 def tokenize(text: str) -> List[str]:
279     """Script-aware word tokeniser (Devanagari, Latin, digits)."""
280     return _TOKEN_RE.findall(text or "")
281
282
283 def sent_tokenize(text: str) -> List[str]:
284     """Split on danda and western sentence punctuation."""
285     parts = _SENT_RE.split((text or "").strip())
286     return [p.strip() for p in parts if p and p.strip()]
287
288
289 def normalize_tokens(text: str, *, use_morph: bool = True,
290                     drop_stopwords: bool = True) -> List[str]:
291     """
292     Full normalisation path used for indexing and similarity.
293
294     `use_morph=False` reproduces the plain-tokenisation baseline, which is what
295     the retrieval ablation compares against.
296     """
297     out = []
298     for tok in tokenize(text):
299         if drop_stopwords and tok in STOPWORDS:
300             continue
301         t = stem(tok) if use_morph else tok
302         if drop_stopwords and t in STOPWORDS:
303             continue
304         if akshara_len(t) >= MIN_STEM or t.isascii():
305             out.append(t)
306     return out
307
308
309 if __name__ == "__main__":
310     import sys
311     sys.stdout.reconfigure(encoding="utf-8")
312
313     print("-- analysis --")
314     demo = [
315         "घर", "घरात", "घरातून", "घरांमधून", "घराच्या", "घरासाठी",
316         "मुलांना", "नद्या", "नदी", "शाळा", "शाळेत", "शाळेतून",
317         "पुण्यातील", "सरकारने", "खेळणारा", "जिकले", "बोलून", "मुंबईपासून",
318     ]
319     for w in demo:
320         print(" ", analyse(w))
321
322     print("\n-- conflation check (each group must share one root) --")
323     groups = [
324         ["घर", "घरात", "घरातून", "घरांमधून", "घराच्या", "घरासाठी", "घरे"],
325         ["शाळा", "शाळेत", "शाळेतून", "शाळांना"],
326         ["नदी", "नद्या", "नदीत"],
327         ["सरकार", "सरकारने", "सरकारच्या", "सरकारला"],
328         ["मुंबई", "मुंबईत", "मुंबईतील", "मुंबईपासून"],
329     ]
330     ok = True
331     for g in groups:
332         roots = {stem(w) for w in g}
333         good = len(roots) == 1
334         ok &= good

```

```
335     print(f"  [{ 'OK ' if good else 'BAD' }] {g[0]:10s} -> {sorted(roots)}")
336     print("\nall groups conflated:", ok)
```

Lazy, cached loading of every transformer model.

```

1  # -*- coding: utf-8 -*-
2  """
3  Lazy, cached model loading.
4
5  Every model in this project is loaded through here so that (a) the Streamlit
6  app and the batch scripts share one copy per process, and (b) a 4 GB laptop
7  GPU does not end up holding five models at once. Models are moved to the GPU
8  on first use and kept in fp16 there; CPU stays fp32 because fp16 matmul on CPU
9  is slower, not faster.
10 """
11 from __future__ import annotations
12
13 import functools
14 import sys
15 from pathlib import Path
16
17 import torch
18
19 sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
20 import config # noqa: E402
21
22
23 def _prepare(model):
24     model.eval()
25     if config.DEVICE == "cuda":
26         model = model.half().to("cuda")
27     elif config.DEVICE != "cpu":
28         model = model.to(config.DEVICE)
29     return model
30
31
32 @functools.lru_cache(maxsize=None)
33 def load_classifier(name: str):
34     """Sequence-classification model + tokenizer (topic, sentiment)."""
35     from transformers import AutoModelForSequenceClassification, AutoTokenizer
36     tok = AutoTokenizer.from_pretrained(name)
37     mod = _prepare(AutoModelForSequenceClassification.from_pretrained(name))
38     return tok, mod
39
40
41 @functools.lru_cache(maxsize=None)
42 def load_token_classifier(name: str):
43     """Token-classification model + tokenizer (NER)."""
44     from transformers import AutoModelForTokenClassification, AutoTokenizer
45     tok = AutoTokenizer.from_pretrained(name)
46     mod = _prepare(AutoModelForTokenClassification.from_pretrained(name))
47     return tok, mod
48
49
50 @functools.lru_cache(maxsize=None)
51 def load_encoder(name: str = config.MODEL_SBERT):
52     """Bare encoder for sentence embeddings (mean pooling applied by caller)."""
53     from transformers import AutoModel, AutoTokenizer
54     tok = AutoTokenizer.from_pretrained(name)
55     mod = _prepare(AutoModel.from_pretrained(name))
56     return tok, mod
57
58
59 @functools.lru_cache(maxsize=None)
60 def load_translator(name: str = config.MODEL_TRANSLATE):
61     """Seq2seq translation model (NLLB-200 distilled)."""
62     from transformers import AutoModelForSeq2SeqLM, AutoTokenizer
63     tok = AutoTokenizer.from_pretrained(name, src_lang=config.NLLB_SRC_LANG)
64     mod = _prepare(AutoModelForSeq2SeqLM.from_pretrained(name))

```

```

65     # NLLB ships generation_config.max_length=200, which clashes with the
66     # max_new_tokens we pass and prints a warning on every batch.
67     mod.generation_config.max_length = None
68     return tok, mod
69
70
71 def free():
72     """Drop every cached model and empty the CUDA allocator."""
73     for fn in (load_classifier, load_token_classifier, load_encoder,
74               load_translator):
75         fn.cache_clear()
76     if torch.cuda.is_available():
77         torch.cuda.empty_cache()
78     elif config.DEVICE == "mps":
79         torch.mps.empty_cache()
80
81
82 @torch.no_grad()
83 def embed(texts, name: str = config.MODEL_SBERT, batch_size: int = None,
84          show_progress: bool = False):
85     """
86     Mean-pooled, L2-normalised sentence embeddings.
87
88     Implemented directly on top of transformers rather than pulling in
89     sentence-transformers, which keeps the dependency list short and avoids a
90     version pin against transformers 5.x.
91     """
92     import numpy as np
93     tok, mod = load_encoder(name)
94     bs = batch_size or config.BATCH_SIZE
95     texts = [t if t else " " for t in texts]
96     out = []
97
98     for i in range(0, len(texts), bs):
99         chunk = texts[i:i + bs]
100         enc = tok(chunk, padding=True, truncation=True,
101                  max_length=config.MAX_LEN, return_tensors="pt").to(config.DEVICE)
102         hidden = mod(**enc).last_hidden_state # (B, T, H)
103         mask = enc["attention_mask"].unsqueeze(-1).to(hidden.dtype)
104         pooled = (hidden * mask).sum(1) / mask.sum(1).clamp(min=1e-9)
105         pooled = torch.nn.functional.normalize(pooled, p=2, dim=1)
106         out.append(pooled.float().cpu().numpy())
107         if show_progress and (i // bs) % 20 == 0:
108             print(f"    embedded {min(i + bs, len(texts))}/{len(texts)}", flush=True)
109
110     return np.vstack(out) if out else np.zeros((0, 768), dtype="float32")

```

(D) Topic categorisation into 12 news topics.

```

1  # -*- coding: utf-8 -*-
2  """
3  Topic categorisation (module D).
4
5  Two backends:
6
7  * 'l3cube-pune/marathi-topic-all-doc' -- 12-way MahaNews document classifier,
8  used out of the box so the pipeline works with no training step.
9  * a locally fine-tuned MahaBERT head produced by scripts/train_classifier.py,
10 selected with MNIP_TOPIC_MODEL=custom. It is NOT the default -- see
11 active_model() for why a better benchmark score would make the pipeline
12 worse.
13
14 Both expose the same 'classify()' signature so callers do not care which one
15 is active.
16 """
17 from __future__ import annotations
18
19 import os
20 import sys
21 from dataclasses import dataclass
22 from pathlib import Path
23 from typing import List
24
25 import torch
26
27 sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
28 import config # noqa: E402
29 from src import models # noqa: E402
30
31
32 @dataclass
33 class TopicPrediction:
34     label: str
35     score: float
36     all_scores: dict
37
38
39 def active_model() -> str:
40     """
41     The model the pipeline uses by default.
42
43     This deliberately stays on the 12-class topic model even after
44     scripts/train_classifier.py has produced a fine-tuned checkpoint. The
45     fine-tuned head is trained on MahaNews, whose label space is only
46     entertainment / sports / state -- it scores better *on that benchmark*
47     (0.954 vs 0.847) precisely because it answers an easier question. Letting
48     it take over the pipeline would collapse Politics, Tech, Health, Crime and
49     the rest into "state" and make every downstream view worse.
50
51     Set MNIP_TOPIC_MODEL=custom to use the fine-tuned head, which is what the
52     classification experiment in scripts/evaluate.py does.
53     """
54     choice = os.environ.get("MNIP_TOPIC_MODEL", "").lower()
55     if choice in ("custom", "finetuned", "local"):
56         if (config.CUSTOM_TOPIC_DIR / "config.json").exists():
57             return str(config.CUSTOM_TOPIC_DIR)
58         raise FileNotFoundError(
59             f"MNIP_TOPIC_MODEL={choice} but no checkpoint at "
60             f"{config.CUSTOM_TOPIC_DIR}. Run scripts/train_classifier.py first.")
61     return config.MODEL_TOPIC
62
63
64 def custom_model_available() -> bool:

```

```

65     return (config.CUSTOM_TOPIC_DIR / "config.json").exists()
66
67
68 @torch.no_grad()
69 def classify(texts: List[str], model_name: str = None,
70             batch_size: int = None) -> List[TopicPrediction]:
71     """Classify a batch of documents into news topics."""
72     if isinstance(texts, str):
73         texts = [texts]
74     model_name = model_name or active_model()
75     tok, mod = models.load_classifier(model_name)
76     id2label = mod.config.id2label
77     bs = batch_size or config.BATCH_SIZE
78     results: List[TopicPrediction] = []
79
80     for i in range(0, len(texts), bs):
81         chunk = [t if t else " " for t in texts[i:i + bs]]
82         enc = tok(chunk, padding=True, truncation=True,
83                  max_length=config.MAX_LEN, return_tensors="pt").to(config.DEVICE)
84         probs = mod(**enc).logits.float().softmax(-1).cpu()
85         for row in probs:
86             idx = int(row.argmax())
87             results.append(TopicPrediction(
88                 label=id2label[idx],
89                 score=float(row[idx]),
90                 all_scores={id2label[j]: float(row[j]) for j in range(len(row))},
91             ))
92     return results
93
94
95 if __name__ == "__main__":
96     sys.stdout.reconfigure(encoding="utf-8")
97     demo = [
98         "भारतीय क्रिकेट संघाने दुसऱ्या कसोटी सामन्यात इंग्लंडचा नऊ गडी राखून पराभव केला.",
99         "मुख्यमंत्र्यांनी विधानसभेत नवीन शेतकरी कर्जमाफी योजनेची घोषणा केली.",
100        "नव्या स्मार्टफोनमध्ये कृत्रिम बुद्धिमत्तेवर आधारित कॅमेरा देण्यात आला आहे.",
101    ]
102     print("model:", active_model())
103     for t, p in zip(demo, classify(demo)):
104         print(f" {p.label:15s} {p.score:.3f} | {t[:55]}")

```

(E) Extractive summarisation: TextRank over a sentence graph, then MMR.

```

1  # -*- coding: utf-8 -*-
2  """
3  Extractive summarisation (module E).
4
5  Two scorers, both of which rank sentences and then de-duplicate with MMR:
6
7  * 'textrank' -- PageRank over a sentence-similarity graph built from
8                  morphologically normalised TF-IDF vectors. No neural model,
9                  so it runs on CPU in milliseconds.
10 * 'embedding' -- the same graph but with MahaBERT-SBERT sentence vectors,
11                  which catches paraphrase that shares no surface tokens.
12
13 A 'lead' baseline (first k sentences) is included because for news it is a
14 genuinely strong baseline, and the evaluation script reports all three so the
15 comparison is honest rather than flattering.
16
17 Morphology matters here: without it, "सरकारने" and "सरकारच्या" contribute to
18 different TF-IDF dimensions and two sentences about the same subject look
19 unrelated.
20 """
21 from __future__ import annotations
22
23 import sys
24 from dataclasses import dataclass
25 from pathlib import Path
26 from typing import List
27
28 import numpy as np
29
30 sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
31 from src.morph import normalize_tokens, sent_tokenize # noqa: E402
32
33 DAMPING = 0.85
34 MAX_ITER = 100
35 TOL = 1e-6
36
37
38 @dataclass
39 class Summary:
40     sentences: List[str]
41     indices: List[int]
42     method: str
43
44     @property
45     def text(self) -> str:
46         return " ".join(self.sentences)
47
48
49 # -----
50 # Similarity graphs
51 # -----
52 def _tfidf_matrix(sentences: List[str]) -> np.ndarray:
53     """Morphologically normalised TF-IDF, built without sklearn's tokenizer."""
54     docs = [normalize_tokens(s) for s in sentences]
55     vocab = {}
56     for d in docs:
57         for t in d:
58             vocab.setdefault(t, len(vocab))
59     if not vocab:
60         return np.zeros((len(sentences), 1), dtype="float64")
61
62     # float64 so the page's JavaScript port (always float64) ranks identically.
63     tf = np.zeros((len(docs), len(vocab)), dtype="float64")
64     for i, d in enumerate(docs):

```

```

65         for t in d:
66             tf[i, vocab[t]] += 1.0
67         tf /= np.maximum(tf.sum(1, keepdims=True), 1e-9)
68
69         df = (tf > 0).sum(0)
70         idf = np.log((1 + len(docs)) / (1 + df)) + 1.0
71         return tf * idf
72
73
74     def _cosine_graph(vectors: np.ndarray) -> np.ndarray:
75         norms = np.linalg.norm(vectors, axis=1, keepdims=True)
76         unit = vectors / np.maximum(norms, 1e-9)
77         sim = unit @ unit.T
78         np.fill_diagonal(sim, 0.0)
79         return np.clip(sim, 0.0, None)
80
81
82     def _pagerank(sim: np.ndarray) -> np.ndarray:
83         """Power iteration; falls back to uniform scores on a disconnected graph."""
84         n = sim.shape[0]
85         if n == 0:
86             return np.zeros(0)
87         row = sim.sum(1, keepdims=True)
88         # Dangling rows get a uniform transition so probability mass is conserved.
89         transition = np.where(row > 0, sim / np.maximum(row, 1e-9), 1.0 / n)
90         scores = np.full(n, 1.0 / n)
91         for _ in range(MAX_ITER):
92             nxt = (1 - DAMPING) / n + DAMPING * (transition.T @ scores)
93             if np.abs(nxt - scores).sum() < TOL:
94                 return nxt
95             scores = nxt
96         return scores
97
98
99     def _mmr(order: List[int], sim: np.ndarray, k: int,
100             lambda_: float = 0.7) -> List[int]:
101         """
102         Greedily pick k sentences trading relevance against redundancy.
103
104         Relevance is fixed up front from each sentence's PageRank position, not
105         recomputed against the shrinking candidate list -- otherwise a sentence's
106         relevance rises simply because better ones were already taken, and the
107         redundancy penalty stops being comparable across iterations.
108         """
109         n = max(len(order), 1)
110         relevance = {idx: 1.0 - pos / n for pos, idx in enumerate(order)}
111
112         selected: List[int] = []
113         candidates = list(order)
114         while candidates and len(selected) < k:
115             best, best_score = None, -np.inf
116             for idx in candidates:
117                 redundancy = max((sim[idx, j] for j in selected), default=0.0)
118                 score = round(lambda_ * relevance[idx] - (1 - lambda_) * redundancy, 12)
119                 if score > best_score:
120                     best, best_score = idx, score
121             selected.append(best)
122             candidates.remove(best)
123         return selected
124
125
126     # -----
127     # Public API
128     # -----
129     def summarize(text: str, k: int = 3, method: str = "textrank",
130                 lambda_: float = 0.7) -> Summary:
131         """Return the k most representative sentences, in original order."""
132         sentences = [s for s in sent_tokenize(text) if len(s.split()) >= 3]
133         if not sentences:

```

```

134         return Summary(sentences=[], indices=[], method=method)
135     if len(sentences) <= k:
136         return Summary(sentences=sentences, indices=list(range(len(sentences))),
137                         method=method)
138
139     if method == "lead":
140         idx = list(range(k))
141         return Summary(sentences=[sentences[i] for i in idx], indices=idx,
142                         method=method)
143
144     if method == "embedding":
145         from src import models
146         vectors = models.embed(sentences)
147     else:
148         vectors = _tfidf_matrix(sentences)
149
150     sim = _cosine_graph(vectors)
151     # Scores are rounded before a stable sort so that exact ties (repeated
152     # sentences are common in news) keep document order. Without the rounding
153     # the order of a tie depends on last-bit floating-point noise, which
154     # differs between numpy and the page's JavaScript port.
155     ranked = [int(i) for i in np.argsort(-np.round(_pagerank(sim), 12), kind="stable")]
156     chosen = sorted(_mmr(ranked, sim, k, lambda_))
157     return Summary(sentences=[sentences[i] for i in chosen], indices=chosen,
158                   method=method)
159
160
161 def summarize_many(texts: List[str], k: int = 3,
162                   method: str = "textrank") -> List[Summary]:
163     return [summarize(t, k=k, method=method) for t in texts]
164
165
166 if __name__ == "__main__":
167     import json
168     sys.stdout.reconfigure(encoding="utf-8")
169     import config
170     row = json.loads(open(config.XLSUM_TEST, encoding="utf-8").readline())
171     print("TITLE  :", row["title"])
172     print("GOLD   :", row["summary"][:220])
173     for m in ("lead", "textrank"):
174         s = summarize(row["text"], k=2, method=m)
175         print(f"\n{s.upper():8s}:", s.text[:300])

```

(G) Entity extraction, span repair and alias merging.

```

1  # -*- coding: utf-8 -*-
2  """
3  Named-entity / information extraction (module G).
4
5  Uses 'l3cube-pune/marathi-ner' (MahaNER). Note that this model emits *flat*
6  labels -- Person, Location, Organization, Date, Time, Measure, Designation,
7  Other -- with no B-/I- prefixes. Consecutive wordpieces carrying the same
8  label therefore have to be merged into spans here, and because there is no
9  B- tag two adjacent distinct entities of the same type are indistinguishable;
10 we accept that and merge them, which is the standard reading of this tagset.
11
12 Spans are recovered through the tokenizer's offset mapping so the surface text
13 is taken verbatim from the input rather than rebuilt from wordpieces (which
14 would lose the original spacing and mangle Devanagari conjuncts).
15 """
16 from __future__ import annotations
17
18 import sys
19 from collections import Counter, defaultdict
20 from dataclasses import dataclass
21 from pathlib import Path
22 from typing import Dict, List
23
24 import torch
25
26 sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
27 import config # noqa: E402
28 from src import models # noqa: E402
29 from src.morph import stem as morph_stem # noqa: E402
30
31 # Types worth surfacing in the dashboard; Measure/Time/Other are kept out of
32 # the entity summary but still returned by extract().
33 HEADLINE_TYPES = ("Person", "Organization", "Location", "Date", "Designation")
34
35
36 @dataclass
37 class Entity:
38     text: str
39     label: str
40     start: int
41     end: int
42     score: float
43
44     @property
45     def canonical(self) -> str:
46         """
47         Case-stripped form used to aggregate mentions across the corpus.
48
49         Marathi declines proper nouns like any other noun -- मुंबई / मुंबईत /
50 मुंबईतील are the same place, and भारतीय जनता पक्षाच्या is the same
51 party as भारतीय जनता पक्ष. Counting surface forms would scatter one
52 entity across half a dozen rows, so the analyser (module I) is applied
53 to the final token, which is the one that carries the case suffix.
54 """
55         parts = self.text.split()
56         if not parts:
57             return self.text
58         return " ".join(parts[:-1] + [morph_stem(parts[-1])]).strip()
59
60     def __str__(self) -> str:
61         return f"{self.text} ({self.label}, {self.score:.2f})"
62
63
64 # Characters that end a word. Anything else (including Devanagari combining

```

```

65 # marks) is treated as word-internal.
66 _BOUNDARY = set("\t\n\r " + "!,,:;!?'\"()[]{}<>%/\|_---")
67
68
69 def _snap_to_word(text: str, start: int, end: int) -> tuple:
70     """
71     Widen a span to whole-word boundaries.
72
73     Wordpiece tokenisers happily split a Devanagari word between a consonant
74     and its vowel sign, so a span can begin at a combining mark -- which is how
75     पंतप्रधान came back as ंतप्रधान. Snapping outwards to the nearest
76     boundary character restores the full orthographic word.
77     """
78     while start > 0 and text[start - 1] not in _BOUNDARY:
79         start -= 1
80     while end < len(text) and text[end] not in _BOUNDARY:
81         end += 1
82     return start, end
83
84
85 @torch.no_grad()
86 def extract(text: str, max_length: int = None) -> List[Entity]:
87     """Extract entities from a single document."""
88     if not text or not text.strip():
89         return []
90
91     tok, mod = models.load_token_classifier(config.MODEL_NER)
92     id2label = mod.config.id2label
93     max_length = max_length or config.MAX_LEN
94
95     enc = tok(text, return_offsets_mapping=True, truncation=True,
96               max_length=max_length, return_tensors="pt")
97     offsets = enc.pop("offset_mapping")[0].tolist()
98     enc.pop("special_tokens_mask", None)
99     enc = {k: v.to(config.DEVICE) for k, v in enc.items()}
100
101     probs = mod(**enc).logits.float().softmax(-1)[0].cpu()
102     label_ids = probs.argmax(-1).tolist()
103     scores = probs.max(-1).values.tolist()
104
105     spans: List[Entity] = []
106     cur = None
107     for (start, end), lid, sc in zip(offsets, label_ids, scores):
108         if start == end:
109             # special token
110             continue
111         label = id2label[lid]
112         if label == "Other":
113             cur = None
114             continue
115         # Continue the current span when the label matches and the gap is only
116         # whitespace; otherwise start a new one.
117         if cur and cur.label == label and start - cur.end <= 1:
118             cur.end = end
119             cur.score = (cur.score + sc) / 2
120         else:
121             cur = Entity(text="", label=label, start=start, end=end, score=sc)
122             spans.append(cur)
123
124     for sp in spans:
125         sp.start, sp.end = _snap_to_word(text, sp.start, sp.end)
126         sp.text = text[sp.start:sp.end].strip()
127
128     # Snapping can make two adjacent spans of the same type identical; collapse
129     # runs of duplicates while preserving order.
130     deduped: List[Entity] = []
131     for sp in spans:
132         if not sp.text:
133             continue
134         if deduped and deduped[-1].text == sp.text and deduped[-1].label == sp.label:

```

```

134         continue
135     deduped.append(sp)
136     return deduped
137
138
139 def extract_many(texts: List[str]) -> List[List[Entity]]:
140     return [extract(t) for t in texts]
141
142
143 def summarize_entities(entities: List[Entity]) -> Dict[str, List[str]]:
144     """
145     Group entities by type, de-duplicated and ordered by frequency.
146
147     Mentions are *grouped* by their canonical (de-inflected) form so that
148     मुंबई and मुंबईत count as one entity, but each group is *displayed* using
149     its most frequent surface form -- de-inflection is a good equivalence key
150     and a poor label, since it turns शिंदे into शिंद.
151     """
152     counts: Dict[tuple, int] = Counter()
153     surfaces: Dict[tuple, Counter] = defaultdict(Counter)
154     for e in entities:
155         key = (e.label, e.canonical)
156         counts[key] += 1
157         surfaces[key][e.text] += 1
158
159     buckets: Dict[str, List[str]] = defaultdict(list)
160     for (label, _canon), _n in counts.most_common():
161         buckets[label].append(surfaces[(label, _canon)].most_common(1)[0][0])
162     return dict(buckets)
163
164
165 def merge_aliases(counts: Counter, min_parts: int = 2,
166                  host_ratio: float = 0.5) -> Counter:
167     """
168     Fold short-form mentions into the full name they abbreviate.
169
170     News text alternates between नरेंद्र मोदी and plain मोदी, and counting
171     them separately splits one person across two rows in the corpus-level
172     tallies. A single-token name is merged into a multi-token name that ends
173     with it, provided the long form is at least 'host_ratio' as frequent as
174     the short one. That threshold keeps a genuinely common surname from being
175     absorbed into some rare full name that happens to share it, while still
176     merging the ordinary case where both forms are used about equally often
177     (द्रप 40 / डोनाल्ड द्रप 39).
178
179     Deliberately conservative: it does not touch initials, honorifics or
180     nicknames, and it never merges two multi-token names. Real coreference
181     would need a dedicated model; this only fixes the one pattern that
182     dominates the counts.
183     """
184     merged = Counter(counts)
185     longs = [n for n in counts if len(n.split()) >= min_parts]
186     for short, n_short in list(counts.items()):
187         if len(short.split()) != 1:
188             continue
189         hosts = [L for L in longs if L.split()[-1] == short]
190         if not hosts:
191             continue
192         host = max(hosts, key=lambda L: counts[L])
193         if counts[host] >= host_ratio * n_short:
194             merged[host] += n_short
195             del merged[short]
196     return merged
197
198
199 if __name__ == "__main__":
200     sys.stdout.reconfigure(encoding="utf-8")
201     demo = ("मुख्यमंत्री एकनाथ शिंदे यांनी सोमवारी मुंबईत भारतीय जनता पक्षाच्या "
202            "बैठकीत नवीन योजनेची घोषणा केली. ही योजना १ जानेवारी २०२४ पासून ")

```

```
203         "लागू होणार आहे.")
204     ents = extract(demo)
205     for e in ents:
206         print(" ", e)
207     print("\n grouped:", summarize_entities(ents))
```

(F) Sentence, document and per-entity sentiment.

```

1  # -*- coding: utf-8 -*-
2  """
3  Sentiment analysis (module F).
4
5  'l3cube-pune/marathi-sentiment-md' is a 3-way Negative/Neutral/Positive model
6  trained on tweet-length Marathi text, so it is applied per sentence rather
7  than to a whole article; document tone is then the length-weighted mean of the
8  sentence polarities. This also gives the per-entity tone used by the
9  dashboard: the sentiment of an entity is the mean polarity of the sentences it
10 appears in.
11 """
12 from __future__ import annotations
13
14 import sys
15 from collections import defaultdict
16 from dataclasses import dataclass, field
17 from pathlib import Path
18 from typing import Dict, List
19
20 import torch
21
22 sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
23 import config # noqa: E402
24 from src import models # noqa: E402
25 from src.morph import sent_tokenize # noqa: E402
26
27 # Maps the model's classes onto a scalar polarity in [-1, 1].
28 POLARITY = {"Negative": -1.0, "Neutral": 0.0, "Positive": 1.0}
29
30
31 @dataclass
32 class SentenceSentiment:
33     text: str
34     label: str
35     score: float
36     polarity: float
37
38
39 @dataclass
40 class DocSentiment:
41     label: str
42     polarity: float
43     sentences: List[SentenceSentiment] = field(default_factory=list)
44
45     @property
46     def distribution(self) -> Dict[str, int]:
47         d = {"Negative": 0, "Neutral": 0, "Positive": 0}
48         for s in self.sentences:
49             d[s.label] = d.get(s.label, 0) + 1
50         return d
51
52
53 def _label_from_polarity(p: float) -> str:
54     if p > 0.15:
55         return "Positive"
56     if p < -0.15:
57         return "Negative"
58     return "Neutral"
59
60
61 @torch.no_grad()
62 def classify_sentences(sentences: List[str],
63                       batch_size: int = None) -> List[SentenceSentiment]:
64     """Polarity of each sentence."""

```

```

65     if not sentences:
66         return []
67     tok, mod = models.load_classifier(config.MODEL_SENTIMENT)
68     id2label = mod.config.id2label
69     bs = batch_size or config.BATCH_SIZE
70     out: List[SentenceSentiment] = []
71
72     for i in range(0, len(sentences), bs):
73         chunk = [s if s else " " for s in sentences[i:i + bs]]
74         enc = tok(chunk, padding=True, truncation=True,
75                 max_length=config.MAX_LEN, return_tensors="pt").to(config.DEVICE)
76         probs = mod(**enc).logits.float().softmax(-1).cpu()
77         for text, row in zip(chunk, probs):
78             idx = int(row.argmax())
79             label = id2label[idx]
80             # Expected polarity rather than the argmax class: keeps a
81             # 0.51/0.49 Negative/Neutral sentence from counting as strongly
82             # negative when averaged over a document.
83             pol = sum(POLARITY[id2label[j]] * float(row[j]) for j in range(len(row)))
84             out.append(SentenceSentiment(text=text, label=label,
85                                         score=float(row[idx]), polarity=pol))
86
87     return out
88
89 def analyse_document(text: str) -> DocSentiment:
90     """Length-weighted document tone built from sentence-level polarity."""
91     sents = sent_tokenize(text)
92     if not sents:
93         return DocSentiment(label="Neutral", polarity=0.0)
94     scored = classify_sentences(sents)
95     weights = [max(len(s.text), 1) for s in scored]
96     total = sum(weights)
97     pol = sum(s.polarity * w for s, w in zip(scored, weights)) / total
98     return DocSentiment(label=_label_from_polarity(pol), polarity=pol,
99                         sentences=scored)
100
101
102 def entity_sentiment(doc: DocSentiment, entities) -> Dict[str, float]:
103     """
104     Mean polarity of the sentences each entity is mentioned in.
105     """
106     This is what answers "how is politician X being covered this month" once
107     aggregated across the corpus.
108     """
109     scores: Dict[str, List[float]] = defaultdict(list)
110     for sent in doc.sentences:
111         for ent in entities:
112             if ent.text and ent.text in sent.text:
113                 scores[ent.text].append(sent.polarity)
114     return {k: sum(v) / len(v) for k, v in scores.items() if v}
115
116
117 if __name__ == "__main__":
118     sys.stdout.reconfigure(encoding="utf-8")
119     demo = ("भारतीय संघाने शानदार विजय मिळवला आणि चाहत्यांनी जल्लोष केला. "
120           "मात्र दुखापतीमुळे कर्णधार पुढील सामन्याला मुकणार आहे. "
121           "सामना मुंबईत खेळवण्यात आला.")
122     d = analyse_document(demo)
123     print(f" document: {d.label} polarity={d.polarity:+.3f} {d.distribution}")
124     for s in d.sentences:
125         print(f"      [{s.label:8s} {s.polarity:+.2f}] {s.text[:60]}")

```

(A) Marathi to English translation with NLLB-200.

```

1  # -*- coding: utf-8 -*-
2  """
3  Marathi -> English machine translation (module A).
4
5  Uses NLLB-200 (distilled, 600M). Two jobs in this pipeline:
6
7  1. Render Marathi summaries and headlines for a non-Marathi reader.
8  2. Provide the cross-lingual query path -- an English query is not
9     translated at search time; instead documents carry an English rendering
10    of their summary, which the retriever can match against. Translating the
11    corpus once is far cheaper than translating every query and gives the
12    user something readable in the results.
13
14    Long inputs are split on sentence boundaries and translated in batches, since
15    NLLB degrades badly past its training length.
16    """
17    from __future__ import annotations
18
19    import sys
20    from pathlib import Path
21    from typing import List
22
23    import torch
24
25    sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
26    import config # noqa: E402
27    from src import models # noqa: E402
28    from src.morph import sent_tokenize # noqa: E402
29
30
31    def _target_lang_id(tok) -> int:
32        """
33        Resolve the forced BOS token for the target language.
34
35        transformers moved this around across versions: 'lang_code_to_id' was
36        removed in 5.x, so fall back to a plain vocabulary lookup.
37        """
38        code = config.NLLB_TGT_LANG
39        mapping = getattr(tok, "lang_code_to_id", None)
40        if isinstance(mapping, dict) and code in mapping:
41            return mapping[code]
42        tid = tok.convert_tokens_to_ids(code)
43        if tid is None or tid == tok.unk_token_id:
44            raise ValueError(f"Could not resolve NLLB language token {code!r}")
45        return tid
46
47
48    @torch.no_grad()
49    def translate(texts, max_new_tokens: int = 256,
50                  batch_size: int = None, num_beams: int = 4) -> List[str]:
51        """Translate Marathi strings to English."""
52        single = isinstance(texts, str)
53        if single:
54            texts = [texts]
55        texts = [t if t and t.strip() else " " for t in texts]
56
57        tok, mod = models.load_translator()
58        bos = _target_lang_id(tok)
59        bs = batch_size or max(1, (config.BATCH_SIZE // 4))
60        out: List[str] = []
61
62        for i in range(0, len(texts), bs):
63            chunk = texts[i:i + bs]
64            enc = tok(chunk, padding=True, truncation=True, max_length=512,

```

```

65         return_tensors="pt").to(config.DEVICE)
66     gen = mod.generate(**enc, forced_bos_token_id=bos,
67                       max_new_tokens=max_new_tokens, num_beams=num_beams)
68     out.extend(tok.batch_decode(gen, skip_special_tokens=True))
69
70     return out[0] if single else out
71
72
73 def translate_document(text: str, max_sentences: int = 12) -> str:
74     """Sentence-split a long document, translate, and re-join."""
75     sents = sent_tokenize(text)[:max_sentences]
76     if not sents:
77         return ""
78     return " ".join(translate(sents))
79
80
81 if __name__ == "__main__":
82     sys.stdout.reconfigure(encoding="utf-8")
83     demo = [
84         "मुख्यमंत्र्यांनी विधानसभेत नवीन शेतकरी कर्जमाफी योजनेची घोषणा केली.",
85         "भारतीय क्रिकेट संघाने दुसऱ्या कसोटी सामन्यात इंग्लंडचा पराभव केला.",
86     ]
87     for src, eng in zip(demo, translate(demo)):
88         print(f" MR: {src}\n EN: {eng}\n")

```

(C) BM25, dense retrieval and reciprocal rank fusion.

```

1  # -*- coding: utf-8 -*-
2  """
3  Information retrieval (module C).
4
5  Three retrievers over the same corpus:
6
7  * BM25      -- Okapi BM25 over morphologically normalised tokens. Implemented
8                here rather than pulled from rank_bm25 so that the tokenisation
9                path (and therefore the morphology ablation) is under our
10               control and the index can be pickled with its own vocabulary.
11  * Dense     -- cosine similarity over MahaBERT-SBERT document embeddings.
12  * Hybrid    -- Reciprocal Rank Fusion of the two. RRF is used instead of a
13                weighted score sum because BM25 scores and cosine similarities
14                live on different, corpus-dependent scales.
15
16  The 'use_morph' flag on the BM25 index is the knob for the central experiment:
17  building the same index with and without the analyser and re-running the same
18  queries is what produces the retrieval numbers in the report.
19  """
20  from __future__ import annotations
21
22  import math
23  import pickle
24  import sys
25  from collections import Counter, defaultdict
26  from dataclasses import dataclass, field
27  from pathlib import Path
28  from typing import Dict, List, Optional
29
30  import numpy as np
31
32  sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
33  import config # noqa: E402
34  from src.morph import normalize_tokens # noqa: E402
35
36  K1 = 1.5
37  B = 0.75
38  RRF_K = 60
39
40
41  @dataclass
42  class Hit:
43      doc_id: int
44      score: float
45      rank: int
46      source: str = "hybrid"
47
48
49  # -----
50  # BM25
51  # -----
52
53  @dataclass
54  class BM25Index:
55      use_morph: bool = True
56      postings: Dict[str, List[tuple]] = field(default_factory=lambda: defaultdict(list))
57      doc_len: List[int] = field(default_factory=list)
58      doc_ids: List[int] = field(default_factory=list)
59      avg_len: float = 0.0
60
61      def build(self, docs: List[str], doc_ids: Optional[List[int]] = None):
62          self.postings = defaultdict(list)
63          self.doc_len = []
64          self.doc_ids = doc_ids if doc_ids is not None else list(range(len(docs)))

```

```

65     for i, text in enumerate(docs):
66         toks = normalize_tokens(text, use_morph=self.use_morph)
67         self.doc_len.append(len(toks))
68         for term, tf in Counter(toks).items():
69             self.postings[term].append((i, tf))
70
71     self.avg_len = (sum(self.doc_len) / len(self.doc_len)) if self.doc_len else 0.0
72     self.postings = dict(self.postings)
73     return self
74
75     def search(self, query: str, top_k: int = 10) -> List[Hit]:
76         n = len(self.doc_len)
77         if n == 0:
78             return []
79         q_terms = normalize_tokens(query, use_morph=self.use_morph)
80         scores = np.zeros(n, dtype="float32")
81
82         for term in q_terms:
83             posting = self.postings.get(term)
84             if not posting:
85                 continue
86             df = len(posting)
87             # BM25 idf with the +0.5 smoothing; clamped at 0 so that terms
88             # appearing in more than half the corpus cannot subtract score.
89             idf = max(math.log((n - df + 0.5) / (df + 0.5) + 1.0), 0.0)
90             for doc_i, tf in posting:
91                 denom = tf + K1 * (1 - B + B * self.doc_len[doc_i] / max(self.avg_len, 1e-9))
92                 scores[doc_i] += idf * (tf * (K1 + 1)) / max(denom, 1e-9)
93
94         top = np.argsort(-scores)[:top_k]
95         return [Hit(doc_id=self.doc_ids[i], score=float(scores[i]), rank=r, source="bm25")
96                 for r, i in enumerate(top) if scores[i] > 0]
97
98     def save(self, path: Path):
99         Path(path).write_bytes(pickle.dumps(self))
100
101     @staticmethod
102     def load(path: Path) -> "BM25Index":
103         return pickle.loads(Path(path).read_bytes())
104
105
106     # -----
107     # Dense
108     # -----
109     @dataclass
110     class DenseIndex:
111         vectors: np.ndarray = None
112         doc_ids: List[int] = field(default_factory=list)
113
114         def build(self, docs: List[str], doc_ids: Optional[List[int]] = None,
115                 show_progress: bool = True):
116             from src import models
117             self.vectors = models.embed(docs, show_progress=show_progress)
118             self.doc_ids = doc_ids if doc_ids is not None else list(range(len(docs)))
119             return self
120
121         def search(self, query: str, top_k: int = 10) -> List[Hit]:
122             from src import models
123             if self.vectors is None or len(self.vectors) == 0:
124                 return []
125             q = models.embed([query])[0]
126             sims = self.vectors @ q # both are L2-normalised
127             top = np.argsort(-sims)[:top_k]
128             return [Hit(doc_id=self.doc_ids[i], score=float(sims[i]), rank=r, source="dense")
129                     for r, i in enumerate(top)]
130
131         def save(self, path: Path):
132             np.savez_compressed(path, vectors=self.vectors,
133                               doc_ids=np.array(self.doc_ids))

```

```

134
135     @staticmethod
136     def load(path: Path) -> "DenseIndex":
137         z = np.load(path)
138         return DenseIndex(vectors=z["vectors"], doc_ids=z["doc_ids"].tolist())
139
140
141     # -----
142     # Fusion
143     # -----
144     def reciprocal_rank_fusion(runs: List[List[Hit]], top_k: int = 10) -> List[Hit]:
145         """Combine ranked lists by summing 1/(k + rank)."""
146         fused: Dict[int, float] = defaultdict(float)
147         for run in runs:
148             for hit in run:
149                 fused[hit.doc_id] += 1.0 / (RRF_K + hit.rank + 1)
150         ordered = sorted(fused.items(), key=lambda kv: -kv[1][:top_k])
151         return [Hit(doc_id=d, score=s, rank=r, source="hybrid")
152                 for r, (d, s) in enumerate(ordered)]
153
154
155     class HybridRetriever:
156         """BM25 + dense with RRF, with either half optional."""
157
158         def __init__(self, bm25: BM25Index = None, dense: DenseIndex = None):
159             self.bm25 = bm25
160             self.dense = dense
161
162         def search(self, query: str, top_k: int = 10,
163                  mode: str = "hybrid") -> List[Hit]:
164             if mode == "bm25":
165                 return self.bm25.search(query, top_k)
166             if mode == "dense":
167                 return self.dense.search(query, top_k)
168             runs = []
169             if self.bm25 is not None:
170                 runs.append(self.bm25.search(query, top_k * 3))
171             if self.dense is not None:
172                 runs.append(self.dense.search(query, top_k * 3))
173             if not runs:
174                 return []
175             return reciprocal_rank_fusion(runs, top_k)
176
177     @staticmethod
178     def load_default() -> "HybridRetriever":
179         bm25 = BM25Index.load(config.BM25_INDEX) if config.BM25_INDEX.exists() else None
180         dense = DenseIndex.load(config.DENSE_INDEX) if config.DENSE_INDEX.exists() else None
181         return HybridRetriever(bm25=bm25, dense=dense)
182
183
184     if __name__ == "__main__":
185         sys.stdout.reconfigure(encoding="utf-8")
186         docs = [
187             "पुण्यातील शाळांमध्ये नवीन शैक्षणिक धोरण लागू करण्यात आले आहे.",
188             "भारतीय क्रिकेट संघाने कसोटी सामन्यात विजय मिळवला.",
189             "मुंबईत मेट्रो प्रकल्पाचे काम वेगाने सुरू आहे.",
190             "शाळेत विद्यार्थ्यांसाठी नवीन अभ्यासक्रम सुरू झाला.",
191         ]
192         # The query says "शाळा"; documents say "शाळांमध्ये" and "शाळेत".
193         query = "शाळा"
194         for use_morph in (False, True):
195             idx = BM25Index(use_morph=use_morph).build(docs)
196             hits = idx.search(query, top_k=3)
197             tag = "morph" if use_morph else "plain"
198             print(f" [{tag}] query={query!r} -> {len(hits)} hits")
199             for h in hits:
200                 print(f"      {h.score:.3f} {docs[h.doc_id][:52]}")

```

Runs every stage over one article.

```

1  # -*- coding: utf-8 -*-
2  """
3  End-to-end document pipeline.
4
5  Takes one raw Marathi news article and returns a fully enriched record:
6
7      text -> morphology -> topic (D)
8              -> entities (G)
9              -> summary (E)
10             -> sentiment, incl. per-entity tone (F)
11             -> English rendering (A, optional)
12
13  Retrieval (C) is not per-document -- it is built over the enriched corpus by
14  scripts/build_index.py.
15
16  Translation is opt-in because NLLB-600M is by far the heaviest model here and
17  most batch runs do not need it.
18  """
19  from __future__ import annotations
20
21  import sys
22  from dataclasses import asdict, dataclass, field
23  from pathlib import Path
24  from typing import Any, Dict, List, Optional
25
26  sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
27  from src import categorize, ner, sentiment, summarize # noqa: E402
28  from src.morph import analyse, tokenize # noqa: E402
29
30
31  @dataclass
32  class Document:
33      """One fully analysed news article."""
34      id: Optional[str] = None
35      title: str = ""
36      text: str = ""
37      url: str = ""
38
39      topic: str = ""
40      topic_score: float = 0.0
41      topic_scores: Dict[str, float] = field(default_factory=dict)
42
43      summary: str = ""
44      summary_sentences: List[str] = field(default_factory=list)
45
46      entities: Dict[str, List[str]] = field(default_factory=dict)
47      entity_sentiment: Dict[str, float] = field(default_factory=dict)
48
49      sentiment: str = ""
50      polarity: float = 0.0
51      sentiment_distribution: Dict[str, int] = field(default_factory=dict)
52
53      summary_en: str = ""
54      title_en: str = ""
55
56      tokens: int = 0
57      inflected_ratio: float = 0.0
58
59      def to_dict(self) -> Dict[str, Any]:
60          return asdict(self)
61
62      @property
63      def search_text(self) -> str:
64          """Field actually indexed: title is repeated to weight it up."""

```

```

65         return f"{self.title} {self.title} {self.text}"
66
67
68 def _morph_stats(text: str) -> tuple:
69     toks = tokenize(text)
70     if not toks:
71         return 0, 0.0
72     inflected = sum(1 for t in toks if analyse(t).is_inflected)
73     return len(toks), inflected / len(toks)
74
75
76 def process(title: str, text: str, *, url: str = "", doc_id: str = None,
77             summary_k: int = 3, summary_method: str = "textrank",
78             translate: bool = False) -> Document:
79     """Run every module over a single article."""
80     doc = Document(id=doc_id, title=title or "", text=text or "", url=url)
81
82     # --- D: topic -----
83     clf_input = f"{doc.title}. {doc.text}" if doc.title else doc.text
84     pred = categorize.classify([clf_input])[0]
85     doc.topic, doc.topic_score, doc.topic_scores = pred.label, pred.score, pred.all_scores
86
87     # --- E: summary -----
88     summ = summarize.summarize(doc.text, k=summary_k, method=summary_method)
89     doc.summary, doc.summary_sentences = summ.text, summ.sentences
90
91     # --- G: entities -----
92     # NER is run over title + summary rather than the full article: the model
93     # truncates at 256 tokens anyway, and the summary is a denser carrier of
94     # the article's principal entities than an arbitrary 256-token prefix.
95     ent_source = f"{doc.title}. {doc.summary}".strip()
96     ents = ner.extract(ent_source)
97     doc.entities = ner.summarize_entities(ents)
98
99     # --- F: sentiment -----
100    sent_doc = sentiment.analyse_document(doc.summary or doc.text)
101    doc.sentiment = sent_doc.label
102    doc.polarity = sent_doc.polarity
103    doc.sentiment_distribution = sent_doc.distribution
104    doc.entity_sentiment = sentiment.entity_sentiment(sent_doc, ents)
105
106    # --- A: translation (optional) -----
107    if translate:
108        from src import translate as mt
109        doc.title_en = mt.translate(doc.title) if doc.title else ""
110        doc.summary_en = mt.translate_document(doc.summary) if doc.summary else ""
111
112    # --- I: morphology statistics -----
113    doc.tokens, doc.inflected_ratio = _morph_stats(doc.text)
114    return doc
115
116
117 def process_many(rows: List[Dict[str, str]], **kwargs) -> List[Document]:
118     """Process a list of {title, text, url, id} dicts."""
119     out = []
120     for r in rows:
121         out.append(process(r.get("title", ""), r.get("text", ""),
122                           url=r.get("url", ""), doc_id=r.get("id"), **kwargs))
123     return out
124
125
126 if __name__ == "__main__":
127     import json
128     import config
129     sys.stdout.reconfigure(encoding="utf-8")
130
131     row = json.loads(open(config.XLSUM_TEST, encoding="utf-8").readline())
132     doc = process(row["title"], row["text"], url=row["url"], doc_id=row["id"])
133

```

```
134 print("TITLE      :", doc.title)
135 print("TOPIC      :", f"{doc.topic} ({doc.topic_score:.3f})")
136 print("SENTIMENT  :", f"{doc.sentiment} ({doc.polarity:+.3f})", doc.sentiment_distribution)
137 print("TOKENS     :", doc.tokens, f"| inflected {doc.inflected_ratio:.1%}")
138 print("ENTITIES   :")
139 for k, v in doc.entities.items():
140     print(f"      {k:14s} {', '.join(v[:6])}")
141 print("SUMMARY    :", doc.summary[:300])
142 print("\nGOLD       :", row["summary"][:200])
```

Live news ingestion from RSS feeds.

```

1  # -*- coding: utf-8 -*-
2  """
3  Live Marathi news ingestion over RSS.
4
5  Lets the pipeline run on today's news instead of only the static XL-Sum
6  corpus, which is what makes the dashboard demo feel real.
7
8  RSS items carry only a headline and a one-line description, so 'fetch_all'
9  can optionally follow each link and pull the article body out of the page.
10 That part is best-effort by design: every outlet has a different template, and
11 a failed body fetch degrades to the RSS description rather than dropping the
12 article.
13
14 Only public RSS endpoints are read, one request at a time with a short delay,
15 and nothing is sent anywhere.
16 """
17 from __future__ import annotations
18
19 import html
20 import re
21 import sys
22 import time
23 import urllib.request
24 import xml.etree.ElementTree as ET
25 from dataclasses import dataclass
26 from pathlib import Path
27 from typing import Dict, List, Optional
28
29 sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
30
31 # Verified reachable and returning Devanagari RSS.
32 FEEDS: Dict[str, str] = {
33     "BBC Marathi": "https://feeds.bbc.co.uk/marathi/rss.xml",
34     "ABP Majha": "https://marathi.abplive.com/home/feed",
35     "Lokmat": "https://www.lokmat.com/rss/maharashtra.xml",
36 }
37
38 USER_AGENT = "Mozilla/5.0 (compatible; MarathiNewsPipeline/1.0; academic research)"
39 TIMEOUT = 20
40 POLITE_DELAY = 0.5
41
42 _TAG_RE = re.compile(r"<[^\>]+>")
43 _WS_RE = re.compile(r"\s+")
44
45
46 @dataclass
47 class Article:
48     id: str
49     title: str
50     text: str
51     url: str
52     source: str
53     published: str = ""
54
55
56 def _clean(raw: str) -> str:
57     """Strip tags and entities out of an RSS description or HTML fragment."""
58     if not raw:
59         return ""
60     txt = _TAG_RE.sub(" ", raw)
61     txt = html.unescape(txt)
62     return _WS_RE.sub(" ", txt).strip()
63
64

```

```

65 def _get(url: str) -> bytes:
66     req = urllib.request.Request(url, headers={"User-Agent": USER_AGENT})
67     with urllib.request.urlopen(req, timeout=TIMEOUT) as resp:
68         return resp.read()
69
70
71 def _text_of(item, *names) -> str:
72     """First non-empty child among 'names', namespace-insensitive."""
73     for name in names:
74         for child in item:
75             tag = child.tag.split("}")[1].lower()
76             if tag == name.lower() and (child.text or "").strip():
77                 return child.text.strip()
78     return ""
79
80
81 def fetch_feed(name: str, url: str, limit: int = 25) -> List[Article]:
82     """Parse one RSS feed into Articles (headline + description only)."""
83     try:
84         root = ET.fromstring(_get(url))
85     except Exception as exc: # noqa: BLE001
86         print(f"! {name}: {exc}")
87         return []
88
89     out: List[Article] = []
90     # Handles both RSS <item> and Atom <entry>.
91     items = root.iter()
92     for node in items:
93         tag = node.tag.split("}")[1].lower()
94         if tag not in ("item", "entry"):
95             continue
96         title = _clean(_text_of(node, "title"))
97         desc = _clean(_text_of(node, "description", "summary", "content"))
98         link = _text_of(node, "link", "guid")
99         pub = _text_of(node, "pubDate", "published", "updated")
100         if not title:
101             continue
102         out.append(Article(id=f"{name}:{len(out)}", title=title, text=desc,
103                             url=link, source=name, published=pub))
104         if len(out) >= limit:
105             break
106     return out
107
108
109 def fetch_body(url: str) -> Optional[str]:
110     """Best-effort article body extraction. Returns None if nothing usable."""
111     try:
112         from bs4 import BeautifulSoup
113         soup = BeautifulSoup(_get(url), "html.parser")
114     except Exception: # noqa: BLE001
115         return None
116
117     for bad in soup(["script", "style", "nav", "header", "footer", "aside"]):
118         bad.decompose()
119
120     # Longest run of <p> text on the page is almost always the article.
121     paras = [p.get_text(" ", strip=True) for p in soup.find_all("p")]
122     paras = [p for p in paras if len(p) > 40]
123     body = " ".join(paras)
124     body = _WS_RE.sub(" ", body).strip()
125     return body if len(body) > 200 else None
126
127
128 def fetch_all(limit: int = 60, with_body: bool = True,
129               feeds: Dict[str, str] = None) -> List[Dict[str, str]]:
130     """
131     Pull headlines from every feed, optionally enriching with article bodies.
132
133     Returns plain dicts shaped like the XL-Sum rows so build_corpus.py can

```

```

134     treat both sources identically.
135     """
136     feeds = feeds or FEEDS
137     per_feed = max(1, limit // max(len(feeds), 1))
138     articles: List[Article] = []
139
140     for name, url in feeds.items():
141         got = fetch_feed(name, url, limit=per_feed)
142         print(f" {name:14s} {len(got):3d} items")
143         articles.extend(got)
144         time.sleep(POLITE_DELAY)
145
146     if with_body:
147         for a in articles:
148             if not a.url:
149                 continue
150             body = fetch_body(a.url)
151             if body:
152                 a.text = body
153             time.sleep(POLITE_DELAY)
154
155     return [{"id": a.id, "title": a.title, "text": a.text or a.title,
156            "url": a.url, "source": a.source, "published": a.published}
157            for a in articles]
158
159
160 if __name__ == "__main__":
161     sys.stdout.reconfigure(encoding="utf-8")
162     rows = fetch_all(limit=9, with_body=True)
163     print(f"\n fetched {len(rows)} articles")
164     for r in rows[:5]:
165         print(f"\n [{r['source']}] {r['title'][:70]}")
166         print(f"      {len(r['text'])} chars | {r['text'][:110]}")

```

Small in-browser student models used by the demo page.

```

1  # -*- coding: utf-8 -*-
2  """
3  Lightweight students of the MahaBERT models, small enough to run in a browser.
4
5  The demo page is a single HTML file, so it cannot load the 12-class topic
6  model, MahaNER or the sentiment model (each is a ~700 MB BERT). Instead it
7  carries three small linear models trained by scripts/train_lite.py to
8  reproduce what those models say on XL-Sum:
9
10     * topic      -- softmax regression over sublinear TF-IDF of analyser roots
11     * sentiment  -- softmax regression over words, roots and word bigrams
12     * entities   -- greedy left-to-right word tagger (a maximum-entropy Markov
13                   model) over word, root, suffix, neighbour and previous-label
14                   features
15
16  How often each agrees with the model it imitates is measured on the held-out
17  XL-Sum test split and shipped with the weights, so the page can say how far
18  to trust it.
19
20  This module is the reference implementation: the page's JavaScript is a line
21  by line port of the feature functions here, and scripts/verify_demo.py checks
22  that the two produce identical outputs.
23  """
24  from __future__ import annotations
25
26  import base64
27  import json
28  import math
29  import re
30  import sys
31  from collections import Counter, defaultdict
32  from pathlib import Path
33  from typing import Dict, List, Tuple
34
35  sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
36  import config # noqa: E402
37  from src.morph import (STOPWORDS, normalize_tokens, sent_tokenize, # noqa: E402
38                        stem, tokenize)
39
40  LITE_MODELS = config.ROOT / "app" / "lite_models.json"
41
42  NER_LABELS = ["O", "Person", "Location", "Organization", "Date", "Time",
43               "Measure", "Designation"]
44  POLARITY = {"Negative": -1.0, "Neutral": 0.0, "Positive": 1.0}
45
46  # Same boundary set as src/ner.py: a "word" for the tagger is a maximal run
47  # of characters not in this set, which is exactly what MahaNER spans snap to.
48  BOUNDARY = set("\t\n\r " + ".,:;!?'\"()[]{}<>%/\\" + "--")
49
50  # How much of an article the topic student reads. The teacher truncates at
51  # 256 wordpieces, so reading the whole article would teach the student to use
52  # evidence the teacher never saw.
53  TOPIC_MAX_TOKENS = 200
54
55  # -----
56  # Features
57  # -----
58
59  def topic_terms(title: str, text: str) -> List[str]:
60     joined = f"{title}. {text}" if title else text
61     return normalize_tokens(joined)[:TOPIC_MAX_TOKENS]
62
63
64  def sentiment_features(sentence: str) -> List[str]:

```

```

65     toks = tokenize(sentence)
66     feats = []
67     for t in toks:
68         feats.append("w:" + t)
69         r = stem(t)
70         if r != t:
71             feats.append("r:" + r)
72     for a, b in zip(toks, toks[1:]):
73         feats.append("b:" + a + "_" + b)
74     return feats
75
76
77 def words(text: str) -> List[Tuple[int, int]]:
78     """(start, end) offsets of every word, splitting on BOUNDARY characters."""
79     out, start = [], None
80     for i, ch in enumerate(text):
81         if ch in BOUNDARY:
82             if start is not None:
83                 out.append((start, i))
84                 start = None
85             elif start is None:
86                 start = i
87     if start is not None:
88         out.append((start, len(text)))
89     return out
90
91
92 _DIGITS_RE = re.compile(r"[0-9\u0966-\u096F]+")
93 _ASCII_RE = re.compile(r"[\x00-\x7F]+")
94
95
96 def _shape(w: str) -> str:
97     # Explicit character classes rather than str.isdigit()/isascii(), whose
98     # Unicode rules differ from JavaScript's and would break parity.
99     if _DIGITS_RE.fullmatch(w):
100         return "digit"
101     if _ASCII_RE.fullmatch(w):
102         return "latin"
103     return "deva"
104
105
106 def ner_features(toks: List[str], i: int, prev: str, prev2: str) -> List[str]:
107     """Features for word i; prev/prev2 are the labels already assigned."""
108     w = toks[i]
109     r = stem(w)
110     p = toks[i - 1] if i > 0 else "<s>"
111     n = toks[i + 1] if i + 1 < len(toks) else "</s>"
112     n2 = toks[i + 2] if i + 2 < len(toks) else "</s>"
113     feats = [
114         "bias:", "w:" + w, "r:" + r, "s1:" + w[-1:], "s2:" + w[-2:], "s3:" + w[-3:],
115         "p3:" + w[:3], "sh:" + _shape(w), "sw:" + str(w in STOPWORDS),
116         "pw:" + p, "pr:" + stem(p), "nw:" + n, "nr:" + stem(n), "nw2:" + n2,
117         "pl:" + prev, "pl2:" + prev + "|" + prev2, "plw:" + prev + "|" + r,
118         "pwn:" + p + "|" + w, "wn:" + w + "|" + n,
119     ]
120     if i == 0:
121         feats.append("first")
122     return feats
123
124
125 # -----
126 # Inference
127 # -----
128 def _softmax(z: List[float]) -> List[float]:
129     m = max(z)
130     e = [math.exp(v - m) for v in z]
131     s = sum(e)
132     return [v / s for v in e]
133

```

```

134
135 def unpack(b64: str, cols: int) -> List[List[int]]:
136     """Rows of int8 weights from the base64 blob written by train_lite.py."""
137     raw = base64.b64decode(b64)
138     vals = [v - 256 if v > 127 else v for v in raw]
139     return [vals[i:i + cols] for i in range(0, len(vals), cols)]
140
141
142 class Lite:
143     """Loaded student models. Weights are int8 values times one scale."""
144
145     def __init__(self, data: dict):
146         self.data = data
147         t = data["topic"]
148         self.topic_labels = t["labels"]
149         self.topic_idf = dict(zip(t["terms"], t["idf"]))
150         self.topic_w = dict(zip(t["terms"], unpack(t["W"], len(t["labels"]))))
151         self.topic_b = t["b"]
152         self.topic_scale = t["scale"]
153
154         s = data["sentiment"]
155         self.sent_labels = s["labels"]
156         self.sent_w = dict(zip(s["feats"], unpack(s["W"], len(s["labels"]))))
157         self.sent_b = s["b"]
158         self.sent_scale = s["scale"]
159
160         n = data["ner"]
161         self.ner_labels = n["labels"]
162         self.ner_w = dict(zip(n["feats"], unpack(n["W"], len(n["labels"]))))
163         self.ner_b = n["b"]
164         self.ner_scale = n["scale"]
165
166     @classmethod
167     def load(cls, path: Path = LITE_MODELS) -> "Lite":
168         return cls(json.loads(Path(path).read_text(encoding="utf-8")))
169
170     # ---- topic -----
171     def topic(self, title: str, text: str) -> Dict[str, float]:
172         tf = Counter(t for t in topic_terms(title, text) if t in self.topic_w)
173         vec = {t: (1 + math.log(c)) * self.topic_idf[t] for t, c in tf.items()}
174         norm = math.sqrt(sum(v * v for v in vec.values())) or 1.0
175         k = len(self.topic_labels)
176         z = [0.0] * k
177         for t, v in vec.items():
178             row = self.topic_w[t]
179             for j in range(k):
180                 z[j] += row[j] * v / norm
181         z = [z[j] * self.topic_scale + self.topic_b[j] for j in range(k)]
182         return dict(zip(self.topic_labels, _softmax(z)))
183
184     # ---- sentiment -----
185     def sentence_sentiment(self, sentence: str) -> Dict[str, float]:
186         feats = [f for f in set(sentiment_features(sentence)) if f in self.sent_w]
187         k = len(self.sent_labels)
188         z = [0.0] * k
189         inv = 1.0 / math.sqrt(len(feats)) if feats else 0.0
190         for f in sorted(feats):
191             row = self.sent_w[f]
192             for j in range(k):
193                 z[j] += row[j] * inv
194         z = [z[j] * self.sent_scale + self.sent_b[j] for j in range(k)]
195         return dict(zip(self.sent_labels, _softmax(z)))
196
197     def document_sentiment(self, text: str) -> dict:
198         """Same aggregation as src/sentiment.analyse_document."""
199         sents = sent_tokenize(text)
200         if not sents:
201             return {"label": "Neutral", "polarity": 0.0,
202                     "distribution": {"Negative": 0, "Neutral": 0, "Positive": 0},

```

```

203         "sentences": []}
204     scored = []
205     for s in sents:
206         p = self.sentence_sentiment(s)
207         label = max(p, key=p.get)
208         pol = sum(POLARITY[k] * v for k, v in p.items())
209         scored.append({"text": s, "label": label, "polarity": pol})
210     weights = [max(len(s["text"]), 1) for s in scored]
211     pol = sum(s["polarity"] * w for s, w in zip(scored, weights)) / sum(weights)
212     label = "Positive" if pol > 0.15 else "Negative" if pol < -0.15 else "Neutral"
213     dist = {"Negative": 0, "Neutral": 0, "Positive": 0}
214     for s in scored:
215         dist[s["label"]] += 1
216     return {"label": label, "polarity": pol, "distribution": dist,
217           "sentences": scored}
218
219     # ---- entities -----
220     def tag(self, text: str) -> List[Tuple[int, int, str]]:
221         """Word-level labels, as (start, end, label) for every word."""
222         offs = words(text)
223         toks = [text[s:e] for s, e in offs]
224         k = len(self.ner_labels)
225         out, prev, prev2 = [], "0", "0"
226         for i in range(len(toks)):
227             z = list(self.ner_b)
228             for f in ner_features(toks, i, prev, prev2):
229                 row = self.ner_w.get(f)
230                 if row is not None:
231                     for j in range(k):
232                         z[j] += row[j] * self.ner_scale
233             best = max(range(k), key=lambda j: z[j])
234             lab = self.ner_labels[best]
235             out.append((offs[i][0], offs[i][1], lab))
236             prev2, prev = prev, lab
237         return out
238
239     def entities(self, text: str) -> List[Tuple[int, int, str]]:
240         """Merge adjacent same-label words separated by one character."""
241         spans: List[list] = []
242         for s, e, lab in self.tag(text):
243             if lab == "0":
244                 continue
245             if spans and spans[-1][2] == lab and s - spans[-1][1] <= 1:
246                 spans[-1][1] = e
247             else:
248                 spans.append([s, e, lab])
249         return [tuple(x) for x in spans]
250
251     def group_entities(text: str, spans) -> Dict[str, List[str]]:
252         """Same grouping as src/ner.summarize_entities: root key, surface label."""
253         counts: Counter = Counter()
254         surfaces: Dict[tuple, Counter] = defaultdict(Counter)
255         for s, e, lab in spans:
256             surf = text[s:e].strip()
257             if not surf:
258                 continue
259             parts = surf.split()
260             canon = " ".join(parts[:-1] + [stem(parts[-1])])
261             counts[(lab, canon)] += 1
262             surfaces[(lab, canon)][surf] += 1
263         out: Dict[str, List[str]] = defaultdict(list)
264         for (lab, canon), _ in counts.most_common():
265             out[lab].append(surfaces[(lab, canon)].most_common(1)[0][0])
266         return dict(out)
267
268
269     def analyse(model: "Lite", title: str, text: str) -> dict:
270         """
271

```

```
272 The whole in-browser pipeline, mirroring src/pipeline.process with the
273 students in place of the MahaBERT models. The page's JavaScript pipeline()
274 must return the same thing; scripts/verify_demo.py checks that it does.
275 """
276 from src.summarize import summarize
277 title, text = (title or "").strip(), (text or "").strip()
278 summ = summarize(text, k=3)
279 summary = summ.text
280 ent_source = f"{title}. {summary}".strip()
281 sent = model.document_sentiment(summary or text)
282 return {
283     "topic_scores": model.topic(title, text),
284     "summary": summary,
285     "entities": group_entities(ent_source, model.entities(ent_source)),
286     "sentiment": sent["label"],
287     "polarity": sent["polarity"],
288     "sentiment_distribution": sent["distribution"],
289 }
```

Scripts — scripts/

scripts/download_data.py

84 lines

Downloads the XL-Sum and MahaNews datasets.

```
1  # -*- coding: utf-8 -*-
2  """
3  Fetch the two public corpora this project is built on.
4
5  python scripts/download_data.py
6
7  * XL-Sum (Marathi) -- 10,903 / 1,362 / 1,362 BBC Marathi articles with
8  human-written abstractive summaries. Used as the document corpus and as
9  the gold standard for the summarisation evaluation.
10  https://huggingface.co/datasets/csebuetnlp/xlsum
11
12  * MahaNews (headline split, via MTEB) -- 9,673 / 2,048 Marathi headlines
13  labelled entertainment / sports / state. Used to fine-tune the topic
14  classifier and to score it.
15  https://huggingface.co/datasets/mteb/MarathiNewsClassification
16
17  Both are downloaded straight over HTTPS rather than through `datasets`, which
18  keeps the dependency list short and means no Hub authentication is needed.
19  Nothing here is uploaded; the requests are read-only downloads.
20  """
21  from __future__ import annotations
22
23  import sys
24  import tarfile
25  import urllib.request
26  from pathlib import Path
27
28  sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
29  sys.stdout.reconfigure(encoding="utf-8")
30
31  import config # noqa: E402
32
33  XLSUM_URL = ("https://huggingface.co/datasets/csebuetnlp/xlsum/resolve/main/"
34              "data/marathi_XLSum_v2.0.tar.bz2")
35  MAHANEWS = {
36      "mahanews_train.parquet": ("https://huggingface.co/datasets/mteb/"
37                               "MarathiNewsClassification/resolve/main/"
38                               "data/train-00000-of-00001.parquet"),
39      "mahanews_test.parquet": ("https://huggingface.co/datasets/mteb/"
40                               "MarathiNewsClassification/resolve/main/"
41                               "data/test-00000-of-00001.parquet"),
42  }
43
44
45  def fetch(url: str, dest: Path):
46      if dest.exists() and dest.stat().st_size > 0:
47          print(f" = {dest.name} (already present, {dest.stat().st_size:,} bytes)")
48          return
49      print(f" > {dest.name} ...", end="", flush=True)
50      urllib.request.urlretrieve(url, dest)
51      print(f" {dest.stat().st_size:,} bytes")
52
53
54  def main():
55      print("XL-Sum (Marathi)")
56      archive = config.RAW / "marathi_xlsum.tar.bz2"
57      fetch(XLSUM_URL, archive)
58      if not config.XLSUM_TRAIN.exists():
59          with tarfile.open(archive) as tf:
```

```

60     # Flatten: the archive stores ./marathi_{split}.jsonl
61     for member in tf.getmembers():
62         if member.isfile() and member.name.endswith(".jsonl"):
63             member.name = Path(member.name).name
64             # filter="data" blocks path traversal; older Pythons
65             # without the argument fall back to plain extract.
66             try:
67                 tf.extract(member, config.RAW, filter="data")
68             except TypeError:
69                 tf.extract(member, config.RAW)
70         print("    extracted train/val/test")
71
72     print("\nMahaNews (headline classification)")
73     for name, url in MAHANEWS.items():
74         fetch(url, config.RAW / name)
75
76     print("\nReady. Next:")
77     print("    python scripts/build_corpus.py --limit 3000")
78     print("    python scripts/build_index.py --plain-bm25")
79     print("    streamlit run app/dashboard.py")
80
81
82 if __name__ == "__main__":
83     main()

```

Runs the pipeline over the corpus.

```

1  # -*- coding: utf-8 -*-
2  """
3  Enrich raw articles into the analysed corpus that the index and dashboard use.
4
5  Runs the whole pipeline over XL-Sum Marathi (or a live RSS pull) and writes one
6  JSON object per line to data/processed/corpus.jsonl.
7
8  The stages are deliberately run *stage-by-stage over the whole batch* rather
9  than document-by-document: the GPU is only worth using when it gets a full
10 batch, and this way each model is resident once instead of being swapped in
11 and out per article.
12
13     python scripts/build_corpus.py --limit 1500
14     python scripts/build_corpus.py --source rss --translate
15 """
16 from __future__ import annotations
17
18 import argparse
19 import json
20 import sys
21 import time
22 from pathlib import Path
23
24 sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
25 sys.stdout.reconfigure(encoding="utf-8")
26
27 import config                                # noqa: E402
28 from src import categorize, ner, sentiment, summarize # noqa: E402
29 from src.morph import analyse, tokenize        # noqa: E402
30 from src.pipeline import Document              # noqa: E402
31
32
33 def load_xlsum(limit: int, split: str = "train"):
34     path = {"train": config.XLSUM_TRAIN, "val": config.XLSUM_VAL,
35            "test": config.XLSUM_TEST}[split]
36     rows = []
37     with open(path, encoding="utf-8") as fh:
38         for line in fh:
39             if limit and len(rows) >= limit:
40                 break
41             r = json.loads(line)
42             if len(r.get("text", "")) < 200:      # skip stubs
43                 continue
44             rows.append({"id": r["id"], "title": r["title"], "text": r["text"],
45                        "url": r.get("url", ""), "gold_summary": r.get("summary", "")})
46     return rows
47
48
49 def load_rss(limit: int):
50     from src.ingest import fetch_all
51     return fetch_all(limit=limit)
52
53
54 def morph_stats(text: str):
55     toks = tokenize(text)
56     if not toks:
57         return 0, 0.0
58     return len(toks), sum(1 for t in toks if analyse(t).is_inflected) / len(toks)
59
60
61 def main():
62     ap = argparse.ArgumentParser()
63     ap.add_argument("--limit", type=int, default=1500)
64     ap.add_argument("--source", choices=["xlsum", "rss"], default="xlsum")

```

```

65 ap.add_argument("--split", default="train")
66 ap.add_argument("--summary-k", type=int, default=3)
67 ap.add_argument("--method", default="textrank")
68 ap.add_argument("--translate", action="store_true",
69                 help="also render English title/summary (slow: NLLB-600M)")
70 ap.add_argument("--out", default=str(config.CORPUS_JSONL))
71 args = ap.parse_args()
72
73 t0 = time.time()
74 rows = load_rss(args.limit) if args.source == "rss" else load_xlsum(args.limit, args.split)
75 print(f"[1/6] loaded {len(rows)} articles from {args.source}")
76
77 docs = [Document(id=str(r.get("id") or i), title=r.get("title", ""),
78                 text=r.get("text", ""), url=r.get("url", ""))
79          for i, r in enumerate(rows)]
80
81 # --- E: summaries (CPU, no model) -----
82 for d in docs:
83     s = summarize.summarize(d.text, k=args.summary_k, method=args.method)
84     d.summary, d.summary_sentences = s.text, s.sentences
85 print(f"[2/6] summarised ({time.time() - t0:.0f}s)")
86
87 # --- D: topic, batched -----
88 preds = categorize.classify([f"{d.title}. {d.text}" for d in docs])
89 for d, p in zip(docs, preds):
90     d.topic, d.topic_score, d.topic_scores = p.label, p.score, p.all_scores
91 print(f"[3/6] categorised ({time.time() - t0:.0f}s)")
92
93 # --- G + F: entities and sentiment -----
94 for n, d in enumerate(docs, 1):
95     ents = ner.extract(f"{d.title}. {d.summary}".strip())
96     d.entities = ner.summarize_entities(ents)
97     sd = sentiment.analyse_document(d.summary or d.text)
98     d.sentiment, d.polarity = sd.label, sd.polarity
99     d.sentiment_distribution = sd.distribution
100    d.entity_sentiment = sentiment.entity_sentiment(sd, ents)
101    if n % 250 == 0:
102        print(f"...{n}/{len(docs)} ({time.time() - t0:.0f}s)", flush=True)
103 print(f"[4/6] entities + sentiment ({time.time() - t0:.0f}s)")
104
105 # --- A: translation (optional) -----
106 if args.translate:
107     from src import translate as mt
108     titles = mt.translate([d.title for d in docs])
109     summaries = mt.translate([d.summary[:900] for d in docs])
110     for d, t, s in zip(docs, titles, summaries):
111         d.title_en, d.summary_en = t, s
112     print(f"[5/6] translated ({time.time() - t0:.0f}s)")
113 else:
114     print("[5/6] translation skipped (pass --translate to enable)")
115
116 # --- I: morphology stats -----
117 for d in docs:
118     d.tokens, d.inflected_ratio = morph_stats(d.text)
119
120 out = Path(args.out)
121 with open(out, "w", encoding="utf-8") as fh:
122     for d, r in zip(docs, rows):
123         rec = d.to_dict()
124         if r.get("gold_summary"):
125             rec["gold_summary"] = r["gold_summary"]
126         fh.write(json.dumps(rec, ensure_ascii=False) + "\n")
127
128 avg_infl = sum(d.inflected_ratio for d in docs) / max(len(docs), 1)
129 print(f"[6/6] wrote {len(docs)} docs -> {out}")
130 print(f"mean inflected-token ratio: {avg_infl:.1%}")
131 print(f"total {time.time() - t0:.0f}s")
132
133

```

```
134 | if __name__ == "__main__":  
135 |     main()
```

Builds the BM25 and dense search indexes.

```

1  # -*- coding: utf-8 -*-
2  """
3  Build the retrieval indexes over the enriched corpus.
4
5      python scripts/build_index.py          # BM25 (morph) + dense
6      python scripts/build_index.py --no-dense  # BM25 only, no GPU needed
7      python scripts/build_index.py --plain-bm25 # ablation index, no morphology
8
9  The '--plain-bm25' variant writes a second index built from raw whitespace
10 tokens. Keeping both on disk is what lets scripts/evaluate.py run the
11 morphology ablation without rebuilding anything.
12 """
13 from __future__ import annotations
14
15 import argparse
16 import json
17 import sys
18 import time
19 from pathlib import Path
20
21 sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
22 sys.stdout.reconfigure(encoding="utf-8")
23
24 import config          # noqa: E402
25 from src.retrieve import BM25Index, DenseIndex  # noqa: E402
26
27 PLAIN_BM25_INDEX = config.INDEX / "bm25_plain.pkl"
28
29
30 def load_corpus(path: Path):
31     if not path.exists():
32         sys.exit(f"corpus not found: {path}\nRun scripts/build_corpus.py first.")
33     with open(path, encoding="utf-8") as fh:
34         return [json.loads(line) for line in fh]
35
36
37 def search_text(rec: dict) -> str:
38     """Title is repeated so it outweighs the body in BM25 term frequency."""
39     return f"{rec.get('title', '')} {rec.get('title', '')} {rec.get('text', '')}"
40
41
42 def main():
43     ap = argparse.ArgumentParser()
44     ap.add_argument("--corpus", default=str(config.CORPUS_JSONL))
45     ap.add_argument("--no-dense", action="store_true")
46     ap.add_argument("--plain-bm25", action="store_true",
47                     help="also build a morphology-free index for the ablation")
48     args = ap.parse_args()
49
50     corpus = load_corpus(Path(args.corpus))
51     texts = [search_text(r) for r in corpus]
52     doc_ids = list(range(len(corpus)))
53     print(f"indexing {len(corpus)} documents")
54
55     t0 = time.time()
56     bm25 = BM25Index(use_morph=True).build(texts, doc_ids)
57     bm25.save(config.BM25_INDEX)
58     print(f"BM25 (morph) : {len(bm25.postings):>7,} terms "
59           f"if avg_len={bm25.avg_len:.0f} ({time.time() - t0:.1f}s)")
60
61     if args.plain_bm25:
62         t1 = time.time()
63         plain = BM25Index(use_morph=False).build(texts, doc_ids)
64         plain.save(PLAIN_BM25_INDEX)

```

```

65     print(f" BM25 (plain) : {len(plain.postings):>7,} terms "
66           f"({time.time() - t1:.1f}s)")
67     shrink = 1 - len(bm25.postings) / max(len(plain.postings), 1)
68     print(f" vocabulary reduced by {shrink:.1%} through morphological normalisation")
69
70     if not args.no_dense:
71         t2 = time.time()
72         # Embed title + summary rather than the full article: the encoder
73         # truncates at 256 tokens, and the summary is a better 256 tokens than
74         # an arbitrary prefix of the body.
75         dense_texts = [f"{r.get('title', '')}. {r.get('summary', '')}" for r in corpus]
76         dense = DenseIndex().build(dense_texts, doc_ids)
77         dense.save(config.DENSE_INDEX)
78         print(f" Dense          : {dense.vectors.shape} ({time.time() - t2:.1f}s)")
79
80     print(f"done in {time.time() - t0:.1f}s -> {config.INDEX}")
81
82
83 if __name__ == "__main__":
84     main()

```

Fine-tunes MahaBERT as a topic classifier.

```

1  #!/usr/bin/env python3
2  """
3  Fine-tune MahaBERT for Marathi news topic classification.
4
5  Trains 'l3cube-pune/marathi-bert-v2' on the MahaNews headline split
6  (entertainment / sports / state) and writes the result to models/topic-clf,
7  which src/categorize.py then picks up automatically.
8
9  python scripts/train_classifier.py --epochs 3
10
11 Written as a plain PyTorch loop rather than transformers.Trainer: it avoids
12 the 'accelerate' dependency, keeps the mixed-precision and gradient-clipping
13 behaviour explicit, and is small enough to read. Mixed precision plus a batch
14 size of 16 keeps a 110M-parameter BERT inside 4 GB of VRAM.
15 """
16 from __future__ import annotations
17
18 import argparse
19 import json
20 import sys
21 import time
22 from pathlib import Path
23
24 import numpy as np
25 import torch
26 from torch.utils.data import DataLoader, Dataset
27
28 sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
29 sys.stdout.reconfigure(encoding="utf-8")
30
31 import config  # noqa: E402
32
33 LABELS = ["entertainment", "sports", "state"]
34 LABEL2ID = {l: i for i, l in enumerate(LABELS)}
35
36
37 class NewsDataset(Dataset):
38     def __init__(self, texts, labels, tokenizer, max_len=128):
39         self.texts, self.labels = texts, labels
40         self.tok, self.max_len = tokenizer, max_len
41
42     def __len__(self):
43         return len(self.texts)
44
45     def __getitem__(self, i):
46         enc = self.tok(self.texts[i], truncation=True, max_length=self.max_len,
47                        padding="max_length", return_tensors="pt")
48         item = {k: v.squeeze(0) for k, v in enc.items()}
49         item["labels"] = torch.tensor(self.labels[i], dtype=torch.long)
50         return item
51
52
53 def load_split(path: Path):
54     import pandas as pd
55     df = pd.read_parquet(path)
56     # The parquet ships integer labels; config.MAHANEWS_LABELS maps them to
57     # the names used everywhere else in the project.
58     texts = df["text"].astype(str).tolist()
59     labels = [LABEL2ID[config.MAHANEWS_LABELS[int(x)]] for x in df["label"]]
60     return texts, labels
61
62
63 @torch.no_grad()
64 def evaluate(model, loader, device):

```

```

65     model.eval()
66     preds, golds = [], []
67     for batch in loader:
68         labels = batch.pop("labels")
69         batch = {k: v.to(device) for k, v in batch.items()}
70         with torch.autocast(device_type=device, dtype=torch.float16,
71                             enabled=(device == "cuda")):
72             logits = model(**batch).logits
73             preds.extend(logits.float().argmax(-1).cpu().tolist())
74             golds.extend(labels.tolist())
75     return np.array(preds), np.array(golds)
76
77
78 def report(preds, golds):
79     from sklearn.metrics import classification_report, confusion_matrix
80     acc = float((preds == golds).mean())
81     rep = classification_report(golds, preds, target_names=LABELS,
82                               digits=4, zero_division=0, output_dict=True)
83     print(classification_report(golds, preds, target_names=LABELS,
84                               digits=4, zero_division=0))
85     print("confusion matrix (rows = gold):")
86     print(confusion_matrix(golds, preds))
87     return acc, rep
88
89
90 def main():
91     ap = argparse.ArgumentParser()
92     ap.add_argument("--epochs", type=int, default=3)
93     ap.add_argument("--batch-size", type=int, default=16)
94     ap.add_argument("--lr", type=float, default=2e-5)
95     ap.add_argument("--max-len", type=int, default=128)
96     ap.add_argument("--limit", type=int, default=0, help="subsample for a quick run")
97     ap.add_argument("--out", default=str(config.CUSTOM_TOPIC_DIR))
98     args = ap.parse_args()
99
100     from transformers import AutoModelForSequenceClassification, AutoTokenizer
101
102     device = config.DEVICE
103     tok = AutoTokenizer.from_pretrained(config.MODEL_BASE_BERT)
104     model = AutoModelForSequenceClassification.from_pretrained(
105         config.MODEL_BASE_BERT, num_labels=len(LABELS),
106         id2label={i: 1 for i, 1 in enumerate(LABELS)}, label2id=LABEL2ID,
107     ).to(device)
108
109     tr_x, tr_y = load_split(config.MAHANEWS_TRAIN)
110     te_x, te_y = load_split(config.MAHANEWS_TEST)
111     if args.limit:
112         tr_x, tr_y = tr_x[:args.limit], tr_y[:args.limit]
113     print(f"train={len(tr_x)} test={len(te_x)} device={device}")
114
115     train_dl = DataLoader(NewsDataset(tr_x, tr_y, tok, args.max_len),
116                          batch_size=args.batch_size, shuffle=True)
117     test_dl = DataLoader(NewsDataset(te_x, te_y, tok, args.max_len),
118                        batch_size=args.batch_size)
119
120     opt = torch.optim.AdamW(model.parameters(), lr=args.lr, weight_decay=0.01)
121     steps = len(train_dl) * args.epochs
122     sched = torch.optim.lr_scheduler.OneCycleLR(
123         opt, max_lr=args.lr, total_steps=steps, pct_start=0.1, anneal_strategy="linear")
124     scaler = torch.amp.GradScaler(device, enabled=(device == "cuda"))
125
126     t0 = time.time()
127     for epoch in range(1, args.epochs + 1):
128         model.train()
129         running = 0.0
130         for step, batch in enumerate(train_dl, 1):
131             batch = {k: v.to(device) for k, v in batch.items()}
132             opt.zero_grad(set_to_none=True)
133             with torch.autocast(device_type=device, dtype=torch.float16,

```

```

134         enabled=(device == "cuda")):
135             loss = model(**batch).loss
136             scaler.scale(loss).backward()
137             scaler.unscale_(opt)
138             torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
139             scaler.step(opt)
140             scaler.update()
141             sched.step()
142             running += loss.item()
143             if step % 100 == 0:
144                 print(f" epoch {epoch} step {step}/{len(train_dl)} "
145                       f"loss={running / step:.4f} ({time.time() - t0:.0f}s)", flush=True)
146
147             preds, golds = evaluate(model, test_dl, device)
148             print(f"\nepoch {epoch}: test accuracy = {(preds == golds).mean():.4f}\n")
149
150     print("=" * 60)
151     acc, rep = report(*evaluate(model, test_dl, device))
152
153     out = Path(args.out)
154     out.mkdir(parents=True, exist_ok=True)
155     model.save_pretrained(out)
156     tok.save_pretrained(out)
157     (out / "metrics.json").write_text(
158         json.dumps({"accuracy": acc, "report": rep,
159                   "epochs": args.epochs, "lr": args.lr,
160                   "base_model": config.MODEL_BASE_BERT}, indent=2),
161         encoding="utf-8")
162     print(f"\nsaved -> {out} (accuracy {acc:.4f}, {time.time() - t0:.0f}s)")
163
164
165 if __name__ == "__main__":
166     main()

```

All experiments, plus a Marathi-aware ROUGE implementation.

```

1  # -*- coding: utf-8 -*-
2  """
3  Evaluation suite. Five experiments, each runnable on its own:
4
5      python scripts/evaluate.py --task retrieval      # full-headline queries
6      python scripts/evaluate.py --task shortquery    # 1-3 word queries
7      python scripts/evaluate.py --task crossinfl     # when morphology decides
8      python scripts/evaluate.py --task summarization
9      python scripts/evaluate.py --task classification
10     python scripts/evaluate.py --task all
11
12     RETRIEVAL (1, 1b, 1c) is what the project is built around: the same BM25 index
13     is constructed with and without the morphological analyser and queried the
14     same way. The three variants exist because the first one alone is misleading.
15
16     1 full-headline queries      -- morphology gives a small gain (+3.9% MRR)
17     1b short, realistic queries  -- morphology gives no gain, and at 1-2 terms
18                                is slightly worse
19     1c the subset where a query term occurs in the body only in a different
20        inflection -- 68% of articles have one, and there the plain index returns
21        an empty result page 14.4% of the time against 1.1% with morphology
22
23     Taken together: morphology is recall insurance for Marathi search, not a
24     ranking improvement. Reporting only experiment 1 would overstate it, and
25     reporting only the hand-picked शिवाजी example would overstate it badly.
26
27     SUMMARIZATION scores extractive summaries against XL-Sum's gold abstracts with
28     ROUGE-1/2/L. ROUGE is implemented here rather than taken from 'rouge_score',
29     whose default tokenizer strips non-ASCII and would score every Marathi summary
30     as 0. Scores are reported over both raw and morphologically normalised tokens:
31     the raw numbers are comparable with published work, the normalised ones are
32     the fairer measurement for a language where the same lemma surfaces in half a
33     dozen forms.
34
35     Note on the ceiling: XL-Sum summaries are abstractive and roughly one
36     sentence, so an extractive system cannot reach a high ROUGE here. The lead
37     baseline is included so the comparison is against something honest.
38     """
39     from __future__ import annotations
40
41     import argparse
42     import json
43     import random
44     import sys
45     import time
46     from collections import Counter
47     from pathlib import Path
48     from typing import List
49
50     import numpy as np
51
52     sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
53     sys.stdout.reconfigure(encoding="utf-8")
54
55     import config                                     # noqa: E402
56     from src.morph import normalize_tokens, tokenize # noqa: E402
57
58     SEED = 13
59
60
61     # =====
62     # ROUGE
63     # =====
64     def _f1(match: int, pred_n: int, gold_n: int) -> float:

```

```

65     if match == 0 or pred_n == 0 or gold_n == 0:
66         return 0.0
67     p, r = match / pred_n, match / gold_n
68     return 2 * p * r / (p + r)
69
70
71 def _ngrams(tokens: List[str], n: int) -> Counter:
72     return Counter(tuple(tokens[i:i + n]) for i in range(len(tokens) - n + 1))
73
74
75 def rouge_n(pred: List[str], gold: List[str], n: int) -> float:
76     p, g = _ngrams(pred, n), _ngrams(gold, n)
77     overlap = sum((p & g).values())
78     return _f1(overlap, max(sum(p.values()), 0), max(sum(g.values()), 0))
79
80
81 def rouge_l(pred: List[str], gold: List[str]) -> float:
82     """F1 over the longest common subsequence (O(n*m), quadratic memory-free)."""
83     if not pred or not gold:
84         return 0.0
85     prev = [0] * (len(gold) + 1)
86     for a in pred:
87         cur = [0]
88         for j, b in enumerate(gold):
89             cur.append(prev[j] + 1 if a == b else max(cur[j], prev[j + 1]))
90         prev = cur
91     return _f1(prev[-1], len(pred), len(gold))
92
93
94 def rouge_all(pred_text: str, gold_text: str, use_morph: bool) -> dict:
95     if use_morph:
96         p = normalize_tokens(pred_text, drop_stopwords=False)
97         g = normalize_tokens(gold_text, drop_stopwords=False)
98     else:
99         p, g = tokenize(pred_text), tokenize(gold_text)
100     return {"rouge1": rouge_n(p, g, 1), "rouge2": rouge_n(p, g, 2),
101           "rougeL": rouge_l(p, g)}
102
103
104 # =====
105 # Experiment 1 -- retrieval
106 # =====
107 def eval_retrieval(corpus, n_queries=300, top_k=10, include_dense=True):
108     """
109     Known-item retrieval: each query is one document's title, and the correct
110     answer is that document. Titles are held out of the indexed text so the
111     task is not trivially solved by an exact string match.
112     """
113     from src.retrieve import BM25Index, DenseIndex, HybridRetriever
114
115     random.seed(SEED)
116     bodies = [r.get("text", "") for r in corpus]
117     doc_ids = list(range(len(corpus)))
118
119     eligible = [i for i, r in enumerate(corpus)
120                 if len(r.get("title", "").split()) >= 4 and len(r.get("text", "")) > 300]
121     queries = random.sample(eligible, min(n_queries, len(eligible)))
122     print(f" {len(corpus)} docs indexed, {len(queries)} known-item queries\n")
123
124     runs = {}
125     built = {}
126
127     for use_morph in (False, True):
128         tag = "BM25 + morphology" if use_morph else "BM25 (plain tokens)"
129         t0 = time.time()
130         idx = BM25Index(use_morph=use_morph).build(bodies, doc_ids)
131         built[use_morph] = idx
132         results = []
133         for qi in queries:

```

```

134         hits = idx.search(corpus[qi]["title"], top_k=top_k)
135         rank = next((h.rank for h in hits if h.doc_id == qi), None)
136         results.append(rank)
137     runs[tag] = results
138     runs[tag + "::vocab"] = len(idx.postings)
139     runs[tag + "::time"] = time.time() - t0
140
141     if include_dense:
142         t0 = time.time()
143         dense_texts = [f"{r.get('summary', '') or r.get('text', '')[:600]}" for r in corpus]
144         dense = DenseIndex().build(dense_texts, doc_ids, show_progress=False)
145         results = []
146         for qi in queries:
147             hits = dense.search(corpus[qi]["title"], top_k=top_k)
148             rank = next((h.rank for h in hits if h.doc_id == qi), None)
149             results.append(rank)
150         runs["Dense (MahaBERT-SBERT)"] = results
151         runs["Dense (MahaBERT-SBERT)::time"] = time.time() - t0
152
153         hybrid = HybridRetriever(bm25=built[True], dense=dense)
154         results = []
155         for qi in queries:
156             hits = hybrid.search(corpus[qi]["title"], top_k=top_k)
157             rank = next((h.rank for h in hits if h.doc_id == qi), None)
158             results.append(rank)
159         runs["Hybrid (RRF)"] = results
160
161     # ---- report ----
162     header = f" {'system':26s} {'R@1':>7s} {'R@5':>7s} {'R@10':>7s} {'MRR@10':>8s}"
163     print(header)
164     print(" " + "-" * (len(header) - 2))
165     table = {}
166     for name, ranks in runs.items():
167         if "::" in name:
168             continue
169         found = [r for r in ranks if r is not None]
170         r1 = sum(1 for r in found if r < 1) / len(ranks)
171         r5 = sum(1 for r in found if r < 5) / len(ranks)
172         r10 = len(found) / len(ranks)
173         mrr = sum(1 / (r + 1) for r in found) / len(ranks)
174         table[name] = {"R@1": r1, "R@5": r5, "R@10": r10, "MRR@10": mrr}
175         print(f" {name:26s} {r1:7.3f} {r5:7.3f} {r10:7.3f} {mrr:8.3f}")
176
177     plain = table.get("BM25 (plain tokens)", {}).get("MRR@10", 0)
178     morph = table.get("BM25 + morphology", {}).get("MRR@10", 0)
179     if plain:
180         print(f"\n morphology lifts BM25 MRR@10 by {(morph - plain) / plain:.1%}")
181     v_plain = runs.get("BM25 (plain tokens)::vocab", 0)
182     v_morph = runs.get("BM25 + morphology::vocab", 0)
183     if v_plain:
184         print(f" vocabulary: {v_plain:,} -> {v_morph:,} types "
185               f"({1 - v_morph / v_plain:.1%} smaller)")
186     return table
187
188
189 def eval_short_queries(corpus, n_queries=300, top_k=10, lengths=(1, 2, 3)):
190     """
191     The same known-item task, but with realistically short queries.
192
193     A full headline contains eight or ten content words, so even if the
194     analyser rescues two of them the other six already found the document --
195     which is why the full-title experiment understates what morphology does.
196     Real search traffic is one to three words, where a single unmatched term
197     is the difference between a hit and an empty result page.
198
199     Query terms are taken verbatim from the document's *title* and matched
200     against its *body*. Nothing is lemmatised on the way in: the two fields
201     simply inflect the same nouns differently, which is the effect being
202     measured. Taking roots as queries would bias the test toward the analyser.

```

```

203 """
204 from src.retrieve import BM25Index
205 from src.morph import STOPWORDS
206
207 random.seed(SEED)
208 bodies = [r.get("text", "") for r in corpus]
209 doc_ids = list(range(len(corpus)))
210
211 def content_words(title):
212     return [t for t in tokenize(title) if t not in STOPWORDS and len(t) > 2]
213
214 eligible = [i for i, r in enumerate(corpus)
215             if len(content_words(r.get("title", ""))) >= max(lengths)
216             and len(r.get("text", "")) > 300]
217 sample = random.sample(eligible, min(n_queries, len(eligible)))
218
219 indexes = {m: BM25Index(use_morph=m).build(bodies, doc_ids) for m in (False, True)}
220
221 print(f" {len(sample)} queries, terms taken verbatim from each article's title\n")
222 print(f" {'query len':>10s} {'plain MRR':>11s} {'morph MRR':>11s} "
223       f"{'plain R@10':>11s} {'morph R@10':>11s} {'zero-hit plain':>15s}")
224 print(" " + "-" * 74)
225
226 table = {}
227 for n in lengths:
228     stats = {}
229     for use_morph, idx in indexes.items():
230         ranks, empty = [], 0
231         for qi in sample:
232             q = " ".join(content_words(corpus[qi]["title"][:n]))
233             hits = idx.search(q, top_k=top_k)
234             if not hits:
235                 empty += 1
236             ranks.append(next((h.rank for h in hits if h.doc_id == qi), None))
237         found = [r for r in ranks if r is not None]
238         stats[use_morph] = {
239             "MRR@10": sum(1 / (r + 1) for r in found) / len(ranks),
240             "R@10": len(found) / len(ranks),
241             "zero_hit_rate": empty / len(ranks),
242         }
243         table[n] = {"plain": stats[False], "morph": stats[True]}
244         print(f" {n:>10d} {stats[False]['MRR@10']:>11.3f} {stats[True]['MRR@10']:>11.3f} "
245               f"{stats[False]['R@10']:>11.3f} {stats[True]['R@10']:>11.3f} "
246               f"{stats[False]['zero_hit_rate']:>14.1%}")
247
248 best = max(lengths, key=lambda n: (table[n]["morph"]["MRR@10"]
249                                   - table[n]["plain"]["MRR@10"]))
250 p, m = table[best]["plain"]["MRR@10"], table[best]["morph"]["MRR@10"]
251 if p:
252     print(f"\n largest gain at {best}-term queries: MRR@10 {p:.3f} -> {m:.3f} "
253           f"({(m - p) / p:.1%}")
254     return table
255
256
257 def eval_cross_inflection(corpus, n_docs=1200, top_k=10):
258     """
259     How often does morphology actually decide a query, and how much does it
260     help when it does?
261
262     Experiments 1 and 1b measure the average query, and on the average query
263     the analyser barely moves the needle -- a title word usually appears in
264     the body in the same form, so there is nothing to rescue. The interesting
265     quantity is the size and difficulty of the subset where that is not
266     true.
267
268     A query here is a single title word chosen so that
269     - its exact surface form does NOT occur in the article body, but
270     - some body token shares its root.
271     That is precisely the case a plain index cannot serve and a normalised one

```

```

272 can. The 'coverage' figure below says how often such a word exists at all,
273 which is the honest measure of how much this matters in practice.
274 """
275 from src.retrieve import BM25Index
276 from src.morph import STOPWORDS, stem
277
278 random.seed(SEED)
279 bodies = [r.get("text", "") for r in corpus]
280 doc_ids = list(range(len(corpus)))
281
282 pool = [i for i, r in enumerate(corpus) if len(r.get("text", "")) > 300]
283 sample = random.sample(pool, min(n_docs, len(pool)))
284
285 cases = []
286 for qi in sample:
287     body_tokens = tokenize(corpus[qi]["text"])
288     surface = set(body_tokens)
289     roots = {stem(t) for t in body_tokens}
290     for w in tokenize(corpus[qi].get("title", "")):
291         if w in STOPWORDS or len(w) <= 2:
292             continue
293         if w not in surface and stem(w) in roots:
294             cases.append((qi, w))
295             break
296
297 coverage = len(cases) / len(sample)
298 print(f" {len(sample)} articles examined")
299 print(f" {len(cases)} ({coverage:.1%}) contain a headline word that is "
300       f"absent from the body\n in its surface form but present as a root "
301       f"-- i.e. only a morphological\n index can match it\n")
302
303 if not cases:
304     return {"coverage": coverage}
305
306 indexes = {m: BM25Index(use_morph=m).build(bodies, doc_ids) for m in (False, True)}
307 out = {}
308 print(f" {'system':22s} {'R@1':>7s} {'R@10':>7s} {'MRR@10':>8s} {'zero-hit':>10s}")
309 print(" " + "-" * 58)
310 for use_morph, idx in indexes.items():
311     ranks, empty = [], 0
312     for qi, w in cases:
313         hits = idx.search(w, top_k=top_k)
314         if not hits:
315             empty += 1
316         ranks.append(next((h.rank for h in hits if h.doc_id == qi), None))
317     found = [r for r in ranks if r is not None]
318     stats = {
319         "R@1": sum(1 for r in found if r < 1) / len(ranks),
320         "R@10": len(found) / len(ranks),
321         "MRR@10": sum(1 / (r + 1) for r in found) / len(ranks),
322         "zero_hit_rate": empty / len(ranks),
323     }
324     tag = "BM25 + morphology" if use_morph else "BM25 (plain tokens)"
325     out[tag] = stats
326     print(f" {tag:22s} {stats['R@1']:7.3f} {stats['R@10']:7.3f} "
327           f"{stats['MRR@10']:8.3f} {stats['zero_hit_rate']:9.1%}")
328
329 out["coverage"] = coverage
330 print(f"\n On this subset the plain index returns nothing for "
331       f"{out['BM25 (plain tokens)']['zero_hit_rate']:.0%} of queries.")
332 print(f" Weighted over all traffic, the subset is {coverage:.1%} of queries.")
333 return out
334
335
336 # =====
337 # Experiment 2 -- summarization
338 # =====
339 def eval_summarization(corpus, n=300, k=2):
340     from src import summarize as S

```

```

341
342 rows = [r for r in corpus if r.get("gold_summary") and len(r.get("text", "")) > 400]
343 random.seed(SEED)
344 rows = random.sample(rows, min(n, len(rows)))
345 if not rows:
346     print(" no gold summaries in corpus (build it from XL-Sum to evaluate)")
347     return {}
348 print(f" scoring {len(rows)} articles against XL-Sum gold summaries (k={k})\n")
349
350 methods = ["lead", "textrank"]
351 out = {}
352 for method in methods:
353     for use_morph in (False, True):
354         scores = []
355         for r in rows:
356             pred = S.summarize(r["text"], k=k, method=method).text
357             scores.append(rouge_all(pred, r["gold_summary"], use_morph))
358         agg = {m: float(np.mean([s[m] for s in scores])) for m in
359             ("rouge1", "rouge2", "rougeL")}
360         tag = f"{method}{use_morph}" if use_morph else ""
361         out[tag] = agg
362
363 print(f" {'system':18s} {'ROUGE-1':>9s} {'ROUGE-2':>9s} {'ROUGE-L':>9s}")
364 print(" " + "-" * 48)
365 for tag, agg in out.items():
366     print(f" {tag:18s} {agg['rouge1']:9.4f} {agg['rouge2']:9.4f} {agg['rougeL']:9.4f}")
367 print("\n (+morph = ROUGE computed over morphologically normalised tokens)")
368 return out
369
370
371 # =====
372 # Experiment 3 -- classification
373 # =====
374 def eval_classification(limit=0, model_name=None):
375     import pandas as pd
376     from sklearn.metrics import classification_report
377
378     from src import categorize
379
380     df = pd.read_parquet(config.MAHANEWS_TEST)
381     if limit:
382         df = df.head(limit)
383     texts = df["text"].astype(str).tolist()
384     gold = [config.MAHANEWS_LABELS[int(x)] for x in df["label"]]
385
386     model = model_name or categorize.active_model()
387     print(f" model: {model}")
388     print(f" {len(texts)} test headlines\n")
389
390     preds = [p.label for p in categorize.classify(texts, model_name=model)]
391
392     # The 12-class L3Cube model has to be mapped onto the 3-class MahaNews
393     # label space before it can be scored against this test set.
394     if model == config.MODEL_TOPIC:
395         mapping = {"Sports": "sports", "Manoranjana": "entertainment",
396             "Politics": "state", "Crime": "state", "Auto": "state",
397             "Bhakti": "state", "Education": "state", "Fashion": "entertainment",
398             "Health": "state", "International": "state", "Tech": "state",
399             "Travel": "state"}
400         preds = [mapping.get(p, "state") for p in preds]
401         print(" (12-class zero-shot predictions mapped onto the 3-class"
402             " MahaNews label space)\n")
403
404     acc = sum(p == g for p, g in zip(preds, gold)) / len(gold)
405     print(classification_report(gold, preds, digits=4, zero_division=0))
406     print(f" accuracy = {acc:.4f}")
407     return {"accuracy": acc}
408
409

```

```

410 # =====
411 def main():
412     ap = argparse.ArgumentParser()
413     ap.add_argument("--task", default="all",
414                     choices=["all", "retrieval", "shortquery", "crossinfl",
415                               "summarization", "classification"])
416     ap.add_argument("--corpus", default=str(config.CORPUS_JSONL))
417     ap.add_argument("--queries", type=int, default=300)
418     ap.add_argument("--summ-n", type=int, default=300)
419     ap.add_argument("--clf-limit", type=int, default=0)
420     ap.add_argument("--no-dense", action="store_true")
421     ap.add_argument("--out", default=str(config.OUTPUTS / "evaluation.json"))
422     args = ap.parse_args()
423
424     corpus = []
425     path = Path(args.corpus)
426     if path.exists():
427         corpus = [json.loads(l) for l in open(path, encoding="utf-8")]
428
429     results = {}
430     if args.task in ("all", "retrieval"):
431         print("\n" + "=" * 68)
432         print("EXPERIMENT 1 -- RETRIEVAL: does morphology help find Marathi news?")
433         print("=" * 68)
434         if not corpus:
435             print(" no corpus; run scripts/build_corpus.py first")
436         else:
437             results["retrieval"] = eval_retrieval(
438                 corpus, n_queries=args.queries, include_dense=not args.no_dense)
439
440     if args.task in ("all", "shortquery"):
441         print("\n" + "=" * 68)
442         print("EXPERIMENT 1b -- RETRIEVAL with short, realistic queries")
443         print("=" * 68)
444         if not corpus:
445             print(" no corpus; run scripts/build_corpus.py first")
446         else:
447             results["short_queries"] = eval_short_queries(
448                 corpus, n_queries=args.queries)
449
450     if args.task in ("all", "crossinfl"):
451         print("\n" + "=" * 68)
452         print("EXPERIMENT 1c -- WHEN does morphology decide the query?")
453         print("=" * 68)
454         if not corpus:
455             print(" no corpus; run scripts/build_corpus.py first")
456         else:
457             results["cross_inflection"] = eval_cross_inflection(corpus)
458
459     if args.task in ("all", "summarization"):
460         print("\n" + "=" * 68)
461         print("EXPERIMENT 2 -- SUMMARIZATION vs XL-Sum gold abstracts")
462         print("=" * 68)
463         if not corpus:
464             print(" no corpus; run scripts/build_corpus.py first")
465         else:
466             results["summarization"] = eval_summarization(corpus, n=args.summ_n)
467
468     if args.task in ("all", "classification"):
469         print("\n" + "=" * 68)
470         print("EXPERIMENT 3 -- TOPIC CLASSIFICATION on the MahaNews test split")
471         print("=" * 68)
472         results["classification"] = eval_classification(limit=args.clf_limit)
473         from src import categorize as _c
474         if _c.custom_model_available():
475             print("\n --- fine-tuned 3-class head ---\n")
476             results["classification_finetuned"] = eval_classification(
477                 limit=args.clf_limit, model_name=str(config.CUSTOM_TOPIC_DIR))
478

```

```
479     Path(args.out).write_text(json.dumps(results, indent=2, ensure_ascii=False),
480                               encoding="utf-8")
481     print(f"\nresults -> {args.out}")
482
483
484 if __name__ == "__main__":
485     main()
```

Labels the corpus with the full models, to train the students.

```

1  # -*- coding: utf-8 -*-
2  """
3  Label XL-Sum with the full models, as training data for the in-browser models.
4
5      python scripts/distill_label.py          # train + val -> fit, test -> score
6      python scripts/distill_label.py --limit 500      # quick run
7
8  The demo page cannot run the MahaBERT models, so it carries small students
9  (src/lite.py) trained to reproduce them. This script produces the targets:
10
11  * topic      -- the 12-class topic distribution for every article
12  * sentiment  -- the 3-class distribution for each sentence
13  * entities   -- MahaNER spans for each sentence
14
15  Sentences are capped per article so long articles do not dominate. Output is
16  one JSON object per article in data/processed/distill_{split}.jsonl.
17  """
18  from __future__ import annotations
19
20  import argparse
21  import json
22  import sys
23  import time
24  from pathlib import Path
25
26  import torch
27
28  sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
29  sys.stdout.reconfigure(encoding="utf-8")
30
31  import config          # noqa: E402
32  from src import categorize, models          # noqa: E402
33  from src.morph import sent_tokenize        # noqa: E402
34  from src.ner import _snap_to_word          # noqa: E402
35
36  SPLITS = {"train": config.XLSUM_TRAIN, "val": config.XLSUM_VAL,
37           "test": config.XLSUM_TEST}
38
39
40  def load(path: Path, limit: int):
41      rows = []
42      with open(path, encoding="utf-8") as fh:
43          for line in fh:
44              r = json.loads(line)
45              if len(r.get("text", "")) < 200:
46                  continue
47              rows.append(r)
48              if limit and len(rows) >= limit:
49                  break
50      return rows
51
52
53  def pick_sentences(text: str, cap: int):
54      sents = [s for s in sent_tokenize(text)
55              if len(s.split()) >= 3 and len(s) <= 600]
56      return sents[:cap]
57
58
59  @torch.no_grad()
60  def ner_batch(texts, batch_size):
61      """Batched MahaNER, returning word-snapped [start, end, label] spans."""
62      tok, mod = models.load_token_classifier(config.MODEL_NER)
63      id2label = mod.config.id2label
64      out = []

```

```

65     for i in range(0, len(texts), batch_size):
66         chunk = texts[i:i + batch_size]
67         enc = tok(chunk, return_offsets_mapping=True, truncation=True,
68                   max_length=config.MAX_LEN, padding=True, return_tensors="pt")
69         offsets = enc.pop("offset_mapping").tolist()
70         enc.pop("special_tokens_mask", None)
71         enc = {k: v.to(config.DEVICE) for k, v in enc.items()}
72         labels = mod(**enc).logits.argmax(-1).cpu().tolist()
73         for text, offs, labs in zip(chunk, offsets, labels):
74             spans, cur = [], None
75             for (s, e), lid in zip(offs, labs):
76                 if s == e:
77                     continue
78                 lab = id2label[lid]
79                 if lab == "Other":
80                     cur = None
81                     continue
82                 if cur and cur[2] == lab and s - cur[1] <= 1:
83                     cur[1] = e
84                 else:
85                     cur = [s, e, lab]
86                     spans.append(cur)
87             snapped = []
88             for s, e, lab in spans:
89                 s, e = _snap_to_word(text, s, e)
90                 if snapped and snapped[-1][:2] == [s, e]:
91                     continue
92                 snapped.append([s, e, lab])
93             out.append(snapped)
94     return out
95
96
97 def main():
98     ap = argparse.ArgumentParser()
99     ap.add_argument("--splits", nargs="+", default=["train", "val", "test"])
100    ap.add_argument("--limit", type=int, default=0, help="articles per split (0 = all)")
101    ap.add_argument("--sent-cap", type=int, default=8)
102    ap.add_argument("--batch", type=int, default=32)
103    args = ap.parse_args()
104
105    for split in args.splits:
106        t0 = time.time()
107        rows = load(SPLITS[split], args.limit)
108        print(f"[{split}] {len(rows)} articles on {config.DEVICE}", flush=True)
109
110        topics = categorize.classify([f"{r['title']}. {r['text']}" for r in rows],
111                                    batch_size=args.batch)
112        print(f"    topics    ({time.time() - t0:.0f}s)", flush=True)
113
114        sents_per_doc = [pick_sentences(r["text"], args.sent_cap) for r in rows]
115        # The title is labelled too: pasted input often starts with one.
116        flat = [s for r, ss in zip(rows, sents_per_doc) for s in [r["title"], *ss]]
117
118        sent_scores = []
119        tok, mod = models.load_classifier(config.MODEL_SENTIMENT)
120        id2label = mod.config.id2label
121        order = [id2label[j] for j in range(len(id2label))]
122        with torch.no_grad():
123            for i in range(0, len(flat), args.batch):
124                chunk = flat[i:i + args.batch]
125                enc = tok(chunk, padding=True, truncation=True,
126                          max_length=config.MAX_LEN, return_tensors="pt").to(config.DEVICE)
127                probs = mod(**enc).logits.float().softmax(-1).cpu().tolist()
128                sent_scores.extend(
129                    [lab: round(p[j], 4) for j, lab in enumerate(order)] for p in probs)
130        print(f"    sentiment {len(flat)} sentences ({time.time() - t0:.0f}s)", flush=True)
131
132        spans = ner_batch(flat, args.batch)
133        print(f"    entities    ({time.time() - t0:.0f}s)", flush=True)

```

```

134 out = config.PROCESSED / f"distill_{split}.jsonl"
135 k = 0
136
137 with open(out, "w", encoding="utf-8") as fh:
138     for r, pred, ss in zip(rows, topics, sents_per_doc):
139         n = len(ss) + 1
140         fh.write(json.dumps({
141             "id": r["id"], "title": r["title"], "text": r["text"],
142             "topic": {lab: round(v, 4) for lab, v in pred.all_scores.items()},
143             "sentences": [{s: s, "sent": sc, "ner": sp} for s, sc, sp in
144                             zip(flat[k:k + n], sent_scores[k:k + n], spans[k:k + n])],
145             }, ensure_ascii=False) + "\n")
146         k += n
147     print(f" wrote {out} ({time.time() - t0:.0f}s)", flush=True)
148
149
150 if __name__ == "__main__":
151     main()

```

Trains and scores the in-browser student models.

```

1  # -*- coding: utf-8 -*-
2  """
3  Train the in-browser student models (src/lite.py) on the full models' output.
4
5      python scripts/distill_label.py      # label XL-Sum with the full models
6      python scripts/train_lite.py        # fit, score on test, write app/lite_models.json
7
8  Each student is a linear softmax model trained against the teacher's
9  probabilities (soft targets), using torch EmbeddingBag so the sparse features
10 never become a dense matrix. Weights are then rounded to small integers times
11 one scale per model -- that is what the page embeds -- and the *rounded*
12 models are what get scored, so the agreement figures describe exactly what
13 ships.
14
15 Agreement is measured against the teacher on the XL-Sum test split, which the
16 students never see:
17
18 * topic      -- top-1 label matches the 12-class topic model
19 * sentiment  -- sentence label matches the sentiment model; also the
20                  document tone after the same length-weighted aggregation
21 * entities   -- span-level precision / recall / F1 against MahaNER
22 """
23 from __future__ import annotations
24
25 import argparse
26 import base64
27 import json
28 import math
29 import sys
30 import time
31 from collections import Counter
32 from pathlib import Path
33
34 import torch
35
36 sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
37 sys.stdout.reconfigure(encoding="utf-8")
38
39 import config                                # noqa: E402
40 from src import lite                         # noqa: E402
41
42 torch.manual_seed(0)
43
44
45 def load(split):
46     path = config.PROCESSED / f"distill_{split}.jsonl"
47     if not path.exists():
48         sys.exit(f"{path} missing -- run scripts/distill_label.py first")
49     return [json.loads(l) for l in open(path, encoding="utf-8")]
50
51
52 # -----
53 # Generic sparse softmax regression
54 # -----
55 def fit_softmax(rows, targets, n_feats, n_classes, *, epochs, lr, l2,
56                 batch=256):
57     """
58     rows: list of (feature_ids, feature_values); targets: list of prob lists.
59     Returns (W [n_feats x n_classes], b [n_classes]) as float tensors.
60     """
61     emb = torch.nn.EmbeddingBag(n_feats, n_classes, mode="sum")
62     torch.nn.init.zeros_(emb.weight)
63     bias = torch.nn.Parameter(torch.zeros(n_classes))
64     opt = torch.optim.Adam(list(emb.parameters()) + [bias], lr=lr)

```

```

65 y_all = torch.tensor(targets, dtype=torch.float32)
66 order = list(range(len(rows)))
67
68 for ep in range(epochs):
69     g = torch.Generator().manual_seed(ep)
70     perm = torch.randperm(len(order), generator=g).tolist()
71     total = 0.0
72     for i in range(0, len(perm), batch):
73         idx = perm[i:i + batch]
74         ids, vals, offs = [], [], []
75         for r in idx:
76             offs.append(len(ids))
77             ids.extend(rows[r][0])
78             vals.extend(rows[r][1])
79         logits = emb(torch.tensor(ids, dtype=torch.long),
80                     torch.tensor(offs, dtype=torch.long),
81                     per_sample_weights=torch.tensor(vals, dtype=torch.float32)) + bias
82         loss = -(y_all[idx] * logits.log_softmax(-1)).sum(-1).mean()
83         loss = loss + l2 * emb.weight.pow(2).sum() / len(rows)
84         opt.zero_grad()
85         loss.backward()
86         opt.step()
87         total += loss.item() * len(idx)
88     print(f"    epoch {ep + 1}/{epochs}    loss {total / len(rows):.4f}", flush=True)
89     return emb.weight.detach(), bias.detach()
90
91
92 def quantize(W, levels=127):
93     """
94     int8 weights plus one scale, packed row-major and base64-encoded -- about
95     a third of the size of a JSON list of numbers.
96     """
97     scale = float(W.abs().max()) / levels or 1.0
98     q = torch.round(W / scale).clamp(-levels, levels).to(torch.int8)
99     return base64.b64encode(q.numpy().tobytes()).decode("ascii"), scale
100
101
102 # -----
103 # Topic
104 # -----
105 def train_topic(train, max_vocab):
106     labels = sorted(train[0]["topic"])
107     docs = [lite.topic_terms(r["title"], r["text"]) for r in train]
108     df = Counter(t for d in docs for t in set(d))
109     vocab = [t for t, c in df.most_common(max_vocab) if c >= 3]
110     index = {t: i for i, t in enumerate(vocab)}
111     n = len(docs)
112     idf = [round(math.log((1 + n) / (1 + df[t])) + 1.0, 4) for t in vocab]
113
114     rows = []
115     for d in docs:
116         tf = Counter(t for t in d if t in index)
117         vec = {t: (1 + math.log(c)) * idf[index[t]] for t, c in tf.items()}
118         norm = math.sqrt(sum(v * v for v in vec.values())) or 1.0
119         rows.append([index[t] for t in vec], [v / norm for v in vec.values()])
120     targets = [[r["topic"][l] for l in labels] for r in train]
121     print(f"    topic: {len(vocab):,} terms, {len(rows):,} articles")
122     W, b = fit_softmax(rows, targets, len(vocab), len(labels),
123                       epochs=12, lr=0.05, l2=1e-4)
124     Wq, scale = quantize(W)
125     return {"labels": labels, "terms": vocab, "idf": idf, "W": Wq,
126           "b": [round(float(x), 4) for x in b], "scale": scale,
127           "max_tokens": lite.TOPIC_MAX_TOKENS}
128
129
130 # -----
131 # Sentiment
132 # -----
133 def train_sentiment(train, max_feats):

```

```

134 labels = ["Negative", "Neutral", "Positive"]
135 sents = [s for r in train for s in r["sentences"]]
136 feats = [set(lite.sentiment_features(s["s"])) for s in sents]
137 cnt = Counter(f for fs in feats for f in fs)
138 vocab = [f for f, c in cnt.most_common(max_feats) if c >= 3]
139 index = {f: i for i, f in enumerate(vocab)}
140 rows = []
141 for fs in feats:
142     known = sorted(f for f in fs if f in index)
143     inv = 1.0 / math.sqrt(len(known)) if known else 0.0
144     rows.append([index[f] for f in known], [inv] * len(known))
145 targets = [[s["sent"][l] for l in labels] for s in sents]
146 print(f" sentiment: {len(vocab):,} features, {len(rows):,} sentences")
147 W, b = fit_softmax(rows, targets, len(vocab), len(labels),
148                    epochs=8, lr=0.03, l2=1e-4, batch=512)
149 Wq, scale = quantize(W)
150 return {"labels": labels, "feats": vocab, "W": Wq,
151         "b": [round(float(x), 4) for x in b], "scale": scale}
152
153
154 # -----
155 # Entities
156 # -----
157 def word_labels(text, spans):
158     offs = lite.words(text)
159     out = []
160     for s, e in offs:
161         lab = "0"
162         for a, z, l in spans:
163             if a <= s and e <= z:
164                 lab = l
165                 break
166         out.append(lab)
167     return [text[s:e] for s, e in offs], out
168
169
170 def train_ner(train, max_feats, keep):
171     labels = lite.NER_LABELS
172     lid = {l: i for i, l in enumerate(labels)}
173     seqs = []
174     for r in train:
175         for s in r["sentences"]:
176             toks, labs = word_labels(s["s"], s["ner"])
177             if toks:
178                 seqs.append((toks, labs))
179
180     # Teacher forcing: previous-label features use the teacher's labels.
181     examples, cnt = [], Counter()
182     for toks, labs in seqs:
183         for i in range(len(toks)):
184             prev = labs[i - 1] if i > 0 else "0"
185             prev2 = labs[i - 2] if i > 1 else "0"
186             fs = lite.ner_features(toks, i, prev, prev2)
187             examples.append((fs, lid[labs[i]]))
188             cnt.update(fs)
189     vocab = [f for f, c in cnt.most_common(max_feats) if c >= 2]
190     index = {f: i for i, f in enumerate(vocab)}
191     rows = []
192     for fs, _ in examples:
193         known = [index[f] for f in fs if f in index]
194         rows.append((known, [1.0] * len(known)))
195     targets = [[1.0 if j == y else 0.0 for j in range(len(labels))] for _, y in examples]
196     print(f" entities: {len(vocab):,} features, {len(rows):,} words, "
197           f"{sum(1 for _, y in examples if y):,} inside entities")
198     W, b = fit_softmax(rows, targets, len(vocab), len(labels),
199                        epochs=6, lr=0.05, l2=1e-5, batch=1024)
200
201     # Keep the features that move the most decisions: weight spread times how
202     # often the feature fires. Spread alone keeps rare, memorised features

```

```

203     # and drops common decisive ones (the weights that say "आहे" is not a
204     # name), which halves precision at the same size.
205     spread = (W.max(1).values - W.min(1).values)
206     freq = torch.tensor([cnt[f] for f in vocab], dtype=torch.float32)
207     order = torch.argsort(spread * freq, descending=True)[:keep].tolist()
208     order.sort()
209     Wk = W[order]
210     Wq, scale = quantize(Wk)
211     return {"labels": labels, "feats": [vocab[i] for i in order], "W": Wq,
212         "b": [round(float(x), 4) for x in b], "scale": scale}
213
214
215     # -----
216     # Scoring the shipped (rounded) models
217     # -----
218     def score(model: lite.Lite, test):
219         top = sum(max(model.topic(r["title"], r["text"]).items(), key=lambda kv: kv[1])[0]
220             == max(r["topic"], key=r["topic"].get) for r in test)
221
222         s_ok = s_n = d_ok = 0
223         for r in test:
224             polys, weights = [], []
225             for s in r["sentences"]:
226                 p = model.sentence_sentiment(s["s"])
227                 s_ok += max(p, key=p.get) == max(s["sent"], key=s["sent"].get)
228                 s_n += 1
229             # Document tone over the article's sentences, both sides aggregated
230             # the same way (length-weighted expected polarity).
231             def tone(get):
232                 num = den = 0.0
233                 for s in r["sentences"]:
234                     p = get(s)
235                     w = max(len(s["s"]), 1)
236                     num += w * sum(lite.POLARITY[k] * v for k, v in p.items())
237                     den += w
238                 pol = num / den if den else 0.0
239                 return "Positive" if pol > 0.15 else "Negative" if pol < -0.15 else "Neutral"
240             d_ok += tone(lambda s: model.sentence_sentiment(s["s"])) == tone(lambda s: s["sent"])
241
242         tp = fp = fn = 0
243         for r in test:
244             for s in r["sentences"]:
245                 gold = {tuple(x) for x in s["ner"]}
246                 pred = set(model.entities(s["s"]))
247                 tp += len(gold & pred)
248                 fp += len(pred - gold)
249                 fn += len(gold - pred)
250         p = tp / max(tp + fp, 1)
251         rc = tp / max(tp + fn, 1)
252         return {
253             "test_articles": len(test),
254             "topic_top1_agreement": round(top / len(test), 4),
255             "sentiment_sentence_agreement": round(s_ok / max(s_n, 1), 4),
256             "sentiment_document_agreement": round(d_ok / len(test), 4),
257             "ner_precision": round(p, 4), "ner_recall": round(rc, 4),
258             "ner_f1": round(2 * p * rc / max(p + rc, 1e-9), 4),
259         }
260
261
262     def main():
263         ap = argparse.ArgumentParser()
264         ap.add_argument("--topic-vocab", type=int, default=20000)
265         ap.add_argument("--sent-feats", type=int, default=40000)
266         ap.add_argument("--ner-feats", type=int, default=400000)
267         ap.add_argument("--ner-keep", type=int, default=40000)
268         ap.add_argument("--out", default=str(lite.LITE_MODELS))
269         args = ap.parse_args()
270
271         t0 = time.time()

```

```

272 train = load("train") + load("val")
273 test = load("test")
274 print(f"{len(train):,} training articles, {len(test):,} test articles")
275
276 data = {
277     "topic": train_topic(train, args.topic_vocab),
278     "sentiment": train_sentiment(train, args.sent_feats),
279     "ner": train_ner(train, args.ner_feats, args.ner_keep),
280 }
281 print(f"trained ({time.time() - t0:.0f}s); scoring the rounded models on test...")
282 data["agreement"] = score(lite.Lite(data), test)
283 for k, v in data["agreement"].items():
284     print(f" {k:32s} {v}")
285
286 out = Path(args.out)
287 out.write_text(json.dumps(data, ensure_ascii=False, separators=(",", ":")),
288               encoding="utf-8")
289 print(f"wrote {out} ({out.stat().st_size / 1024:.0f} KB, {time.time() - t0:.0f}s)")
290
291
292 if __name__ == "__main__":
293     main()

```

Exports the demo page's data.

```

1  # -*- coding: utf-8 -*-
2  """
3  Export everything the demo page needs into one JSON file.
4
5      python scripts/export_demo.py --docs 400 --showcase 8
6
7  Two things are exported, and the distinction matters:
8
9  * REAL PIPELINE OUTPUT -- topics, entities, sentiment, summaries and
10 translations that the actual models produced, copied verbatim out of
11 data/processed/corpus.jsonl. The demo page does not re-run any model; it
12 displays what the pipeline already computed.
13
14 * THE ANALYSER'S RULE TABLES -- the suffix inventory, stopword list and
15 exception map are dumped straight from src/morph.py so the JavaScript
16 port on the page runs the *same* rules rather than a hand-retyped copy
17 that could silently drift out of sync.
18
19 Translation is done here rather than in build_corpus so only the handful of
20 showcase articles pay the NLLB cost. Run with CUDA_VISIBLE_DEVICES= to keep it
21 off a GPU that is busy training.
22 """
23 from __future__ import annotations
24
25 import argparse
26 import json
27 import sys
28 from collections import Counter, defaultdict
29 from pathlib import Path
30
31 sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
32 sys.stdout.reconfigure(encoding="utf-8")
33
34 import config                                # noqa: E402
35 from src import morph                        # noqa: E402
36 from src.lite import LITE_MODELS            # noqa: E402
37 from src.ner import merge_aliases           # noqa: E402
38
39
40 def morph_tables() -> dict:
41     """Dump the analyser's rules so the JS port stays in lockstep."""
42     def flat(table):
43         return [[t[0], t[1], t[2] if len(t) > 2 else ""] for t in table]
44
45     return {
46         "clitics": flat(morph.CLITICS),
47         "postpositions": flat(morph.POSTPOSITIONS),
48         "verb": flat(morph.VERB_SUFFIXES),
49         "case": flat(morph.CASE_NUMBER),
50         "stopwords": sorted(morph.STOPWORDS),
51         "exceptions": morph.EXCEPTIONS,
52         "marks": "".join(sorted(morph._MARKS)),
53         "finalMatras": "".join(sorted(morph._FINAL_MATRAS)),
54         "minStem": morph.MIN_STEM,
55         "maxStrips": morph.MAX_STRIPS,
56         "virama": morph.VIRAMA,
57     }
58
59
60 def pick_showcase(corpus, n):
61     """Spread the showcase across topics, preferring entity-rich articles."""
62     by_topic = defaultdict(list)
63     for r in corpus:
64         by_topic[r.get("topic", "?")].append(r)

```

```

65
66 def richness(r):
67     return sum(len(v) for v in (r.get("entities") or {}).values())
68
69 picks, topics = [], sorted(by_topic, key=lambda t: -len(by_topic[t]))
70 while len(picks) < n and topics:
71     for t in list(topics):
72         pool = [r for r in by_topic[t]
73                 if 400 < len(r.get("text", "")) < 6000 and richness(r) >= 4]
74         pool.sort(key=richness, reverse=True)
75         pool = [r for r in pool if r not in picks]
76         if pool:
77             picks.append(pool[0])
78         else:
79             topics.remove(t)
80         if len(picks) >= n:
81             break
82     return picks[:n]
83
84
85 def slim(rec, keep_text=False):
86     out = {
87         "id": rec.get("id"),
88         "title": rec.get("title", ""),
89         "summary": rec.get("summary", ""),
90         "topic": rec.get("topic", ""),
91         "topic_score": round(float(rec.get("topic_score", 0)), 3),
92         "sentiment": rec.get("sentiment", ""),
93         "polarity": round(float(rec.get("polarity", 0)), 3),
94         "entities": rec.get("entities") or {},
95         "url": rec.get("url", ""),
96         "tokens": rec.get("tokens", 0),
97         "inflected_ratio": round(float(rec.get("inflected_ratio", 0)), 3),
98     }
99     if keep_text:
100         out["text"] = rec.get("text", "")
101         out["gold_summary"] = rec.get("gold_summary", "")
102         out["topic_scores"] = {k: round(float(v), 4)
103                               for k, v in (rec.get("topic_scores") or {}).items()}
104         out["sentiment_distribution"] = rec.get("sentiment_distribution") or {}
105     return out
106
107
108 def main():
109     ap = argparse.ArgumentParser()
110     ap.add_argument("--docs", type=int, default=400, help="docs in the search index")
111     ap.add_argument("--showcase", type=int, default=8)
112     ap.add_argument("--no-translate", action="store_true")
113     ap.add_argument("--out", default=str(config.OUTPUTS / "demo_data.json"))
114     args = ap.parse_args()
115
116     if not LITE_MODELS.exists():
117         sys.exit(f"{LITE_MODELS} missing -- run scripts/train_lite.py first")
118     corpus = [json.loads(l) for l in open(config.CORPUS_JSONL, encoding="utf-8")]
119     print(f"corpus: {len(corpus)} docs")
120
121     showcase = pick_showcase(corpus, args.showcase)
122     print(f"showcase: {len(showcase)} articles "
123           f"({'', '.join(r['topic'] for r in showcase)}")
124
125     # Search index: the showcase articles plus a topic-spread sample.
126     chosen = {id(r): r for r in showcase}
127     per_topic = defaultdict(int)
128     cap = max(1, args.docs // max(len(set(r.get('topic') for r in corpus)), 1))
129     for r in corpus:
130         if len(chosen) >= args.docs:
131             break
132         t = r.get("topic", "?")
133         if id(r) in chosen or per_topic[t] >= cap or len(r.get("text", "")) < 300:

```

```

134         continue
135     chosen[id(r)] = r
136     per_topic[t] += 1
137     for r in corpus: # top up if caps left us short
138         if len(chosen) >= args.docs:
139             break
140         if id(r) not in chosen and len(r.get("text", "")) >= 300:
141             chosen[id(r)] = r
142     docs = list(chosen.values())
143     print(f"search index: {len(docs)} docs")
144
145     # Translation of the showcase only.
146     if not args.no_translate:
147         from src import translate as mt
148         print(f"translating {len(showcase)} showcase articles on {config.DEVICE}...")
149         titles = mt.translate([r["title"] for r in showcase])
150         summaries = mt.translate([r["summary"][:700] for r in showcase])
151         for r, t, s in zip(showcase, titles, summaries):
152             r["_title_en"], r["_summary_en"] = t, s
153         print(" done")
154
155     show_out = []
156     for r in showcase:
157         d = slim(r, keep_text=True)
158         d["_title_en"] = r.get("_title_en", "")
159         d["_summary_en"] = r.get("_summary_en", "")
160         show_out.append(d)
161
162     # Corpus-level aggregates, computed over the whole 3,000 not the sample.
163     ent_buckets = defaultdict(Counter)
164     for r in corpus:
165         for label, vals in (r.get("entities") or {}).items():
166             for v in vals:
167                 ent_buckets[label][v] += 1
168     top_entities = {
169         label: merge_aliases(c).most_common(12)
170         for label, c in ent_buckets.items()
171         if label in ("Person", "Organization", "Location")
172     }
173
174     tone_by_topic = defaultdict(lambda: Counter())
175     for r in corpus:
176         tone_by_topic[r.get("topic", "?")][r.get("sentiment", "Neutral")] += 1
177
178     payload = {
179         "meta": {
180             "corpus_size": len(corpus),
181             "index_size": len(docs),
182             "mean_inflected_ratio": round(
183                 sum(r.get("inflected_ratio", 0) for r in corpus) / len(corpus), 4),
184             "source": "XL-Sum Marathi (BBC Marathi)",
185         },
186         "morph": morph_tables(),
187         "showcase": show_out,
188         "docs": [slim(r) for r in docs],
189         "topics": Counter(r.get("topic", "?") for r in corpus).most_common(),
190         "sentiment": Counter(r.get("sentiment", "?") for r in corpus).most_common(),
191         "tone_by_topic": {k: dict(v) for k, v in tone_by_topic.items()},
192         "top_entities": top_entities,
193         # The in-browser students (src/lite.py), so pasted text can be
194         # analysed with no Python -- with their measured agreement.
195         "lite": json.loads(LITE_MODELS.read_text(encoding="utf-8")),
196     }
197
198     out = Path(args.out)
199     out.write_text(json.dumps(payload, ensure_ascii=False), encoding="utf-8")
200     print(f"wrote {out} ({out.stat().st_size / 1024:.0f} KB)")
201
202

```

```
203 | if __name__ == "__main__":  
204 |     main()
```

Assembles the single-file demo page.

```

1  # -*- coding: utf-8 -*-
2  """
3  Assemble the standalone demo page, Marathi-NLP-Demo.html.
4
5      python scripts/export_demo.py      # data + model weights -> outputs/demo_data.json
6      python scripts/build_demo_page.py  # -> Marathi-NLP-Demo.html
7      python scripts/verify_demo.py     # JavaScript vs Python parity
8
9  The page is a single self-contained HTML file: the JSON payload goes into a
10 <script type="application/json"> tag rather than being fetched, and the
11 engine (app/demo_engine.js) is inlined, so the page works offline and can be
12 emailed or opened from a pendrive.
13 """
14 from __future__ import annotations
15
16 import json
17 import sys
18 from pathlib import Path
19
20 sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
21 sys.stdout.reconfigure(encoding="utf-8")
22
23 import config # noqa: E402
24
25 TEMPLATE = config.ROOT / "app" / "demo_template.html"
26 ENGINE = config.ROOT / "app" / "demo_engine.js"
27 DATA = config.OUTPUTS / "demo_data.json"
28 OUT = config.ROOT / "Marathi-NLP-Demo.html"
29
30
31 def main():
32     if not DATA.exists():
33         sys.exit("run scripts/export_demo.py first")
34     payload = json.loads(DATA.read_text(encoding="utf-8"))
35
36     # A literal </script> inside the JSON would close the tag early;
37     # escaping the slash keeps the JSON valid but hides it from the
38     # HTML parser.
39     blob = json.dumps(payload, ensure_ascii=False, separators=(",", ":")) \
40         .replace("</", "<" + chr(92) + "/")
41     engine = ENGINE.read_text(encoding="utf-8")
42     if "</script" in engine:
43         sys.exit("demo_engine.js must not contain '</script'")
44
45     html = TEMPLATE.read_text(encoding="utf-8")
46     for marker in ("__DATA__", "/*__ENGINE__*/"):
47         if html.count(marker) != 1:
48             sys.exit(f"template must contain {marker} exactly once")
49     html = html.replace("__DATA__", blob).replace("/*__ENGINE__*/", engine)
50     OUT.write_text(html, encoding="utf-8")
51     print(f"wrote {OUT} ({OUT.stat().st_size/1024:.0f} KB)")
52
53
54 if __name__ == "__main__":
55     main()

```

Checks the page's JavaScript against the Python.

```

1  # -*- coding: utf-8 -*-
2  """
3  Check that the demo page's JavaScript gives the same answers as the Python.
4
5      python scripts/verify_demo.py          (needs node on PATH)
6      python scripts/verify_demo.py --n 1000 # more test articles
7
8  The page re-implements the analyser, TextRank and the three in-browser
9  students (src/lite.py) in JavaScript. That port is only trustworthy if it
10 agrees with the Python, so this script pulls the engine and the embedded
11 payload *out of the built page itself* -- i.e. exactly what ships -- runs
12 them under Node, and compares, stage by stage:
13
14     I roots of every Devanagari word in the page's search corpus
15     E TextRank summaries of held-out XL-Sum test articles
16     D topic probabilities                (same label, |diff| < 1e-6)
17     F sentence sentiment probabilities    (same label, |diff| < 1e-6)
18     G entity spans and the grouped entity lists
19     and the whole pipeline() output
20 """
21 from __future__ import annotations
22
23 import argparse
24 import json
25 import re
26 import subprocess
27 import sys
28 import tempfile
29 from pathlib import Path
30
31 sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
32 sys.stdout.reconfigure(encoding="utf-8")
33
34 import config                # noqa: E402
35 from src import lite          # noqa: E402
36 from src.morph import sent_tokenize, stem # noqa: E402
37 from src.summarize import summarize # noqa: E402
38
39 PAGE = config.ROOT / "Marathi-NLP-Demo.html"
40 TOL = 1e-6
41
42 RUNNER = r"""
43 const fs = require("fs"), vm = require("vm");
44 const [enginePath, payloadPath, casesPath] = process.argv.slice(2);
45 const ctx = { atob: s => Buffer.from(s, "base64").toString("latin1") };
46 vm.createContext(ctx);
47 vm.runInContext(fs.readFileSync(enginePath, "utf8") + "\nthis.makeEngine = makeEngine;", ctx);
48 const E = ctx.makeEngine(JSON.parse(fs.readFileSync(payloadPath, "utf8")));
49 const C = JSON.parse(fs.readFileSync(casesPath, "utf8"));
50 const out = {
51   stems: C.words.map(w => E.stem(w)),
52   summaries: C.docs.map(d => E.summarize(d.text, 3).join(" ")),
53   topics: C.docs.map(d => E.topic(d.title, d.text)),
54   sentiment: C.sentences.map(s => E.sentenceSentiment(s)),
55   spans: C.sentences.map(s => E.entitySpans(s).map(([a, b, l]) => [s.slice(a, b), l])),
56   pipeline: C.docs.map(d => {
57     const r = E.pipeline(d.title, d.text);
58     return { topic_scores: r.topic_scores, summary: r.summary, entities: r.entities,
59             sentiment: r.sentiment, polarity: r.polarity,
60             sentiment_distribution: r.sentiment_distribution };
61   }),
62 };
63 process.stdout.write(JSON.stringify(out));
64 """

```

```

65
66
67 def extract(html: str):
68     m = re.search(r'<script id="payload" type="application/json">(.*?)</script>', html, re.S)
69     if not m:
70         sys.exit("no embedded payload found in the page")
71     payload = json.loads(m.group(1).replace("<" + chr(92) + "/", "</"))
72     engines = [s for s in re.findall(r"<script>(.*?)</script>", html, re.S)
73                 if "function makeEngine" in s]
74     if len(engines) != 1:
75         sys.exit("could not find the inlined engine in the page")
76     return payload, engines[0]
77
78
79 def close(a: dict, b: dict) -> bool:
80     return (max(a, key=a.get) == max(b, key=b.get)
81             and all(abs(a[k] - b[k]) < TOL for k in a))
82
83
84 def main():
85     ap = argparse.ArgumentParser()
86     ap.add_argument("--n", type=int, default=300, help="held-out test articles")
87     args = ap.parse_args()
88
89     if not PAGE.exists():
90         sys.exit(f"{PAGE} not found -- run scripts/build_demo_page.py first")
91     payload, engine = extract(PAGE.read_text(encoding="utf-8"))
92     model = lite.Lite(payload["lite"])
93
94     words = set()
95     for rec in payload["docs"]:
96         words.update(re.findall(r"[^\s]+", rec["title"] + " " + rec["summary"]))
97     words = sorted(words)
98
99     docs = []
100     with open(config.XLSUM_TEST, encoding="utf-8") as fh:
101         for line in fh:
102             r = json.loads(line)
103             if len(r["text"]) >= 200:
104                 docs.append({"title": r["title"], "text": r["text"]})
105             if len(docs) >= args.n:
106                 break
107     sentences = [s for d in docs for s in [d["title"], *sent_tokenize(d["text"])[6]]]
108     print(f"checking {len(words):,} words, {len(docs)} articles, "
109           f"{len(sentences):,} sentences")
110
111     with tempfile.TemporaryDirectory() as td:
112         td = Path(td)
113         (td / "engine.js").write_text(engine, encoding="utf-8")
114         (td / "payload.json").write_text(json.dumps(payload, ensure_ascii=False), encoding="utf-8")
115         (td / "cases.json").write_text(json.dumps(
116             {"words": words, "docs": docs, "sentences": sentences}, ensure_ascii=False),
117             encoding="utf-8")
118         (td / "run.js").write_text(RUNNER, encoding="utf-8")
119         try:
120             r = subprocess.run(["node", str(td / "run.js"), str(td / "engine.js"),
121                                str(td / "payload.json"), str(td / "cases.json")],
122                                capture_output=True, text=True, encoding="utf-8")
123         except FileNotFoundError:
124             sys.exit("node not found on PATH -- install Node.js to run this check")
125     if r.returncode != 0:
126         sys.exit("node failed:\n" + r.stderr)
127     js = json.loads(r.stdout)
128
129     checks = {}
130
131     def record(name, bad, total, example=None):
132         checks[name] = (bad, total)
133         if bad and example:

```

```

134         print(f"    first mismatch in {name}: {example}")
135
136     bad = [(w, j) for w, j in zip(words, js["stems"]) if stem(w) != j]
137     record("I  roots", len(bad), len(words), bad[:1])
138
139     py_sum = [summarize(d["text"], k=3).text for d in docs]
140     bad = [i for i, (p, j) in enumerate(zip(py_sum, js["summaries"])) if p != j]
141     record("E  summaries", len(bad), len(docs), bad[:1])
142
143     bad = [i for i, (d, j) in enumerate(zip(docs, js["topics"]))
144           if not close(model.topic(d["title"], d["text"]), j)]
145     record("D  topic", len(bad), len(docs), bad[:1])
146
147     bad = [s for s, j in zip(sentences, js["sentiment"])
148           if not close(model.sentence_sentiment(s), j)]
149     record("F  sentiment", len(bad), len(sentences), bad[:1])
150
151     bad = []
152     for s, j in zip(sentences, js["spans"]):
153         py = [[s[a:b], lab] for a, b, lab in model.entities(s)]
154         if py != j:
155             bad.append((s, py, j))
156     record("G  entity spans", len(bad), len(sentences), bad[:1])
157
158     bad = []
159     for d, j in zip(docs, js["pipeline"]):
160         p = lite.analyse(model, d["title"], d["text"])
161         same = (p["summary"] == j["summary"] and p["entities"] == j["entities"]
162               and p["sentiment"] == j["sentiment"]
163               and p["sentiment_distribution"] == j["sentiment_distribution"]
164               and abs(p["polarity"] - j["polarity"]) < TOL
165               and close(p["topic_scores"], j["topic_scores"]))
166         if not same:
167             bad.append(d["title"])
168     record("  whole pipeline", len(bad), len(docs), bad[:1])
169
170     failed = False
171     for name, (b, n) in checks.items():
172         print(f"    {name:18s} {n - b:>6,} / {n:,} identical")
173         failed |= b > 0
174     if failed:
175         sys.exit("PARITY FAILED: the page disagrees with the Python modules")
176     print("PARITY OK -- the page computes exactly what the Python modules compute")
177
178
179 if __name__ == "__main__":
180     main()

```

Applications — app/

app/dashboard.py

330 lines

Streamlit dashboard.

```
1  # -*- coding: utf-8 -*-
2  """
3  Marathi News Intelligence -- Streamlit dashboard.
4
5      streamlit run app/dashboard.py
6
7  Five tabs, one per part of the pipeline:
8      Search      -- morphology-aware hybrid retrieval over the enriched corpus (C)
9      Overview    -- topic and sentiment distribution, most-covered entities (D/F/G)
10     Entity tone -- how a given person/party is being covered (F over G)
11     Morphology  -- the analyser, and a side-by-side plain vs morph search (I)
12     Analyse     -- run the full pipeline on pasted text or live RSS (A/D/E/F/G)
13
14     Heavy objects are cached: the corpus and indexes load once per session, and
15     each transformer is held by src.models for the process lifetime.
16     """
17     from __future__ import annotations
18
19     import json
20     import sys
21     from collections import Counter, defaultdict
22     from pathlib import Path
23
24     import pandas as pd
25     import streamlit as st
26
27     sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
28
29     import config                                # noqa: E402
30     from src.morph import analyse, normalize_tokens # noqa: E402
31     from src.ner import merge_aliases              # noqa: E402
32     from src.retrieve import BM25Index, HybridRetriever # noqa: E402
33
34     st.set_page_config(page_title="Marathi News Intelligence", page_icon="📰",
35                        layout="wide")
36
37     TOPIC_COLORS = {
38         "Politics": "#c0392b", "Sports": "#27ae60", "Tech": "#2980b9",
39         "Manoranjana": "#8e44ad", "Crime": "#7f8c8d", "Health": "#16a085",
40         "International": "#d35400", "Education": "#2c3e50", "Auto": "#34495e",
41         "Bhakti": "#b7950b", "Fashion": "#e91e63", "Travel": "#00838f",
42         "state": "#c0392b", "sports": "#27ae60", "entertainment": "#8e44ad",
43     }
44     SENT_ICON = {"Positive": "🟢", "Neutral": "⚪", "Negative": "🔴"}
45
46
47     # -----
48     # Loading
49     # -----
50     @st.cache_data(show_spinner=False)
51     def load_corpus():
52         if not config.CORPUS_JSONL.exists():
53             return []
54         with open(config.CORPUS_JSONL, encoding="utf-8") as fh:
55             return [json.loads(line) for line in fh]
56
57
58     @st.cache_resource(show_spinner=False)
59     def load_retriever():
```

```

60     return HybridRetriever.load_default()
61
62
63 @st.cache_resource(show_spinner=False)
64 def load_plain_index(corpus_len: int):
65     """Morphology-free index, built lazily for the side-by-side comparison."""
66     corpus = load_corpus()
67     texts = [f"{r.get('title','')} {r.get('text','')}" for r in corpus]
68     return BM25Index(use_morph=False).build(texts, list(range(len(corpus))))
69
70
71 @st.cache_data(show_spinner=False)
72 def entity_counts():
73     """Corpus-wide entity tallies, with short-form aliases folded in."""
74     buckets = defaultdict(Counter)
75     for r in load_corpus():
76         for label, vals in (r.get("entities") or {}).items():
77             for v in vals:
78                 buckets[label][v] += 1
79     return {label: merge_aliases(c) for label, c in buckets.items()}
80
81
82 def topic_badge(topic: str) -> str:
83     color = TOPIC_COLORS.get(topic, "#555")
84     return (f"<span style='background:{color};color:#fff;padding:2px 9px;"
85           f"border-radius:11px;font-size:0.76rem;font-weight:600'>{topic}</span>")
86
87
88 def render_article(rec: dict, score: float = None, key_prefix: str = ""):
89     """One result card."""
90     sent = rec.get("sentiment", "Neutral")
91     with st.container(border=True):
92         st.markdown(
93             f"{topic_badge(rec.get('topic','?'))} &nbsp; "
94             f"<b>{rec.get('title','(untitled)')}</b>",
95             unsafe_allow_html=True)
96
97         meta = [f"tone **{sent}** ({rec.get('polarity',0):+.2f})"]
98         if score is not None:
99             meta.append(f"score `{score:.3f}`")
100         if rec.get("tokens"):
101             meta.append(f"{rec['tokens']} tokens, "
102                       f"{rec.get('inflected_ratio',0):.0%} inflected")
103         st.caption(" · ".join(meta))
104
105         if rec.get("summary"):
106             st.write(rec["summary"])
107         if rec.get("summary_en"):
108             st.caption("🌐 " + rec["summary_en"])
109
110         ents = rec.get("entities") or {}
111         if ents:
112             chips = []
113             for label in ("Person", "Organization", "Location", "Date", "Designation"):
114                 for v in (ents.get(label) or [])[:4]:
115                     chips.append(f"`{label[:3]}: {v}`")
116             if chips:
117                 st.markdown(" · ".join(chips))
118         if rec.get("url"):
119             st.caption(f"[source]({rec['url']})")
120
121
122 # -----
123 corpus = load_corpus()
124
125 st.title("📰 Marathi News Intelligence")
126 st.caption("Morphology-aware retrieval, categorisation, summarisation, "
127           "entity extraction, sentiment and translation over Marathi news.")
128

```

```

129 if not corpus:
130     st.error("No corpus found. Build one first:\n\n"
131             "```\npython scripts/build_corpus.py --limit 3000\n"
132             "python scripts/build_index.py --plain-bm25\n```")
133     st.stop()
134
135 tab_search, tab_overview, tab_entity, tab_morph, tab_live = st.tabs(
136     ["🔍 Search", "📊 Overview", "🗂 Entity tone", "🔤 Morphology", "📈 Analyse"])
137
138
139 # ----- search -----
140 with tab_search:
141     c1, c2, c3 = st.columns([5, 1.4, 1.4])
142     query = c1.text_input("Search the corpus (Marathi)", value="शाळा",
143                           key="q", label_visibility="collapsed",
144                           placeholder="उदा. शाळा, निवडणूक, क्रिकेट")
145     mode = c2.selectbox("Mode", ["hybrid", "bm25", "dense"],
146                        label_visibility="collapsed")
147     top_k = c3.number_input("Top-k", 1, 50, 10, label_visibility="collapsed")
148
149     topics = sorted({r.get("topic", "?") for r in corpus})
150     f1, f2 = st.columns(2)
151     want_topics = f1.multiselect("Filter by topic", topics, default=[])
152     want_sent = f2.multiselect("Filter by tone",
153                               ["Positive", "Neutral", "Negative"], default=[])
154
155     if query:
156         retriever = load_retriever()
157         if retriever.bm25 is None and retriever.dense is None:
158             st.warning("No index found. Run `python scripts/build_index.py`.")
159         else:
160             use_mode = mode
161             if mode == "dense" and retriever.dense is None:
162                 st.info("No dense index; falling back to BM25.")
163                 use_mode = "bm25"
164             hits = retriever.search(query, top_k=int(top_k) * 3, mode=use_mode)
165
166             shown = 0
167             for h in hits:
168                 rec = corpus[h.doc_id]
169                 if want_topics and rec.get("topic") not in want_topics:
170                     continue
171                 if want_sent and rec.get("sentiment") not in want_sent:
172                     continue
173                 render_article(rec, score=h.score)
174                 shown += 1
175                 if shown >= top_k:
176                     break
177             if shown == 0:
178                 st.info("No matching documents.")
179             else:
180                 st.caption(f"{shown} result(s) · mode = {use_mode} · "
181                           f"query normalised to `{' '.join(normalize_tokens(query))}`")
182
183
184 # ----- overview -----
185 with tab_overview:
186     st.subheader(f"Corpus overview — {len(corpus):,} articles")
187
188     topic_counts = Counter(r.get("topic", "?") for r in corpus)
189     sent_counts = Counter(r.get("sentiment", "?") for r in corpus)
190
191     c1, c2 = st.columns(2)
192     with c1:
193         st.markdown("**Topic distribution** (module D)")
194         st.bar_chart(pd.Series(topic_counts).sort_values(ascending=False))
195     with c2:
196         st.markdown("**Tone distribution** (module F)")
197         st.bar_chart(pd.Series(sent_counts))

```

```

198
199 st.markdown("**Tone by topic**")
200 rows = defaultdict(lambda: {"Positive": 0, "Neutral": 0, "Negative": 0})
201 for r in corpus:
202     rows[r.get("topic", "?")][r.get("sentiment", "Neutral")] += 1
203 st.dataframe(pd.DataFrame(rows).T.sort_values("Negative", ascending=False),
204             use_container_width=True)
205
206 st.markdown("**Most-covered entities** (module G, de-inflected by module I)")
207 buckets = entity_counts()
208 cols = st.columns(4)
209 for col, label in zip(cols, ["Person", "Organization", "Location", "Date"]):
210     with col:
211         st.caption(label)
212         top = buckets[label].most_common(12)
213         if top:
214             st.dataframe(pd.DataFrame(top, columns=[label, "n"]),
215                         hide_index=True, use_container_width=True)
216
217 infl = [r.get("inflected_ratio", 0) for r in corpus if r.get("tokens")]
218 if infl:
219     st.metric("Mean inflected-token ratio",
220             f"{sum(infl)/len(infl):.1%}",
221             help="Share of tokens carrying at least one strippable "
222                 "suffix – the reason plain keyword search underperforms.")
223
224
225 # ----- entity tone --
226 with tab_entity:
227     st.subheader("How is an entity being covered?")
228     st.caption("Mean tone of the articles each entity appears in "
229             "(module F aggregated over module G).")
230
231     buckets = entity_counts()
232
233     label = st.selectbox("Entity type",
234                         ["Person", "Organization", "Location", "Designation"])
235     options = [e for e, n in buckets[label].most_common(150) if n >= 2]
236     if not options:
237         st.info("Not enough repeated entities of this type in the corpus.")
238     else:
239         ent = st.selectbox("Entity", options)
240         mentions = [r for r in corpus
241                     if ent in (r.get("entities") or {}).get(label, [])]
242         pols = [r.get("polarity", 0.0) for r in mentions]
243         mean_pol = sum(pols) / len(pols) if pols else 0.0
244
245         m1, m2, m3 = st.columns(3)
246         m1.metric("Articles", len(mentions))
247         m2.metric("Mean tone", f"{mean_pol:+.3f}")
248         m3.metric("Dominant topic",
249                 Counter(r.get("topic", "?") for r in mentions).most_common(1)[0][0])
250
251         st.bar_chart(pd.Series(Counter(r.get("sentiment", "?") for r in mentions)))
252         st.markdown("**Articles mentioning this entity**")
253         for r in mentions[:15]:
254             render_article(r)
255
256
257 # ----- morphology --
258 with tab_morph:
259     st.subheader("Morphological analyser (module I)")
260
261     word = st.text_input("Analyse a Marathi word", value="घरामधून")
262     if word:
263         a = analyse(word.strip())
264         c1, c2, c3 = st.columns(3)
265         c1.metric("Root", a.root)
266         c2.metric("Features", "+".join(a.features) or "-")

```

```

267         c3.metric("Suffixes stripped", " + ".join(a.stripped) or "-")
268
269     st.divider()
270     st.markdown("**Why it matters for search**")
271     st.caption("The same query run against a morphology-normalised index and a "
272               "plain whitespace index over the identical corpus.")
273
274     q = st.text_input("Query", value="३११", key="morph_q")
275     if q:
276         retriever = load_retriever()
277         plain = load_plain_index(len(corpus))
278         c1, c2 = st.columns(2)
279         with c1:
280             st.markdown("❌ **Plain tokens**")
281             hits = plain.search(q, top_k=5)
282             st.caption(f"{len(hits)} hit(s) · indexed terms: "
283                       f"`{' '.join(normalize_tokens(q, use_morph=False))}`")
284             for h in hits:
285                 st.write(f"• {corpus[h.doc_id].get('title','')[:90]}")
286             if not hits:
287                 st.info("No documents matched.")
288         with c2:
289             st.markdown("✅ **Morphology-normalised**")
290             hits = retriever.bm25.search(q, top_k=5) if retriever.bm25 else []
291             st.caption(f"{len(hits)} hit(s) · indexed terms: "
292                       f"`{' '.join(normalize_tokens(q))}`")
293             for h in hits:
294                 st.write(f"• {corpus[h.doc_id].get('title','')[:90]}")
295             if not hits:
296                 st.info("No documents matched.")
297
298
299     # ----- live --
300     with tab_live:
301         st.subheader("Run the full pipeline on new text")
302         src = st.radio("Input", ["Paste text", "Fetch live RSS"], horizontal=True)
303         do_translate = st.checkbox("Also translate to English (module A, slow)",
304                                   value=False)
305
306         if src == "Paste text":
307             title = st.text_input("Headline (optional)")
308             body = st.text_area("Marathi article text", height=220)
309             if st.button("Analyse", type="primary") and body.strip():
310                 from src.pipeline import process
311                 with st.spinner("Running pipeline..."):
312                     doc = process(title, body, translate=do_translate)
313                     render_article(doc.to_dict())
314                 with st.expander("Full topic distribution"):
315                     st.bar_chart(pd.Series(doc.topic_scores).sort_values(ascending=False))
316         else:
317             n = st.slider("Articles to fetch", 3, 21, 6, step=3)
318             if st.button("Fetch & analyse", type="primary"):
319                 from src.ingest import fetch_all
320                 from src.pipeline import process
321                 with st.spinner("Fetching feeds..."):
322                     rows = fetch_all(limit=n, with_body=True)
323                     st.success(f"Fetched {len(rows)} articles")
324                     bar = st.progress(0.0)
325                     for i, r in enumerate(rows, 1):
326                         doc = process(r["title"], r["text"], url=r.get("url", ""),
327                                       translate=do_translate)
328                         render_article(doc.to_dict())
329                         bar.progress(i / len(rows))

```

Local model server the demo page can use.

```

1  # -*- coding: utf-8 -*-
2  """
3  Local model server for the demo page.
4
5      python app/server.py          # http://127.0.0.1:8765
6
7  The demo page (Marathi-NLP-Demo.html) always works on its own, using the
8  small in-browser models from src/lite.py. When this server is running, the
9  page detects it and sends pasted text here instead, so the full pipeline --
10 MahaBERT topic model, MahaNER, the sentiment model and NLLB-200 -- analyses
11 it. Nothing else changes on the page.
12
13 Endpoints (JSON):
14 GET /api/health          -> {"ok": true, "device": ...}
15 POST /api/analyse {title, text, translate}
16                                     -> the pipeline's Document as a dict
17 POST /api/translate {texts: [...]} -> {"translations": [...]}
18
19 Standard library only. It listens on 127.0.0.1, so it is reachable from this
20 machine and nowhere else.
21 """
22 from __future__ import annotations
23
24 import json
25 import os
26 import sys
27 import threading
28 from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
29 from pathlib import Path
30
31 sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
32 sys.stdout.reconfigure(encoding="utf-8")
33
34 import config # noqa: E402
35
36 HOST = "127.0.0.1"
37 PORT = int(os.environ.get("MNIP_PORT", 8765))
38 MAX_CHARS = 20000
39
40 # The models are not thread-safe to load concurrently, and one request at a
41 # time is plenty for a demo.
42 _LOCK = threading.Lock()
43
44
45 def analyse(payload: dict) -> dict:
46     from src.pipeline import process
47     title = str(payload.get("title") or "")[:1000]
48     text = str(payload.get("text") or "")[:MAX_CHARS]
49     if not text.strip():
50         raise ValueError("text is empty")
51     with _LOCK:
52         doc = process(title, text, translate=bool(payload.get("translate")))
53     return doc.to_dict()
54
55
56 def translate(payload: dict) -> dict:
57     from src import translate as mt
58     texts = [str(t)[:2000] for t in (payload.get("texts") or [])[:40]]
59     if not texts:
60         return {"translations": []}
61     with _LOCK:
62         return {"translations": mt.translate(texts)}
63
64

```

```

65 class Handler(BaseHTTPRequestHandler):
66     server_version = "MarathiNLP/1.0"
67
68     def _headers(self, status=200, body_len=0):
69         self.send_response(status)
70         self.send_header("Content-Type", "application/json; charset=utf-8")
71         # The page is usually opened from file://, whose origin is "null".
72         self.send_header("Access-Control-Allow-Origin", "*")
73         self.send_header("Access-Control-Allow-Methods", "GET, POST, OPTIONS")
74         self.send_header("Access-Control-Allow-Headers", "Content-Type")
75         # Chrome's Private Network Access preflight for pages on https://.
76         self.send_header("Access-Control-Allow-Private-Network", "true")
77         self.send_header("Content-Length", str(body_len))
78         self.end_headers()
79
80     def _json(self, obj, status=200):
81         body = json.dumps(obj, ensure_ascii=False).encode("utf-8")
82         self._headers(status, len(body))
83         self.wfile.write(body)
84
85     def do_OPTIONS(self): # noqa: N802
86         self._headers(204)
87
88     def do_GET(self): # noqa: N802
89         if self.path.rstrip("/") == "/api/health":
90             self._json({"ok": True, "device": config.DEVICE,
91                         "models": {"topic": config.MODEL_TOPIC,
92                                   "ner": config.MODEL_NER,
93                                   "sentiment": config.MODEL_SENTIMENT,
94                                   "translation": config.MODEL_TRANSLATE}})
95         else:
96             self._json({"error": "not found"}, 404)
97
98     def do_POST(self): # noqa: N802
99         routes = {"api/analyse": analyse, "api/translate": translate}
100         fn = routes.get(self.path.rstrip("/"))
101         if fn is None:
102             return self._json({"error": "not found"}, 404)
103         try:
104             n = int(self.headers.get("Content-Length") or 0)
105             payload = json.loads(self.rfile.read(n) or b"{}".format(""))
106             self._json(fn(payload))
107         except ValueError as exc:
108             self._json({"error": str(exc)}, 400)
109         except Exception as exc: # noqa: BLE001
110             self._json({"error": f"type({exc}).__name__: {exc}"}, 500)
111
112     def log_message(self, fmt, *args):
113         sys.stderr.write(" %s %s\n" % (self.command, self.path))
114
115
116 def main():
117     print(f"Marathi NLP model server on http://{HOST}:{PORT} (device: {config.DEVICE})")
118     print("Open Marathi-NLP-Demo.html -- it will switch to the full models.")
119     print("Models load on the first request (the first one takes a while).")
120     ThreadingHTTPServer((HOST, PORT), Handler).serve_forever()
121
122
123 if __name__ == "__main__":
124     main()

```

The demo page's NLP engine: a JavaScript port of the modules above.

```

1  /*
2   * Marathi NLP engine for the demo page -- a JavaScript port of the Python
3   * reference implementations, run in the browser on whatever text is pasted.
4   *
5   * I analyse / stem      src/morph.py
6   * C BM25                src/retrieve.py
7   * E TextRank + MMR      src/summarize.py
8   * D topic student       src/lite.py (imitates marathi-topic-all-doc)
9   * F sentiment student   src/lite.py (imitates marathi-sentiment-md)
10  * G entity tagger student src/lite.py (imitates marathi-ner)
11  *
12  * build_demo_page.py inlines this file into the page, and verify_demo.py
13  * loads this same file in Node and checks it against the Python modules, so
14  * what is checked is exactly what ships.
15  */
16  function makeEngine(DATA) {
17    "use strict";
18    const M = DATA.morph;
19
20    /* ===== I: morphological analyser (src/morph.py) ===== */
21    const MARKS = new Set(Array.from(M.marks));
22    const FINAL = new Set(Array.from(M.finalMatras));
23    const STOP = new Set(M.stopwords);
24    const byLen = t => t.slice().sort((a, b) => b[0].length - a[0].length);
25    const LAYERS = [byLen(M.clitics), byLen(M.postpositions), byLen(M.verb), byLen(M.case)];
26    const aksh = s => { let n = 0; for (const c of s) if (!MARKS.has(c)) n++; return n; };
27
28    function restore(s) {
29      while (s.endsWith(M.virama)) s = s.slice(0, -1);
30      if (s && FINAL.has(s[s.length - 1]) && aksh(s) >= M.minStem) s = s.slice(0, -1);
31      return s;
32    }
33    function strip1(w) {
34      for (const table of LAYERS) for (const [suf, feat, repl] of table) {
35        if (!w.endsWith(suf)) continue;
36        const c = w.slice(0, w.length - suf.length) + repl;
37        if (aksh(c) >= M.minStem) return [c, suf, feat];
38      }
39      return null;
40    }
41    const cache = new Map();
42    function analyse(word) {
43      if (cache.has(word)) return cache.get(word);
44      let w = word.normalize("NFC"), res;
45      if (Object.prototype.hasOwnProperty.call(M.exceptions, word)) {
46        res = { root: M.exceptions[word], features: [], stripped: [] };
47      } else {
48        const features = [], stripped = [];
49        for (let i = 0; i < M.maxStrips; i++) {
50          const h = strip1(w); if (!h) break;
51          w = h[0]; stripped.push(h[1]); features.push(h[2]);
52        }
53        res = { root: restore(w), features, stripped };
54      }
55      cache.set(word, res);
56      return res;
57    }
58    const stem = w => analyse(w).root;
59    const tokenize = t => (t || "").match(/[\u0900-\u097F]+|[A-Za-z]+|\d+/g) || [];
60    const isAscii = t => /^[^\x00-\x7F]+$/.test(t);
61    function normTokens(text, useMorph = true) {
62      const out = [];
63      for (const tok of tokenize(text)) {
64        if (STOP.has(tok)) continue;

```

```

65     const t = useMorph ? stem(tok) : tok;
66     if (STOP.has(t)) continue;
67     if (aksh(t) >= M.minStem || isAscii(t)) out.push(t);
68 }
69 return out;
70 }
71 function sentTokenize(text) {
72     return (text || "").trim().split(/(?<=[!.?])\s+/)
73         .map(s => s.trim()).filter(Boolean);
74 }
75
76 /* ===== C: BM25 (src/retrieve.py) ===== */
77 const K1 = 1.5, B = 0.75;
78 function buildIndex(texts, useMorph) {
79     const postings = new Map(), len = [];
80     texts.forEach((txt, i) => {
81         const toks = normTokens(txt, useMorph); len.push(toks.length);
82         const tf = new Map();
83         for (const t of toks) tf.set(t, (tf.get(t) || 0) + 1);
84         for (const [t, f] of tf) {
85             if (!postings.has(t)) postings.set(t, []);
86             postings.get(t).push([i, f]);
87         }
88     });
89     const avg = len.length ? len.reduce((a, b) => a + b, 0) / len.length : 0;
90     return { postings, len, avg, n: texts.length, useMorph };
91 }
92 function search(idx, q, topK) {
93     const sc = new Float64Array(idx.n);
94     for (const term of normTokens(q, idx.useMorph)) {
95         const p = idx.postings.get(term); if (!p) continue;
96         const idf = Math.max(Math.log((idx.n - p.length + 0.5) / (p.length + 0.5) + 1), 0);
97         for (const [d, f] of p) {
98             const dn = f + K1 * (1 - B + B * idx.len[d] / Math.max(idx.avg, 1e-9));
99             sc[d] += idf * (f * (K1 + 1)) / Math.max(dn, 1e-9);
100         }
101     }
102     const hits = [];
103     for (let i = 0; i < idx.n; i++) if (sc[i] > 0) hits.push([i, sc[i]]);
104     hits.sort((a, b) => b[1] - a[1]);
105     return topK ? hits.slice(0, topK) : hits;
106 }
107
108 /* ===== E: TextRank + MMR (src/summarize.py) ===== */
109 const DAMPING = 0.85, MAX_ITER = 100, TOL = 1e-6;
110 function summarize(text, k = 3, lambda = 0.7) {
111     const sentences = sentTokenize(text)
112         .filter(s => s.split(/\s+/).filter(Boolean).length >= 3);
113     if (!sentences.length) return [];
114     if (sentences.length <= k) return sentences;
115
116     // morphologically normalised TF-IDF
117     const docs = sentences.map(s => normTokens(s, true));
118     const vocab = new Map();
119     for (const d of docs) for (const t of d) if (!vocab.has(t)) vocab.set(t, vocab.size);
120     const n = sentences.length, V = Math.max(vocab.size, 1);
121     const X = docs.map(() => new Float64Array(V));
122     if (vocab.size) {
123         docs.forEach((d, i) => { for (const t of d) X[i][vocab.get(t)] += 1; });
124         for (const row of X) {
125             let s = 0; for (const v of row) s += v;
126             s = Math.max(s, 1e-9);
127             for (let j = 0; j < V; j++) row[j] /= s;
128         }
129         for (let j = 0; j < V; j++) {
130             let df = 0; for (const row of X) if (row[j] > 0) df++;
131             const idf = Math.log((1 + n) / (1 + df)) + 1;
132             for (const row of X) row[j] *= idf;
133         }
134     }

```

```

134 }
135 // cosine graph
136 const unit = X.map(row => {
137   let s = 0; for (const v of row) s += v * v;
138   const nrm = Math.max(Math.sqrt(s), 1e-9);
139   return row.map(v => v / nrm);
140 });
141 const sim = unit.map((a, i) => unit.map((b, j) => {
142   if (i === j) return 0;
143   let s = 0; for (let t = 0; t < V; t++) s += a[t] * b[t];
144   return Math.max(s, 0);
145 }));
146 // PageRank by power iteration
147 const T = sim.map(row => {
148   const r = row.reduce((a, b) => a + b, 0);
149   return row.map(v => r > 0 ? v / Math.max(r, 1e-9) : 1 / n);
150 });
151 let scores = new Array(n).fill(1 / n), done = false;
152 for (let it = 0; it < MAX_ITER && !done; it++) {
153   const nxt = new Array(n);
154   for (let j = 0; j < n; j++) {
155     let s = 0; for (let i = 0; i < n; i++) s += T[i][j] * scores[i];
156     nxt[j] = (1 - DAMPING) / n + DAMPING * s;
157   }
158   let diff = 0; for (let j = 0; j < n; j++) diff += Math.abs(nxt[j] - scores[j]);
159   scores = nxt;
160   if (diff < TOL) done = true;
161 }
162 // rounded, then stable: exact ties keep document order (see summarize.py)
163 const r12 = x => Math.round(x * 1e12) / 1e12;
164 const order = scores.map((s, i) => [r12(s), i]).sort((a, b) => b[0] - a[0]).map(x => x[1]);
165 // MMR
166 const rel = new Map(order.map((idx, pos) => [idx, 1 - pos / Math.max(order.length, 1)]));
167 const selected = [], cand = order.slice();
168 while (cand.length && selected.length < k) {
169   let best = null, bestScore = -Infinity;
170   for (const idx of cand) {
171     let red = 0;
172     for (const j of selected) red = Math.max(red, sim[idx][j]);
173     const s = r12(lambda * rel.get(idx) - (1 - lambda) * red);
174     if (s > bestScore) { best = idx; bestScore = s; }
175   }
176   selected.push(best);
177   cand.splice(cand.indexOf(best), 1);
178 }
179 return selected.sort((a, b) => a - b).map(i => sentences[i]);
180 }
181
182 /* ===== D / F / G: students (src/lite.py) ===== */
183 const L = DATA.lite;
184 // int8 weights, row-major, base64 (see quantize() in scripts/train_lite.py)
185 function unpack(b64, cols) {
186   const bin = atob(b64), rows = [];
187   for (let i = 0; i < bin.length; i += cols) {
188     const row = new Int8Array(cols);
189     for (let j = 0; j < cols; j++) row[j] = (bin.charCodeAt(i + j) << 24) >> 24;
190     rows.push(row);
191   }
192   return rows;
193 }
194 const POLARITY = { Negative: -1, Neutral: 0, Positive: 1 };
195 const softmax = z => {
196   const m = Math.max(...z), e = z.map(v => Math.exp(v - m));
197   const s = e.reduce((a, b) => a + b, 0);
198   return e.map(v => v / s);
199 };
200 const argmax = z => { let b = 0; for (let j = 1; j < z.length; j++) if (z[j] > z[b]) b = j; return b;
201 };

```

```

202 const T_ = L.topic, topicRow = new Map(), topicIdf = new Map();
203 const topicW = unpack(T_.W, T_.labels.length);
204 T_.terms.forEach((t, i) => { topicRow.set(t, topicW[i]); topicIdf.set(t, T_.idf[i]); });
205 function topic(title, text) {
206     const joined = title ? `${title}. ${text}` : text;
207     const terms = normTokens(joined, true).slice(0, T_.max_tokens);
208     const tf = new Map();
209     for (const t of terms) if (topicRow.has(t)) tf.set(t, (tf.get(t) || 0) + 1);
210     const vec = new Map();
211     for (const [t, c] of tf) vec.set(t, (1 + Math.log(c)) * topicIdf.get(t));
212     let ss = 0; for (const v of vec.values()) ss += v * v;
213     const norm = Math.sqrt(ss) || 1;
214     const k = T_.labels.length, z = new Array(k).fill(0);
215     for (const [t, v] of vec) {
216         const row = topicRow.get(t);
217         for (let j = 0; j < k; j++) z[j] += row[j] * v / norm;
218     }
219     const p = softmax(z.map((v, j) => v * T_.scale + T_.b[j]));
220     const out = {}; T_.labels.forEach((l, j) => { out[l] = p[j]; });
221     return out;
222 }
223
224 const S_ = L.sentiment, sentRow = new Map();
225 const sentW = unpack(S_.W, S_.labels.length);
226 S_.feats.forEach((f, i) => sentRow.set(f, sentW[i]));
227 function sentimentFeatures(sentence) {
228     const toks = tokenize(sentence), feats = [];
229     for (const t of toks) {
230         feats.push("w:" + t);
231         const r = stem(t);
232         if (r !== t) feats.push("r:" + r);
233     }
234     for (let i = 0; i + 1 < toks.length; i++) feats.push("b:" + toks[i] + "_" + toks[i + 1]);
235     return feats;
236 }
237 function sentenceSentiment(sentence) {
238     const feats = [...new Set(sentimentFeatures(sentence))].filter(f => sentRow.has(f)).sort();
239     const k = S_.labels.length, z = new Array(k).fill(0);
240     const inv = feats.length ? 1 / Math.sqrt(feats.length) : 0;
241     for (const f of feats) { const row = sentRow.get(f); for (let j = 0; j < k; j++) z[j] += row[j] *
inv; }
242     const p = softmax(z.map((v, j) => v * S_.scale + S_.b[j]));
243     const out = {}; S_.labels.forEach((l, j) => { out[l] = p[j]; });
244     return out;
245 }
246 function documentSentiment(text) {
247     const sents = sentTokenize(text);
248     const dist = { Negative: 0, Neutral: 0, Positive: 0 };
249     if (!sents.length) return { label: "Neutral", polarity: 0, distribution: dist, sentences: [] };
250     const scored = sents.map(s => {
251         const p = sentenceSentiment(s);
252         let label = S_.labels[0];
253         for (const l of S_.labels) if (p[l] > p[label]) label = l;
254         let pol = 0; for (const l of S_.labels) pol += POLARITY[l] * p[l];
255         return { text: s, label, polarity: pol };
256     });
257     let num = 0, den = 0;
258     for (const s of scored) { const w = Math.max(Array.from(s.text).length, 1); num += s.polarity * w;
den += w; }
259     const pol = num / den;
260     for (const s of scored) dist[s.label]++;
261     const label = pol > 0.15 ? "Positive" : pol < -0.15 ? "Negative" : "Neutral";
262     return { label, polarity: pol, distribution: dist, sentences: scored };
263 }
264
265 const N_ = L.ner, nerRow = new Map();
266 const nerW = unpack(N_.W, N_.labels.length);
267 N_.feats.forEach((f, i) => nerRow.set(f, nerW[i]));
268 const BOUNDARY = new Set(Array.from(" \t\n\r " + "!,.,:!?\"'()[]{}<>%/\|_---"));

```

```

269 function words(text) {
270   const out = []; let start = null;
271   for (let i = 0; i < text.length; i++) {
272     if (BOUNDARY.has(text[i])) { if (start !== null) { out.push([start, i]); start = null; } }
273     else if (start === null) start = i;
274   }
275   if (start !== null) out.push([start, text.length]);
276   return out;
277 }
278 const shape = w => /^[0-9\u0080-\u00ff]+$/.test(w) ? "digit" : isAscii(w) ? "latin" : "deva";
279 function nerFeatures(toks, i, prev, prev2) {
280   const w = toks[i], r = stem(w);
281   const p = i > 0 ? toks[i - 1] : "<s>";
282   const n = i + 1 < toks.length ? toks[i + 1] : "</s>";
283   const n2 = i + 2 < toks.length ? toks[i + 2] : "</s>";
284   const f = [
285     "bias", "w:" + w, "r:" + r, "s1:" + w.slice(-1), "s2:" + w.slice(-2), "s3:" + w.slice(-3),
286     "p3:" + w.slice(0, 3), "sh:" + shape(w), "sw:" + (STOP.has(w) ? "True" : "False"),
287     "pw:" + p, "pr:" + stem(p), "nw:" + n, "nr:" + stem(n), "nw2:" + n2,
288     "pl:" + prev, "pl2:" + prev + "|" + prev2, "plw:" + prev + "|" + r,
289     "pwn:" + p + "|" + w, "wn:" + w + "|" + n,
290   ];
291   if (i === 0) f.push("first");
292   return f;
293 }
294 function entitySpans(text) {
295   const offs = words(text), toks = offs.map(([s, e]) => text.slice(s, e));
296   const k = N_.labels.length, spans = [];
297   let prev = "0", prev2 = "0";
298   for (let i = 0; i < toks.length; i++) {
299     const z = N_.b.slice();
300     for (const f of nerFeatures(toks, i, prev, prev2)) {
301       const row = nerRow.get(f);
302       if (row) for (let j = 0; j < k; j++) z[j] += row[j] * N_.scale;
303     }
304     const lab = N_.labels[argmax(z)];
305     if (lab !== "0") {
306       const [s, e] = offs[i], last = spans[spans.length - 1];
307       if (last && last[2] === lab && s - last[1] <= 1) last[1] = e;
308       else spans.push([s, e, lab]);
309     }
310     prev2 = prev; prev = lab;
311   }
312   return spans;
313 }
314 function groupEntities(text, spans) {
315   const counts = new Map(), surfaces = new Map();
316   for (const [s, e, lab] of spans) {
317     const surf = text.slice(s, e).trim(); if (!surf) continue;
318     const parts = surf.split(/\s+/);
319     const canon = [...parts.slice(0, -1), stem(parts[parts.length - 1])].join(" ");
320     const key = lab + " " + canon;
321     counts.set(key, (counts.get(key) || 0) + 1);
322     if (!surfaces.has(key)) surfaces.set(key, new Map());
323     const m = surfaces.get(key); m.set(surf, (m.get(surf) || 0) + 1);
324   }
325   const mostCommon = m => [...m.entries()].sort((a, b) => b[1] - a[1]); // stable
326   const out = {};
327   for (const [key] of mostCommon(counts)) {
328     const lab = key.split(" ")[0];
329     (out[lab] = out[lab] || []).push(mostCommon(surfaces.get(key))[0][0]);
330   }
331   return out;
332 }
333
334 /* ===== the lite pipeline (mirrors src/pipeline.process) ===== */
335 function pipeline(title, text) {
336   title = (title || "").trim(); text = (text || "").trim();
337   const topicScores = topic(title, text);

```

```

338     const summarySents = summarize(text, 3);
339     const summary = summarySents.join(" ");
340     const entSource = `${title}. ${summary}`.trim();
341     const entities = groupEntities(entSource, entitySpans(entSource));
342     const sent = documentSentiment(summary || text);
343     const toks = tokenize(text);
344     const inflected = toks.filter(t => analyse(t).stripped.length > 0).length;
345     return {
346         title, topic_scores: topicScores, summary, summary_sentences: summarySents,
347         entities, sentiment: sent.label, polarity: sent.polarity,
348         sentiment_distribution: sent.distribution, sentence_sentiment: sent.sentences,
349         tokens: toks.length, inflected_ratio: toks.length ? inflected / toks.length : 0,
350     };
351 }
352
353 return { analyse, stem, tokenize, normTokens, sentTokenize, buildIndex, search,
354         summarize, topic, sentenceSentiment, documentSentiment, entitySpans,
355         groupEntities, pipeline };
356 }
357
358 if (typeof module !== "undefined" && module.exports) module.exports = { makeEngine };

```

The demo page's layout and interface code.

```

1  <!DOCTYPE html>
2  <html lang="mr">
3  <head>
4  <meta charset="utf-8">
5  <meta name="viewport" content="width=device-width, initial-scale=1">
6  <title>Marathi News Intelligence</title>
7  <link rel="stylesheet" href="https://fonts.googleapis.com/css2?
family=DM+Sans:opsz,wght@9..40,400;9..40,500;9..40,700&family=Tiro+Devanagari+Marathi&family=IBM+Plex+Mono:wght@
400;500&display=swap">
8  <style>
9    /* Light, single-theme palette. Every colour is declared here. */
10   :root{
11     --bg:#f6f9fa; --card:#ffffff; --soft:#eef4f5; --line:#dde7ea;
12     --ink:#1e3238; --ink-soft:#5b757d; --ink-mute:#8ca3a9;
13     --accent:#1f7d7d; --accent-soft:#e2f2f1;
14     --pos:#37785a; --pos-bg:#e6f3ec;
15     --neg:#a85341; --neg-bg:#fbae6;
16     --neu:#7d9299; --neu-bg:#eef2f3;
17     --mono:"IBM Plex Mono",ui-monospace,Menlo,monospace;
18     --deva:"Tiro Devanagari Marathi","Nirmala UI","Noto Sans Devanagari",serif;
19     --sans:"DM Sans",system-ui,-apple-system,Segoe UI,sans-serif;
20     --r:10px;
21   }
22   *{box-sizing:border-box}
23   body{background:var(--bg);color:var(--ink);font-family:var(--sans);
24     font-size:16px;line-height:1.55}
25   .deva{font-family:var(--deva);line-height:1.9}
26   .wrap{max-width:900px;margin:0 auto;padding-inline:20px}
27
28   /* header */
29   .head{padding-block:30px 20px}
30   .title{display:flex;align-items:baseline;gap:10px;flex-wrap:wrap}
31   .title .mx{font-family:var(--deva);color:var(--accent);font-size:1.5rem}
32   .title h1{font-size:1.5rem;margin:0;font-weight:700;letter-spacing:-.02em}
33   .title .meta{font-family:var(--mono);font-size:.7rem;color:var(--ink-mute);
34     margin-left:auto;text-transform:uppercase;letter-spacing:.08em}
35   .head p{color:var(--ink-soft);margin:8px 0 0;font-size:.95rem;max-width:64ch}
36
37   /* tabs */
38   .tabs{display:flex;gap:5px;flex-wrap:wrap;border-bottom:1px solid var(--line);
39     padding-bottom:0;margin-top:22px}
40   .tab{font-family:var(--sans);font-size:.92rem;font-weight:500;border:none;
41     background:none;color:var(--ink-mute);padding:9px 14px;cursor:pointer;
42     border-bottom:2px solid transparent;margin-bottom:-1px}
43   .tab:hover{color:var(--accent)}
44   .tab[aria-selected=true]{color:var(--accent);border-bottom-color:var(--accent);
45     font-weight:700}
46   .tab .n{font-family:var(--mono);font-size:.7rem;opacity:.65;margin-right:5px}
47
48   .panelwrap{padding-block:26px 60px}
49   .lead{color:var(--ink-soft);font-size:.95rem;margin:0 0 18px;max-width:66ch}
50
51   .card{background:var(--card);border:1px solid var(--line);border-radius:var(--r);
52     padding:20px}
53
54   label.fl{display:block;font-family:var(--mono);font-size:.68rem;
55     text-transform:uppercase;letter-spacing:.1em;color:var(--ink-mute);
56     margin-bottom:7px}
57   input[type=text]{font-family:var(--deva);font-size:1.25rem;width:100%;
58     padding:12px 14px;border:1px solid var(--line);border-radius:8px;
59     background:var(--bg);color:var(--ink)}
60   input[type=text]:focus{outline:2px solid var(--accent);outline-offset:1px;
61     border-color:transparent}
62   select{font-family:var(--deva);font-size:1rem;width:100%;padding:11px 13px;

```

```

63     border:1px solid var(--line);border-radius:8px;background:var(--card);
64     color:var(--ink)}
65 select:focus{outline:2px solid var(--accent);outline-offset:1px}
66 .chips{display:flex;gap:6px;flex-wrap:wrap;margin-top:11px}
67 .chip{font-family:var(--deva);font-size:.95rem;padding:4px 12px;border-radius:999px;
68     border:1px solid var(--line);background:var(--card);color:var(--ink-soft);
69     cursor:pointer}
70 .chip:hover{border-color:var(--accent);color:var(--accent)}
71 .chip[aria-pressed=true]{background:var(--accent);color:#fff;border-color:var(--accent)}
72
73 /* gloss */
74 .gloss{display:flex;align-items:flex-end;gap:8px;flex-wrap:wrap;
75     margin-top:24px;min-height:78px}
76 .mb{display:flex;flex-direction:column;align-items:center;gap:6px}
77 .mb .seg{font-family:var(--deva);font-size:1.5rem;padding:8px 15px;border-radius:9px;
78     border:1px solid var(--line);background:var(--soft);line-height:1.5}
79 .mb.root .seg{background:var(--accent);color:#fff;border-color:var(--accent);
80     font-weight:500}
81 .mb .feat{font-family:var(--mono);font-size:.63rem;letter-spacing:.06em;
82     color:var(--ink-mute);text-transform:uppercase}
83 .plus{font-family:var(--mono);color:var(--ink-mute);font-size:1.2rem;padding-bottom:24px}
84 .note{font-size:.9rem;color:var(--ink-soft);margin-top:20px;padding-top:15px;
85     border-top:1px solid var(--line)}
86 .note b{color:var(--ink);font-weight:500}
87
88 .fams{display:grid;grid-template-columns:repeat(auto-fit,minmax(200px,1fr));
89     gap:10px;margin-top:16px}
90 .fam{border:1px solid var(--line);border-radius:8px;padding:11px 13px;
91     background:var(--card)}
92 .fam .f{font-family:var(--deva);font-size:.95rem;line-height:2;color:var(--ink-soft)}
93 .fam .r{font-family:var(--deva);font-size:1.1rem;color:var(--accent);font-weight:500;
94     margin-top:4px}
95 .fam .r::before{content:"→ ";font-family:var(--mono);color:var(--ink-mute);
96     font-size:.85rem}
97
98 /* search */
99 .vs{display:grid;grid-template-columns:1fr 1fr;gap:14px;margin-top:20px}
100 @media(max-width:680px){.vs{grid-template-columns:1fr}}
101 .side{border:1px solid var(--line);border-radius:9px;overflow:hidden;
102     background:var(--card)}
103 .side-h{padding:10px 14px;border-bottom:1px solid var(--line);display:flex;
104     align-items:center;justify-content:space-between;gap:8px}
105 .side-h .nm{font-family:var(--mono);font-size:.68rem;text-transform:uppercase;
106     letter-spacing:.09em}
107 .side.a .side-h{background:var(--neg-bg);color:var(--neg)}
108 .side.b .side-h{background:var(--pos-bg);color:var(--pos)}
109 .cnt{font-family:var(--mono);font-size:1rem;font-weight:500;
110     font-variant-numeric:tabular-nums}
111 .side-b{max-height:300px;overflow-y:auto}
112 .res{padding:10px 14px;border-bottom:1px solid var(--line)}
113 .res:last-child{border-bottom:none}
114 .res .t{font-family:var(--deva);font-size:.95rem;line-height:1.7;display:block}
115 .res .m{font-family:var(--mono);font-size:.63rem;color:var(--ink-mute);margin-top:3px;
116     text-transform:uppercase;letter-spacing:.05em;display:block}
117 .none{padding:34px 14px;text-align:center;color:var(--ink-mute);font-size:.88rem}
118 .none b{display:block;font-family:var(--mono);font-size:1.7rem;color:var(--neg);
119     margin-bottom:6px;font-weight:500}
120
121 /* pipeline */
122 .stage{border:1px solid var(--line);border-radius:9px;margin-top:11px;
123     background:var(--card);overflow:hidden}
124 .stage-h{display:flex;align-items:center;gap:9px;padding:9px 14px;
125     background:var(--soft);border-bottom:1px solid var(--line)}
126 .bdg{font-family:var(--mono);font-size:.66rem;font-weight:500;width:20px;height:20px;
127     display:grid;place-items:center;border-radius:5px;background:var(--accent);
128     color:#fff;flex:none}
129 .stage-n{font-family:var(--mono);font-size:.68rem;text-transform:uppercase;
130     letter-spacing:.1em;color:var(--ink-soft)}
131 .stage-b{padding:15px 14px}

```

```

132 .kv{display:flex;justify-content:space-between;gap:12px;font-size:.9rem;
133   align-items:baseline}
134 .kv .v{font-family:var(--mono);font-size:.78rem;color:var(--ink-mute);
135   font-variant-numeric:tabular-nums}
136 .bar{height:6px;background:var(--soft);border-radius:99px;overflow:hidden;
137   margin-top:5px}
138 .bar i{display:block;height:100%;background:var(--accent)}
139 .lab{font-family:var(--mono);font-size:.63rem;text-transform:uppercase;
140   letter-spacing:.1em;color:var(--ink-mute);margin-bottom:5px}
141 .ents{display:flex;gap:5px;flex-wrap:wrap;margin-bottom:11px}
142 .ent{font-family:var(--deva);font-size:.92rem;padding:3px 10px;border-radius:6px;
143   background:var(--accent-soft);color:var(--accent)}
144 .tone{display:inline-flex;align-items:center;font-family:var(--mono);font-size:.68rem;
145   text-transform:uppercase;letter-spacing:.08em;padding:3px 10px;border-radius:99px}
146 .tone.Positive{background:var(--pos-bg);color:var(--pos)}
147 .tone.Negative{background:var(--neg-bg);color:var(--neg)}
148 .tone.Neutral{background:var(--neu-bg);color:var(--neu)}
149 .stack{display:flex;height:14px;border-radius:4px;overflow:hidden;margin-top:12px}
150 .stack i{display:block;height:100%}
151 .en{font-size:.92rem;color:var(--ink-soft);border-left:2px solid var(--accent);
152   padding-left:12px;margin-top:10px}
153 footer{padding-block:0 40px;color:var(--ink-mute);font-size:.8rem}
154 @media(prefers-reduced-motion:reduce){*{transition:none!important}}
155 /* ---- pasted-text additions ---- */
156 textarea{font-family:var(--deva);font-size:1.05rem;line-height:1.8;width:100%;
157   padding:11px 14px;border:1px solid var(--line);border-radius:8px;
158   background:var(--bg);color:var(--ink);resize:vertical}
159 textarea:focus{outline:2px solid var(--accent);outline-offset:1px;border-color:transparent}
160 #mword{font-size:1.25rem;line-height:1.6}
161 .fld{margin-top:14px}
162 .fld:first-child{margin-top:0}
163 .actions{display:flex;gap:8px;align-items:center;flex-wrap:wrap;margin-top:14px}
164 .btn{font-family:var(--sans);font-size:.9rem;font-weight:500;padding:8px 16px;
165   border-radius:8px;border:1px solid var(--line);background:var(--card);
166   color:var(--ink);cursor:pointer}
167 .btn:hover{border-color:var(--accent);color:var(--accent)}
168 .btn.primary{background:var(--accent);border-color:var(--accent);color:#fff}
169 .btn.primary:hover{filter:brightness(1.08);color:#fff}
170 .btn.disabled{opacity:.55;cursor:default}
171 .status{font-size:.85rem;color:var(--ink-soft)}
172 .status.err{color:var(--neg)}
173 .mode{display:inline-flex;align-items:center;gap:6px;font-family:var(--mono);
174   font-size:.66rem;text-transform:uppercase;letter-spacing:.08em;padding:3px 10px;
175   border-radius:99px;background:var(--neu-bg);color:var(--ink-soft)}
176 .mode.full{background:var(--pos-bg);color:var(--pos)}
177 .mode::before{content:"";width:7px;height:7px;border-radius:50%;background:currentColor}
178 .src{margin-left:auto;font-family:var(--mono);font-size:.6rem;letter-spacing:.06em;
179   text-transform:uppercase;color:var(--ink-mute)}
180 .cmp{font-size:.8rem;color:var(--ink-mute);margin-top:10px}
181 .mtable{width:100%;border-collapse:collapse;margin-top:16px;font-size:.9rem}
182 .mtable th{font-family:var(--mono);font-size:.62rem;text-transform:uppercase;
183   letter-spacing:.08em;color:var(--ink-mute);text-align:left;font-weight:500;
184   padding:6px 8px;border-bottom:1px solid var(--line)}
185 .mtable td{padding:5px 8px;border-bottom:1px solid var(--line);vertical-align:top}
186 .mtable td.d{font-family:var(--deva)}
187 .mtable td.r{font-family:var(--deva);color:var(--accent);font-weight:500}
188 .mtable td.f{font-family:var(--mono);font-size:.68rem;color:var(--ink-mute)}
189 .mtable td.cmp{font-family:var(--sans);margin:0}
190 .tblwrap{max-height:360px;overflow:auto}
191 .stats{display:flex;gap:22px;flex-wrap:wrap;margin-top:16px}
192 .stat b{display:block;font-family:var(--mono);font-size:1.3rem;font-weight:500;color:var(--accent)}
193 .stat span{font-size:.78rem;color:var(--ink-mute)}
194 details.add{margin-top:16px;border-top:1px solid var(--line);padding-top:12px}
195 details.add summary{cursor:pointer;font-size:.9rem;color:var(--accent);font-weight:500}
196 .mine{color:var(--accent)}
197 .sentl{list-style:none;padding:0;margin:12px 0 0}
198 .sentl li{display:flex;gap:10px;align-items:baseline;padding:5px 0;
199   border-top:1px solid var(--line);font-family:var(--deva);font-size:.9rem;line-height:1.7}
200 .sentl .tone{flex:none}

```

```

201     code{font-family:var(--mono);font-size:.85em;background:var(--soft);padding:1px 5px;border-radius:4px}
202 </style>
203 </head>
204 <body>
205
206 <script id="payload" type="application/json">__DATA__</script>
207
208 <div class="wrap head">
209     <div class="title">
210         <span class="mr">मराठी</span>
211         <h1>News Intelligence</h1>
212         <span class="meta" id="meta"></span>
213     </div>
214     <p>Paste any Marathi news text &mdash; category, summary, entities, sentiment,
215         English translation and search come out.</p>
216
217     <div class="tabs" role="tablist">
218         <button class="tab" role="tab" id="t0" aria-controls="p0" aria-selected="true"><span
class="n">I</span>Morphology</button>
219         <button class="tab" role="tab" id="t1" aria-controls="p1" aria-selected="false"><span
class="n">C</span>Search</button>
220         <button class="tab" role="tab" id="t2" aria-controls="p2" aria-selected="false"><span class="n">D E
F G A</span>Full pipeline</button>
221     </div>
222 </div>
223
224 <div class="wrap panelwrap">
225
226     <div id="p0" role="tabpanel" aria-labelledby="t0">
227         <p class="lead">Marathi joins a whole phrase into one word. Type a word, or paste
228             a whole sentence or article &mdash; the analyser strips each word's suffix chain
229             back to its root, live.</p>
230         <div class="card">
231             <label class="fl" for="mword">Marathi word or text</label>
232             <textarea id="mword" rows="2" spellcheck="false">घरांमधून</textarea>
233             <div class="chips" id="mchips"></div>
234             <div id="mout"></div>
235         </div>
236         <div class="fams" id="fams"></div>
237     </div>
238
239     <div id="p1" role="tabpanel" aria-labelledby="t1" hidden>
240         <p class="lead">Two real search indexes over the same <span id="ndocs">400</span>
241             articles, built in your browser. The only difference is whether words pass
242             through the analyser first. Type or paste any query.</p>
243         <div class="card">
244             <label class="fl" for="q">Search query</label>
245             <input type="text" id="q" value="न्यायालय" spellcheck="false">
246             <div class="chips" id="qchips"></div>
247             <div class="vs">
248                 <div class="side a">
249                     <div class="side-h"><span class="nm">Without morphology</span><span class="cnt" id="c-
a">0</span></div>
250                     <div class="side-b" id="r-a"></div>
251                 </div>
252                 <div class="side b">
253                     <div class="side-h"><span class="nm">With morphology</span><span class="cnt" id="c-b">0</span></div>
254                     <div class="side-b" id="r-b"></div>
255                 </div>
256             </div>
257             <div class="note" id="qnote"></div>
258             <details class="add">
259                 <summary>Add your own articles to the index</summary>
260                 <div class="fld"><label class="fl" for="atitle">Headline</label>
261                     <input type="text" id="atitle" spellcheck="false"></div>
262                 <div class="fld"><label class="fl" for="atext">Marathi article text</label>
263                     <textarea id="atext" rows="5" spellcheck="false"></textarea></div>
264                 <div class="actions"><button class="btn primary" type="button" id="aadd">Add to both

```

```

indexes</button>
265     <span class="status" id="astatus"></span></div>
266 </details>
267 </div>
268 </div>
269
270 <div id="p2" role="tabpanel" aria-labelledby="t2" hidden>
271   <p class="lead">Paste any Marathi article, or start from an example, then press
272   <b>Analyse</b>. It runs in your browser; if the local model server is running
273   (<code>python app/server.py</code>) it uses the full models instead.</p>
274   <div class="card">
275     <div class="fld"><label class="fl" for="pick">Start from an example</label>
276     <select id="pick"></select></div>
277     <div class="fld"><label class="fl" for="ptitle">Headline (optional)</label>
278     <input type="text" id="ptitle" spellcheck="false"></div>
279     <div class="fld"><label class="fl" for="ptext">Marathi article text</label>
280     <textarea id="ptext" rows="8" spellcheck="false"></textarea></div>
281     <div class="actions">
282       <button class="btn primary" type="button" id="run">Analyse</button>
283       <button class="btn" type="button" id="clear">Clear</button>
284       <span class="mode" id="mode">In-browser models</span>
285       <span class="status" id="runnote"></span>
286     </div>
287   </div>
288   <div id="artview"></div>
289 </div>
290
291 </div>
292
293 <footer class="wrap">Data: XL-Sum Marathi (BBC Marathi). Models: L3Cube-MahaNLP, NLLB-200.
294 In-browser models are small students trained to imitate the full models; run
295 <code>python app/server.py</code> to analyse with the full models.</footer>
296
297 <script>
298 /*__ENGINE__*/
299 </script>
300 <script>
301 (function(){
302   "use strict";
303   const DATA = JSON.parse(document.getElementById("payload").textContent);
304   const E = makeEngine(DATA);
305   const AG = DATA.lite.agreement;
306   const SERVER = "http://127.0.0.1:8765";
307   const $ = s => document.querySelector(s);
308   const esc = s => String(s).replace(/[\<>"]/g,c=>({ "&": "&amp;", "<": "&lt;", ">": "&gt;", "\"": "&quot;" }
[c]));
309   const pct = v => (v*100).toFixed(1) + "%";
310   const hasDeva = t => /[ॐ-ॱ]/.test(t || "");
311
312   /* ---- tabs ---- */
313   const tabs = [...document.querySelectorAll(".tab")];
314   tabs.forEach(t => t.addEventListener("click", ()=>{
315     tabs.forEach(o=>{
316       const on = o === t;
317       o.setAttribute("aria-selected", String(on));
318       document.getElementById(o.getAttribute("aria-controls")).hidden = !on;
319     });
320   }));
321   $("#meta").textContent = DATA.meta.corpus_size.toLocaleString() + " articles processed";
322
323   /* ===== tab 1: morphology ===== */
324   const WORDS = ["घरांमधून", "शाळेतून", "न्यायालयात", "मुलांना", "पुण्यातील", "विद्यार्थ्यांसाठी", "जिकले", "मुंबईपासून"];
325   $("#mchips").innerHTML = WORDS.map(w=><button class="chip" type="button" data-w="${esc(w)}">${esc(w)}
</button>').join("");
326
327   function gloss(word){
328     const a = E.analyse(word);
329     const segs=[`<div class="mb root"><span class="seg">${esc(a.root)}</span><span
class="feat">root</span></div>`];

```

```

330     for(let i=a.stripped.length-1;i>=0;i--){
331         segs.push(`+</span>`);
332         segs.push(`

Marathi News Intelligence Pipeline · Source code



95 / 107


```

```

393     $("#docs").textContent = DOCS.length.toLocaleString();
394 }
395 reindex();
396
397 const QS = ["न्यायालय", "सरकार", "महिला", "शाळा", "पोलीस", "पाऊस"];
398 $("#qchips").innerHTML = QS.map(q=>'<button class="chip" type="button" data-q="${esc(q)}">${esc(q)}
</button>').join("");
399
400 function side(elId,cntId,hits){
401     $(cntId).textContent = hits.length;
402     $(elId).innerHTML = hits.length
403     ? hits.slice(0,10).map(([i,s])=>{
404         const d=DOCS[i];
405         return '<div class="res"><span class="t">${esc(d.title)}</span>
406             <span class="m">${d.mine?'<span class="mine">your article</span> · ':''}${esc(d.topic)} ·
score ${s.toFixed(2)}</span></div>';
407     }).join("")
408     : '<div class="none"><b>0</b>No documents matched.</div>';
409 }
410 function runSearch(q){
411     q=(q|| "").trim();
412     const ha = q?E.search(iA,q,50):[], hb = q?E.search(iB,q,50):[];
413     side("#r-a", "#c-a", ha); side("#r-b", "#c-b", hb);
414     const d = hb.length-ha.length;
415     let v;
416     if(!q) v="Type or paste a query above.";
417     else if(!hasDeva(q)) v="Try a Marathi (Devanagari) query &mdash; the analyser only changes Marathi
words.";
418     else if(d>0) v=`Morphology found <b>${d}</b> more</b> article${d===1?"":"s"} the plain index cannot
reach.`;
419     else if(d===0) v=`<b>No difference for this query</b> &mdash; the articles happen to use it in the
same form.`;
420     else v=`Plain found ${-d} more here &mdash; over-stemming can merge distinct words.`;
421     $("#qnote").innerHTML = v;
422     document.querySelectorAll("#qchips .chip").forEach(c=>
423         c.setAttribute("aria-pressed", String(c.dataset.q===q)));
424 }
425 $("#q").addEventListener("input", e=>runSearch(e.target.value));
426 $("#qchips").addEventListener("click", e=>{
427     const b=e.target.closest(".chip"); if(!b) return;
428     $("#q").value=b.dataset.q; runSearch(b.dataset.q);
429 });
430 runSearch($("#q").value);
431
432 $("#aadd").addEventListener("click", ()=>{
433     const title = $("#atitle").value.trim(), text = $("#atext").value.trim();
434     if(!text){ $("#astatus").textContent = "Paste some article text first."; return; }
435     const scores = E.topic(title, text);
436     const topic = Object.entries(scores).sort((a,b)=>b[1]-a[1])[0][0];
437     DOCS.push({title: title || text.slice(0,70) + (text.length>70?"...":""), summary: text, topic, mine:
true});
438     reindex();
439     const mine = DOCS.filter(d=>d.mine).length;
440     $("#astatus").textContent = `Added as ${topic}. Your articles in the index: ${mine}.`;
441     $("#atitle").value = ""; $("#atext").value = "";
442     runSearch($("#q").value);
443 });
444
445 /* ===== local model server ===== */
446 let server = false;
447 async function detectServer(){
448     try{
449         const r = await fetch(SERVER + "/api/health", {signal: AbortSignal.timeout(900)});
450         server = r.ok && (await r.json()).ok === true;
451     }catch(_){ server = false; }
452     const m = $("#mode");
453     m.textContent = server ? "Full models · local server" : "In-browser models";
454     m.classList.toggle("full", server);
455     return server;

```

```

456 }
457 async function post(path, body){
458     const r = await fetch(SERVER + path, {method:"POST",
459         headers:{"Content-Type":"application/json"}, body: JSON.stringify(body)});
460     const j = await r.json();
461     if(!r.ok || j.error) throw new Error(j.error || ("HTTP " + r.status));
462     return j;
463 }
464
465 /* ===== in-browser translation (NLLB-200) ===== */
466 // The worker holds no static import, so it always starts; it then fetches
467 // transformers.js from the first CDN that answers. Some networks (several
468 // Indian mobile carriers, for one) block cdn.jsdelivr.net outright, so the
469 // same file is tried on unpkg and on jsDelivr's alternate hostnames, and the
470 // ONNX runtime's WebAssembly is loaded from whichever CDN worked -- by
471 // default transformers.js would fetch it from cdn.jsdelivr.net regardless.
472 const WORKER_SRC = `
473     const PKG = "@huggingface/transformers@3.8.1/dist/";
474     const CDNS = ["https://cdn.jsdelivr.net/npm/", "https://unpkg.com/",
475         "https://fastly.jsdelivr.net/npm/", "https://gcore.jsdelivr.net/npm/"];
476     const MODEL = "Xenova/nllb-200-distilled-600M";
477     let lib = null, tr = null;
478
479     const withTimeout = (p, ms) => Promise.race([p,
480         new Promise( (_, rej) => setTimeout(() => rej(new Error("timed out")), ms))]);
481
482     async function loadLibrary() {
483         const tried = [];
484         for (const cdn of CDNS) {
485             const base = cdn + PKG, host = new URL(cdn).host;
486             self.postMessage({ type: "status", text: "Loading the translation program from " + host + "..." });
487             try {
488                 const m = await withTimeout(import(base + "transformers.min.js"), 20000);
489                 m.env.allowLocalModels = false;
490                 m.env.backends.onnx.wasm.wasmPaths = base;
491                 return m;
492             } catch (err) { tried.push(host); }
493         }
494         const e = new Error("Could not load the translation program – this network blocks " +
495             tried.join(", ") + ". Try another network (e.g. Wi-Fi), or use “Show saved translation” " +
496             "for the examples, or run python app/server.py.");
497         e.stage = "program";
498         throw e;
499     }
500
501     self.onmessage = async (e) => {
502         const { texts } = e.data;
503         try {
504             if (!lib) lib = await loadLibrary();
505             if (!tr) {
506                 const files = {};
507                 try {
508                     tr = await lib.pipeline("translation", MODEL, {
509                         dtype: "q8",
510                         progress_callback: (p) => {
511                             if (p.status === "progress" && p.total) {
512                                 files[p.file] = [p.loaded, p.total];
513                                 let l = 0, t = 0;
514                                 for (const [a, b] of Object.values(files)) { l += a; t += b; }
515                                 self.postMessage({ type: "download", loaded: l, total: t });
516                             }
517                         },
518                     });
519                 } catch (err) {
520                     const m = new Error("Could not download the translation model from huggingface.co (" +
521                         String((err && err.message) || err) + "). Check the connection and try again, or run " +
522                         "python app/server.py.");
523                     m.stage = "model";
524                     throw m;

```

```

525     }
526 }
527 const out = [];
528 for (let i = 0; i < texts.length; i++) {
529     self.postMessage({ type: "step", i, n: texts.length });
530     const r = await tr(texts[i], { src_lang: "mar_Deva", tgt_lang: "eng_Latn", max_new_tokens: 256
});
531     out.push(r[0].translation_text);
532 }
533 self.postMessage({ type: "done", out });
534 } catch (err) {
535     self.postMessage({ type: "error", stage: err.stage || "translate",
536         message: String((err && err.message) || err) });
537 }
538 };`;
539 let worker = null;
540 function browserTranslate(texts, onStatus){
541     return new Promise((resolve, reject)=>{
542         if(!worker){
543             const url = URL.createObjectURL(new Blob([WORKER_SRC], {type:"text/javascript"}));
544             worker = new Worker(url, {type:"module"});
545         }
546         worker.onmessage = e=>{
547             const m = e.data;
548             if(m.type==="status") onStatus(m.text);
549             else if(m.type==="download") onStatus(`Downloading the translation model (once, then cached):
550 ${m.loaded/1e6}.toFixed(0)} / ${m.total/1e6}.toFixed(0)} MB`);
551             else if(m.type==="step") onStatus(`Translating sentence ${m.i+1} of ${m.n}...`);
552             else if(m.type==="done") resolve(m.out);
553             else if(m.type==="error") reject(new Error(m.message));
554         };
555         worker.onerror = e=>{ worker = null;
556             reject(new Error("The browser could not start the translator (" + (e.message || "worker error")
+
557             "). Try Chrome or Edge, or run python app/server.py.")); };
558         worker.postMessage({texts});
559     });
560 }
561 /* ===== tab 3: full pipeline ===== */
562 const SHOW = DATA.showcase;
563 $("#pick").innerHTML = `<option value="">- paste your own below -</option>` + SHOW.map((d,i)=>
564     `<option value="${i}">${esc(d.topic)} - ${esc(d.title.slice(0,70))}</option>`).join("");
565
566 let current = null;    // {result, mode, example}
567
568 function stageHead(badge, name, src){
569     return `<div class="stage-h"><span class="bdg">${badge}</span><span class="stage-n">${name}</span>${src?`<span class="src">${src}</span>`:""}</div>`;
570 }
571 const liteTag = (key, what) => `in-browser · ${pct(AG[key])} ${what}`;
572
573 function render(){
574     const {result: d, mode, example: ex} = current;
575     const full = mode === "full";
576     const top = Object.entries(d.topic_scores||{}).sort((a,b)=>b[1]-a[1]).slice(0,3);
577     const dist = d.sentiment_distribution||{};
578     const tot = Math.max(Object.values(dist).reduce((a,b)=>a+b,0),1);
579     const ORDER = ["Person", "Organization", "Location", "Date", "Designation", "Time", "Measure"];
580     const ents = ORDER.filter(k=>(d.entities||{})[k]&&d.entities[k].length);
581     const same = ex && d.summary === ex.summary;
582
583     let cmpTopic = "";
584     if(!full && ex){
585         const t = Object.entries(ex.topic_scores||{}).sort((a,b)=>b[1]-a[1])[0];
586         if(t) cmpTopic = `<div class="cmp">Full model on this article: <b>${esc(t[0])}</b> (${pct(t[1])}).
</div>`;
587     }
588

```

```

589     const sentList = (d.sentence_sentiment||[]).length
590     ? `<ul class="sentl">${d.sentence_sentiment.map(s=>`<li><span class="tone
${esc(s.label)}">${esc(s.label)}</span><span>${esc(s.text)}</span></li>`).join("")}</ul>` : "";
591
592     let trBody;
593     if(d.title_en || d.summary_en){
594         trBody = ` ${d.title_en?`<div style="font-weight:500">${esc(d.title_en)}</div>`: ""}
595         <div class="en">${esc(d.summary_en|| "")}</div>
596         <div class="cmp">${esc(d.translation_source|| "")}</div>`;
597     } else {
598         trBody = `<div class="actions" style="margin-top:0">
599             <button class="btn" type="button" id="trbtn">Translate to English</button>
600             <span class="status" id="trstatus">${full?"Uses NLLB-200 on the local server."
601             : "Runs NLLB-200 in your browser. The first time it downloads the model (~900 MB, needs
internet); after that it is cached."}</span>
602         </div>`;
603     }
604
605     $("#artview").innerHTML = `
606         <div class="stage">${stageHead(".", "Input article")}</div>
607         <div class="stage-b">${d.title?`<div class="deva" style="font-size:1.05rem;font-
weight:500">${esc(d.title)}</div>`: ""}
608         <div class="cmp">${d.tokens} words · ${pct(d.inflected_ratio||0)} carry a suffix the analyser
(I) strips</div></div></div>
609
610         <div class="stage">${stageHead("D", "Category", full?"full
model":liteTag("topic_top1_agreement", "agreement"))}</div>
611         <div class="stage-b">${top.map([[k,v],j])=>`
612             <div style="margin-bottom:${j===top.length-1?0:9}px">
613                 <div class="kv"><span>${j===0?` style="font-weight:500"': ''}>${esc(k)}</span><span
class="v">${pct(v)}</span></div>
614                 <div class="bar"><i style="width:${(v*100).toFixed(1)}%;opacity:${j===0?1:.35}"></i></div>
615                 </div>`).join("")}</div></div>
616
617         <div class="stage">${stageHead("E", "Summary", "TextRank · same algorithm as Python")}</div>
618         <div class="stage-b"><div class="deva" style="font-size:.96rem">${d.summary?
esc(d.summary):`<span style="color:var(--ink-mute)">Too short to summarise &mdash; paste at least a few
sentences.</span>`}</div></div></div>
619
620         <div class="stage">${stageHead("G", "Entities", full?"full model":`in-browser · F1
${AG.ner_f1.toFixed(2)} vs full model`)}</div>
621         <div class="stage-b">${ents.map(k=>`<div class="lab">${esc(k)}</div>
622         <div class="ents">${d.entities[k].slice(0,8).map(v=>`<span class="ent">${esc(v)}
</span>`).join("")}</div>`).join("")
623         || `<span style="color:var(--ink-mute);font-size:.9rem">None found.</span>`}</div></div>
624
625         <div class="stage">${stageHead("F", "Sentiment", full?"full
model":liteTag("sentiment_document_agreement", "agreement"))}</div>
626         <div class="stage-b">
627             <span class="tone ${esc(d.sentence_sentiment)}">${esc(d.sentence_sentiment)}</span>
628             <span style="font-family:var(--mono);font-size:.7rem;color:var(--ink-mute);margin-
left:8px">polarity ${d.polarity||0}.toFixed(2)}</span>
629             <div class="stack">
630                 <i style="width:${(dist.Positive||0)/tot*100}%;background:var(--pos)"></i>
631                 <i style="width:${(dist.Neutral||0)/tot*100}%;background:var(--neu)"></i>
632                 <i style="width:${(dist.Negative||0)/tot*100}%;background:var(--neg)"></i>
633             </div>
634             <div style="font-family:var(--mono);font-size:.66rem;color:var(--ink-mute);margin-top:6px">
635                 ${dist.Positive||0} positive · ${dist.Neutral||0} neutral · ${dist.Negative||0} negative
sentences in the summary</div>
636             ${sentList}
637         </div></div>
638
639         <div class="stage">${stageHead("A", "English translation", full?"NLLB-200 · local server":"NLLB-
200")}</div>
640         <div class="stage-b" style="font-size:.95rem;color:var(--ink-soft)">${trBody}</div></div>`;
641
642     const btn = $("#trbtn");
643     if(btn) btn.addEventListener("click", ()=>translate(btn));

```

```

644     if(!full && same && ex.summary_en && !d.title_en && !d.summary_en){
645         // An unedited example: the saved NLLB output for exactly this summary.
646         const s = $("#trstatus");
647         if(s) s.insertAdjacentHTML("afterend", `<button class="btn" type="button" id="trsaved">Show saved
translation</button>`);
648         const b2 = $("#trsaved");
649         if(b2) b2.addEventListener("click", ()=>{
650             d.title_en = ex.title_en; d.summary_en = ex.summary_en;
651             d.translation_source = "Saved NLLB-200 output for this example.";
652             render();
653         });
654     }
655 }
656
657 async function translate(btn){
658     const d = current.result, st = $("#trstatus");
659     const set = (t, err) => { if(st){ st.textContent = t; st.classList.toggle("err", !!err); } };
660     const sents = E.sentTokenize(d.summary).slice(0,12);
661     const texts = [...(d.title?[d.title]:[]), ...sents];
662     if(!texts.length){ set("Nothing to translate yet.", true); return; }
663     btn.disabled = true;
664     try{
665         let out, source;
666         if(await detectServer()){
667             set("Translating on the local server...");
668             out = (await post("/api/translate", {texts})).translations;
669             source = "NLLB-200 on the local server.";
670         } else {
671             set("Loading the translation model...");
672             out = await browserTranslate(texts, set);
673             source = "NLLB-200, run in your browser.";
674         }
675         if(current.result !== d) return;
676         d.title_en = d.title ? out[0] : "";
677         d.summary_en = (d.title ? out.slice(1) : out).join(" ");
678         d.translation_source = source;
679         render();
680     }catch(err){
681         btn.disabled = false;
682         set("Translation failed: " + err.message, true);
683     }
684 }
685
686 async function analyse(){
687     const title = $("#ptitle").value.trim(), text = $("#ptext").value.trim();
688     const note = $("#runnote");
689     note.classList.remove("err");
690     if(!text){ note.textContent = "Paste some Marathi article text first."; note.classList.add("err");
return; }
691     if(!hasDeva(text)){ note.textContent = "That doesn't look like Marathi (Devanagari) text.";
note.classList.add("err"); return; }
692     const exIdx = $("#pick").value;
693     const ex = exIdx !== "" && SHOW[+exIdx].text === text && SHOW[+exIdx].title === title ? SHOW[+exIdx]
: null;
694     $("#run").disabled = true;
695     let result, mode = "lite";
696     try{
697         if(await detectServer()){
698             note.textContent = "Running the full models on the local server...";
699             try{
700                 const r = await post("/api/analyse", {title, text, translate:false});
701                 result = {title, topic_scores:r.topic_scores, summary:r.summary, entities:r.entities,
702                     sentiment:r.sentiment, polarity:r.polarity, sentiment_distribution:r.sentiment_distribution,
703                     tokens:r.tokens, inflected_ratio:r.inflected_ratio};
704                 mode = "full";
705             }catch(err){
706                 note.textContent = "Local server failed (" + err.message + "); used the in-browser models.";
707             }
708         }

```

```

709     if(!result){
710         result = E.pipeline(title, text);
711         if(mode === "lite" && !note.textContent.startsWith("Local server")) note.textContent = "";
712     } else note.textContent = "";
713     current = {result, mode, example: ex};
714     render();
715 } finally {
716     $("#run").disabled = false;
717 }
718 }
719
720 function loadExample(i){
721     const d = SHOW[i];
722     $("#ptitle").value = d.title; $("#ptext").value = d.text;
723     analyse();
724 }
725 $("#pick").addEventListener("change", e=>{ if(e.target.value !== "") loadExample(+e.target.value); });
726 $("#run").addEventListener("click", analyse);
727 $("#clear").addEventListener("click", ()=>{
728     $("#ptitle").value = ""; $("#ptext").value = ""; $("#pick").value = "";
729     $("#artview").innerHTML = ""; $("#runnote").textContent = ""; current = null;
730     $("#ptext").focus();
731 });
732 $("#ptext").addEventListener("keydown", e=>{ if((e.metaKey||e.ctrlKey) && e.key==="Enter") analyse();
733 });
734 $("#pick").value = "0";
735 loadExample(0);
736 }());
737 </script>
738 </body>
739 </html>

```

Tests — tests/

tests/test_pipeline.py

391 lines

51 fast tests plus 8 that load the models.

```
1  # -*- coding: utf-8 -*-
2  """
3  Unit tests.
4
5  python -m pytest tests/ -v
6  python tests/test_pipeline.py      # runs without pytest too
7
8  The model-free components (morphology, BM25, TextRank, ROUGE) are tested
9  directly. Tests that need a transformer are marked 'slow' and skipped unless
10 RUN_SLOW_TESTS=1, so the suite stays runnable on a machine with no GPU and no
11 model cache.
12 """
13 from __future__ import annotations
14
15 import os
16 import sys
17 from pathlib import Path
18
19 import pytest
20
21 sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
22
23 from src.morph import (Analysis, akshara_len, analyse, normalize_tokens, # noqa: E402
24                        sent_tokenize, stem, tokenize)
25 from src.retrieve import BM25Index, Hit, reciprocal_rank_fusion # noqa: E402
26 from src.summarize import summarize # noqa: E402
27
28 SLOW = pytest.mark.skipif(os.environ.get("RUN_SLOW_TESTS") != "1",
29                           reason="set RUN_SLOW_TESTS=1 to run model-backed tests")
30
31
32 # ----- morphology
33 class TestMorphology:
34     def test_akshara_length_ignores_matras(self):
35         # घरा is 3 code points but 2 aksharas (घ + रा with a vowel sign).
36         assert len("घरा") == 3
37         assert akshara_len("घरा") == 2
38         assert akshara_len("घर") == 2
39
40     @pytest.mark.parametrize("surface,expected", [
41         ("घरात", "घर"), ("घरातून", "घर"), ("घरांमधून", "घर"),
42         ("घराच्या", "घर"), ("घरासाठी", "घर"),
43         ("मुंबईपासून", "मुंबई"), ("बोलून", "बोल"),
44     ])
45     def test_known_analyses(self, surface, expected):
46         assert stem(surface) == expected
47
48     @pytest.mark.parametrize("group", [
49         ["घर", "घरात", "घरातून", "घरांमधून", "घराच्या", "घरासाठी", "घरे"],
50         ["शाळा", "शाळेत", "शाळेतून", "शाळांना"],
51         ["नदी", "नद्या", "नदीत"],
52         ["सरकार", "सरकारने", "सरकारच्या", "सरकारला"],
53         ["मुंबई", "मुंबईत", "मुंबईतील", "मुंबईपासून"],
54     ])
55     def test_inflectional_family_shares_one_root(self, group):
56         """The whole point of the analyser: one lemma, one index term."""
57         assert len({stem(w) for w in group}) == 1
58
59     def test_no_overstripping_of_lexical_suffixes(self):
```

```

60     # कर is a real derivational suffix, but सरकार is not सर + कर.
61     assert stem("सरकारने") == "सरकार"
62
63     def test_stem_never_emptyies_a_word(self):
64         for w in ["ला", "ने", "त", "च", "आहे", "घर", "अ"]:
65             assert stem(w), f"{w} stemmed to empty string"
66
67     def test_features_are_reported(self):
68         a = analyse("घरांमधून")
69         assert isinstance(a, Analysis)
70         assert a.is_inflected
71         assert "ABL" in a.features
72
73     def test_uninflected_word_is_unchanged(self):
74         a = analyse("घर")
75         assert a.root == "घर" and not a.is_inflected
76
77     def test_tokenizer_handles_mixed_scripts(self):
78         toks = tokenize("भारत India 2024 मध्ये")
79         assert toks == ["भारत", "India", "2024", "मध्ये"]
80
81     def test_sentence_split_on_danda(self):
82         assert len(sent_tokenize("हे पहिले वाक्य आहे। हे दुसरे वाक्य आहे।")) == 2
83
84     def test_stopwords_removed(self):
85         assert "आणि" not in normalize_tokens("शाळा आणि महाविद्यालय")
86
87     def test_morph_can_be_disabled(self):
88         """The ablation switch must actually change the output."""
89         assert normalize_tokens("घरात", use_morph=False) == ["घरात"]
90         assert normalize_tokens("घरात", use_morph=True) == ["घर"]
91
92     def test_empty_and_none_input(self):
93         assert tokenize("") == [] and tokenize(None) == []
94         assert normalize_tokens("") == []
95
96
97     # ----- retrieval
98     DOCS = [
99         "पुण्यातील शाळांमध्ये नवीन शैक्षणिक धोरण लागू करण्यात आले आहे.",
100         "भारतीय क्रिकेट संघाने कसोटी सामन्यात विजय मिळवला.",
101         "मुंबईत मेट्रो प्रकल्पाचे काम वेगाने सुरु आहे.",
102         "शाळेत विद्यार्थ्यांसाठी नवीन अभ्यासक्रम सुरु झाला.",
103     ]
104
105
106     class TestRetrieval:
107         def test_morphology_finds_what_plain_tokens_miss(self):
108             """
109             The corpus says शाळांमध्ये and शाळेत; the query says शाळा. A plain
110             index cannot match either, a normalised one matches both.
111             """
112             plain = BM25Index(use_morph=False).build(DOCS)
113             morph = BM25Index(use_morph=True).build(DOCS)
114             assert len(plain.search("शाळा", top_k=5)) == 0
115             assert len(morph.search("शाळा", top_k=5)) == 2
116
117         def test_morphology_shrinks_the_vocabulary(self):
118             plain = BM25Index(use_morph=False).build(DOCS)
119             morph = BM25Index(use_morph=True).build(DOCS)
120             assert len(morph.postings) < len(plain.postings)
121
122         def test_hits_are_rank_ordered(self):
123             idx = BM25Index().build(DOCS)
124             hits = idx.search("नवीन", top_k=5)
125             assert [h.rank for h in hits] == list(range(len(hits)))
126             assert all(a.score >= b.score for a, b in zip(hits, hits[1:]))
127
128         def test_empty_query_and_empty_index(self):

```

```

129     assert BM25Index().build(DOCS).search("", top_k=5) == []
130     assert BM25Index().build([]).search("शाळा", top_k=5) == []
131
132     def test_unknown_term_returns_nothing(self):
133         assert BM25Index().build(DOCS).search("झेन्ना", top_k=5) == []
134
135     def test_rrf_prefers_documents_both_runs_agree_on(self):
136         run_a = [Hit(doc_id=1, score=9.0, rank=0), Hit(doc_id=2, score=8.0, rank=1)]
137         run_b = [Hit(doc_id=2, score=0.9, rank=0), Hit(doc_id=3, score=0.8, rank=1)]
138         fused = reciprocal_rank_fusion([run_a, run_b], top_k=3)
139         assert fused[0].doc_id == 2          # only doc ranked highly by both
140
141     def test_bm25_index_round_trips(self, tmp_path):
142         idx = BM25Index().build(DOCS)
143         p = tmp_path / "idx.pkl"
144         idx.save(p)
145         assert BM25Index.load(p).search("शाळा", 5) == idx.search("शाळा", 5)
146
147
148 # ----- summarization
149 class TestSummarization:
150     TEXT = ( "पुण्यात आज शिक्षण परिषद पार पडली। "
151             "परिषदेत नवीन शैक्षणिक धोरणावर चर्चा झाली। "
152             "शिक्षणमंत्र्यांनी विद्यार्थ्यांसाठी नवीन योजना जाहीर केली। "
153             "अनेक शाळांचे मुख्याध्यापक उपस्थित होते। "
154             "पुढील वर्षापासून धोरण लागू होणार आहे।")
155
156     def test_returns_requested_number_of_sentences(self):
157         assert len(summarize(self.TEXT, k=2).sentences) == 2
158
159     def test_summary_sentences_come_from_the_source(self):
160         s = summarize(self.TEXT, k=2)
161         for sent in s.sentences:
162             assert sent in self.TEXT
163
164     def test_original_order_is_preserved(self):
165         assert summarize(self.TEXT, k=3).indices == sorted(summarize(self.TEXT, k=3).indices)
166
167     def test_lead_baseline_takes_the_opening(self):
168         assert summarize(self.TEXT, k=1, method="lead").indices == [0]
169
170     def test_short_input_is_returned_whole(self):
171         one = "फक्त एकच वाक्य आहे।"
172         assert summarize(one, k=3).sentences == [one]
173
174     def test_empty_input(self):
175         assert summarize("", k=3).sentences == []
176
177     @SLOW
178     def test_embedding_method_runs(self):
179         assert len(summarize(self.TEXT, k=2, method="embedding").sentences) == 2
180
181
182 # ----- rouge
183 class TestRouge:
184     def setup_method(self):
185         sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "scripts"))
186         import evaluate
187         self.ev = evaluate
188
189     def test_identical_text_scores_one(self):
190         r = self.ev.rouge_all("शाळेत नवीन अभ्यासक्रम", "शाळेत नवीन अभ्यासक्रम", False)
191         assert r["rouge1"] == pytest.approx(1.0)
192         assert r["rougeL"] == pytest.approx(1.0)
193
194     def test_disjoint_text_scores_zero(self):
195         assert self.ev.rouge_all("क्रिकेट सामना", "शैक्षणिक धोरण", False)["rouge1"] == 0.0
196
197     def test_morphology_credits_inflectional_variants(self):

```

```

198         """
199         Same content, different case endings: raw ROUGE-1 misses it, the
200         normalised variant catches it. This is why the evaluation reports both.
201         """
202         pred, gold = "शाळेत विद्यार्थी", "शाळांमध्ये विद्यार्थ्यांना"
203         assert self.ev.rouge_all(pred, gold, False)["rouge1"] == 0.0
204         assert self.ev.rouge_all(pred, gold, True)["rouge1"] > 0.0
205
206     def test_empty_inputs_do_not_crash(self):
207         assert self.ev.rouge_all("", "काहीतरी", False)["rouge1"] == 0.0
208
209
210     # ----- aggregation
211     class TestAliasMerging:
212         def setup_method(self):
213             from src.ner import merge_aliases
214             self.merge = merge_aliases
215
216         def test_short_name_folds_into_full_name(self):
217             from collections import Counter
218             out = self.merge(Counter({"नरेंद्र मोदी": 70, "मोदी": 55}))
219             assert out["नरेंद्र मोदी"] == 125
220             assert "मोदी" not in out
221
222         def test_merges_when_forms_are_equally_common(self):
223             from collections import Counter
224             out = self.merge(Counter({"डोनाल्ड ट्रंप": 39, "ट्रंप": 40}))
225             assert out["डोनाल्ड ट्रंप"] == 79
226
227         def test_common_surname_not_absorbed_by_rare_full_name(self):
228             from collections import Counter
229             out = self.merge(Counter({"पवार": 100, "अजित पवार": 2}))
230             assert out["पवार"] == 100 and out["अजित पवार"] == 2
231
232         def test_two_full_names_are_never_merged(self):
233             from collections import Counter
234             counts = Counter({"अजित पवार": 30, "शरद पवार": 62})
235             assert self.merge(counts) == counts
236
237         def test_unrelated_names_untouched(self):
238             from collections import Counter
239             counts = Counter({"मुंबई": 10, "पुणे": 8})
240             assert self.merge(counts) == counts
241
242
243     # ----- in-browser students
244     LITE = Path(__file__).resolve().parent.parent / "app" / "lite_models.json"
245     NEEDS_LITE = pytest.mark.skipif(not LITE.exists(),
246                                     reason="run scripts/train_lite.py to build app/lite_models.json")
247
248
249     class TestLite:
250         def test_words_split_on_boundaries_and_keep_offsets(self):
251             from src.lite import words
252             text = "इंडो-तिबेटन पोलीस, मुंबईत."
253             toks = [text[s:e] for s, e in words(text)]
254             assert toks == ["इंडो", "तिबेटन", "पोलीस", "मुंबईत"]
255
256         def test_digit_shape_is_explicit(self):
257             from src.lite import _shape
258             assert _shape("2024") == "digit"
259             assert _shape("२०२४") == "digit"
260             assert _shape("BBC") == "latin"
261             assert _shape("मुंबई") == "deva"
262
263     @NEEDS_LITE
264     def test_topic_student_recognises_sport(self):
265         from src.lite import Lite
266         m = Lite.load()

```

```

267 p = m.topic("भारताचा विजय", "भारतीय क्रिकेट संघाने कसोटी सामन्यात इंग्लंडचा "
268 "नऊ गडी राखून पराभव केला. कर्णधाराने संघाचे कौतुक केले.")
269 assert max(p, key=p.get) == "Sports"
270 assert abs(sum(p.values()) - 1.0) < 1e-9
271
272 @NEEDS_LITE
273 def test_entity_spans_are_whole_words(self):
274     from src.lite import Lite, words
275     m = Lite.load()
276     text = "मुख्यमंत्री एकनाथ शिंदे यांनी सोमवारी मुंबईत घोषणा केली."
277     starts = {s for s, _ in words(text)}
278     ends = {e for _, e in words(text)}
279     for s, e, _ in m.entities(text):
280         assert s in starts and e in ends
281
282 @NEEDS_LITE
283 def test_pipeline_output_shape(self):
284     from src.lite import Lite, analyse
285     out = analyse(Lite.load(), "शीर्षक", "पहिले वाक्य इथे आहे. दुसरे वाक्य इथे आहे. "
286 "तिसरे वाक्य इथे आहे. चौथे वाक्य इथे आहे.")
287     assert set(out) >= {"topic_scores", "summary", "entities", "sentiment",
288 "polarity", "sentiment_distribution"}
289     assert out["sentiment"] in ("Positive", "Neutral", "Negative")
290
291
292 class TestSummaryTies:
293     def test_exact_ties_keep_document_order(self):
294         """Repeated sentences tie exactly; the earlier one must win, so the
295         page's JavaScript port (different float rounding) agrees."""
296         from src.summarize import summarize
297         s = "हा पहिला वेगळा मुद्दा आहे."
298         dup = "सरकारने नवीन योजना जाहीर केली आहे."
299         text = " ".join([dup, s, "आणखी एक वेगळे वाक्य आहे.", dup, "शेवटचे वेगळे वाक्य आहे."])
300         # The two copies (indices 0 and 3) have identical PageRank scores.
301         for k in (1, 2, 3):
302             idx = summarize(text, k=k).indices
303             assert not (3 in idx and 0 not in idx)
304
305
306 # ----- model server
307 class TestServer:
308     def test_health_and_cors_preflight(self):
309         import json as _json
310         import threading
311         import urllib.request
312         from http.server import ThreadingHTTPServer
313         from app.server import Handler
314
315         srv = ThreadingHTTPServer(("127.0.0.1", 0), Handler)
316         threading.Thread(target=srv.serve_forever, daemon=True).start()
317         base = f"http://127.0.0.1:{srv.server_address[1]}"
318         try:
319             with urllib.request.urlopen(base + "/api/health") as r:
320                 assert _json.loads(r.read())["ok"] is True
321                 assert r.headers["Access-Control-Allow-Origin"] == "*"
322             req = urllib.request.Request(base + "/api/analyse", method="OPTIONS")
323             with urllib.request.urlopen(req) as r:
324                 assert r.status == 204
325                 assert r.headers["Access-Control-Allow-Private-Network"] == "true"
326             req = urllib.request.Request(base + "/api/analyse", data=b'{"text": " {}"',
327                                         method="POST",
328                                         headers={"Content-Type": "application/json"})
329             try:
330                 urllib.request.urlopen(req)
331                 raise AssertionError("empty text should be rejected")
332             except urllib.error.HTTPError as err:
333                 assert err.code == 400
334         finally:
335             srv.shutdown()

```

```

336
337
338 # ----- model-backed
339 class TestModels:
340     @SLOW
341     def test_topic_classifier(self):
342         from src import categorize
343         p = categorize.classify(["भारतीय क्रिकेट संघाने सामना जिंकला."])[0]
344         assert p.label == "Sports" and p.score > 0.5
345
346     @SLOW
347     def test_ner_finds_person_and_location(self):
348         from src import ner
349         ents = ner.extract("मुख्यमंत्री एकनाथ शिंदे यांनी मुंबईत घोषणा केली.")
350         labels = {e.label for e in ents}
351         assert "Person" in labels and "Location" in labels
352
353     @SLOW
354     def test_ner_spans_are_whole_words(self):
355         """Regression: wordpiece splits used to yield पंतप्रधान for पंतप्रधान."""
356         from src import ner
357         text = "पंतप्रधान नरेंद्र मोदी यांनी भाषण केले."
358         for e in ner.extract(text):
359             assert not e.text[0] in "िीँ", f"span starts mid-akshara: {e.text}"
360
361     @SLOW
362     def test_entity_canonical_form_deinflects(self):
363         from src.ner import Entity
364         e = Entity(text="मुंबईत", label="Location", start=0, end=6, score=1.0)
365         assert e.canonical == "मुंबई"
366
367     @SLOW
368     def test_sentiment_polarity_direction(self):
369         from src import sentiment
370         pos = sentiment.analyse_document("भारताने शानदार विजय मिळवला.")
371         neg = sentiment.analyse_document("भीषण अपघातात अनेक लोक जखमी झाले.")
372         assert pos.polarity > neg.polarity
373
374     @SLOW
375     def test_embeddings_are_normalised(self):
376         import numpy as np
377         from src import models
378         v = models.embed(["शाळेत नवीन अभ्यासक्रम", "क्रिकेट सामना"])
379         assert v.shape[0] == 2
380         assert np.allclose(np.linalg.norm(v, axis=1), 1.0, atol=1e-3)
381
382     @SLOW
383     def test_translation_produces_english(self):
384         from src import translate
385         out = translate.translate("भारतीय क्रिकेट संघाने सामना जिंकला.")
386         assert out and out.isascii()
387
388
389 if __name__ == "__main__":
390     sys.exit(pytest.main([__file__, "-v", "--tb=short"]))

```